

# 循序渐进，学习开发 xv6-riscv

## 第 14 章 页表

---

汪辰



# 目录

01

地址空间

02

地址转换

03

RISC-V 页表

04

xv6 的内核地址空间设计

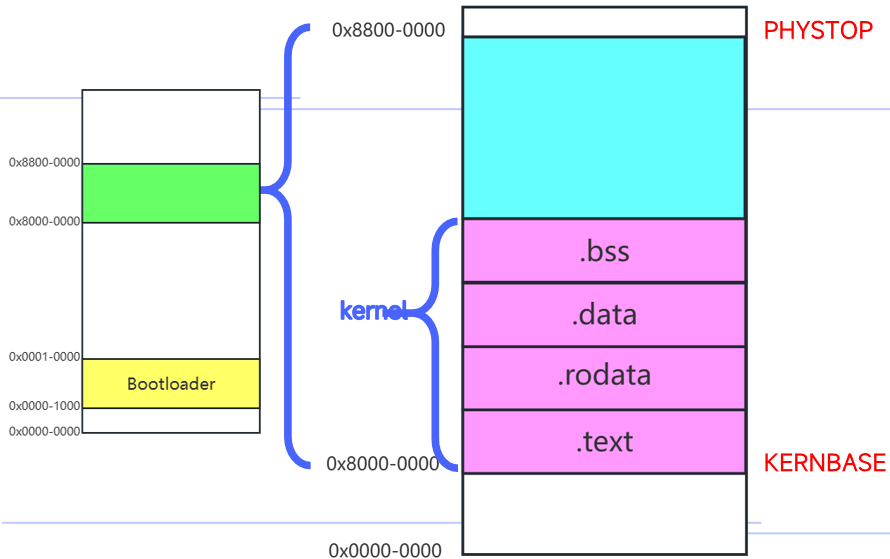


01

# 地址空间

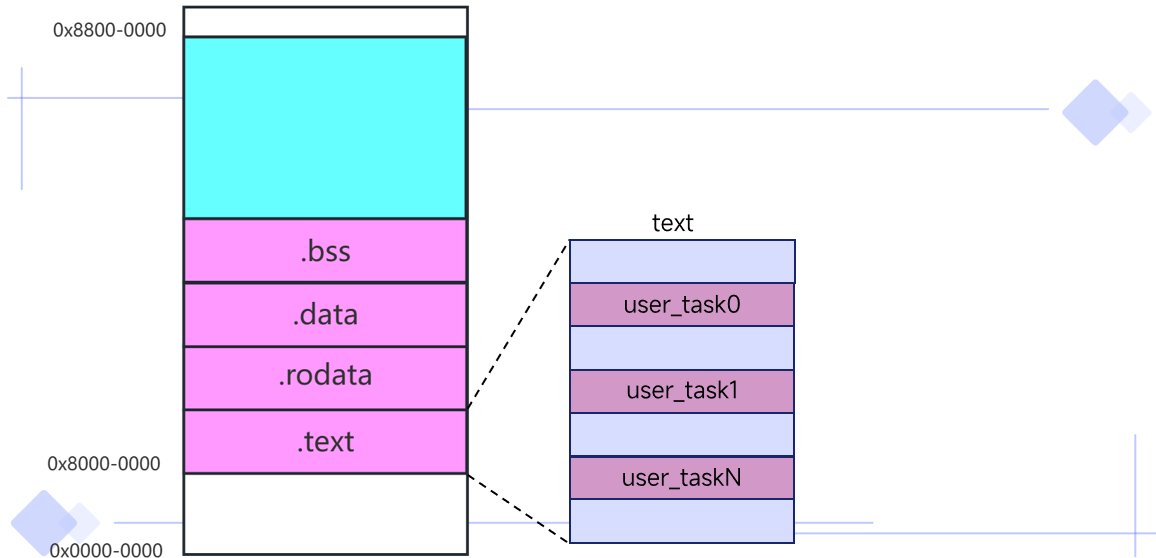


# 地址空间

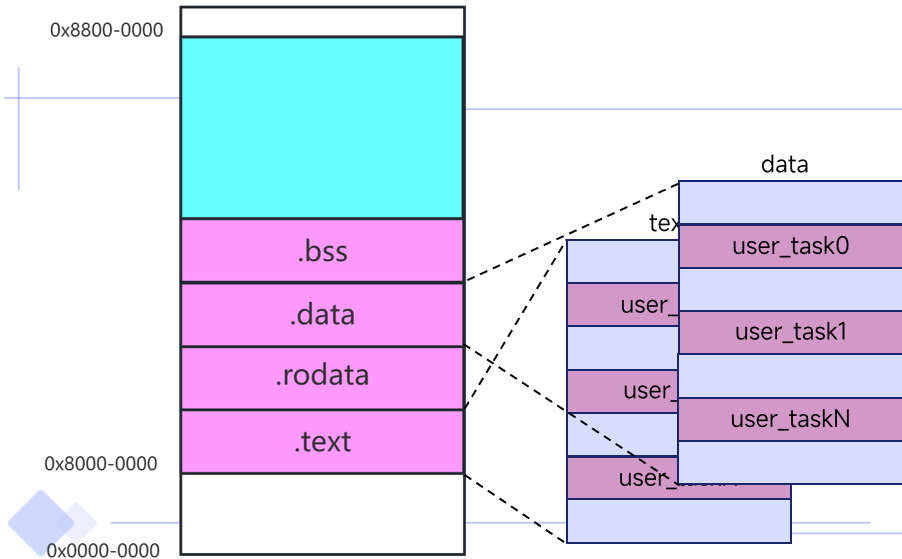




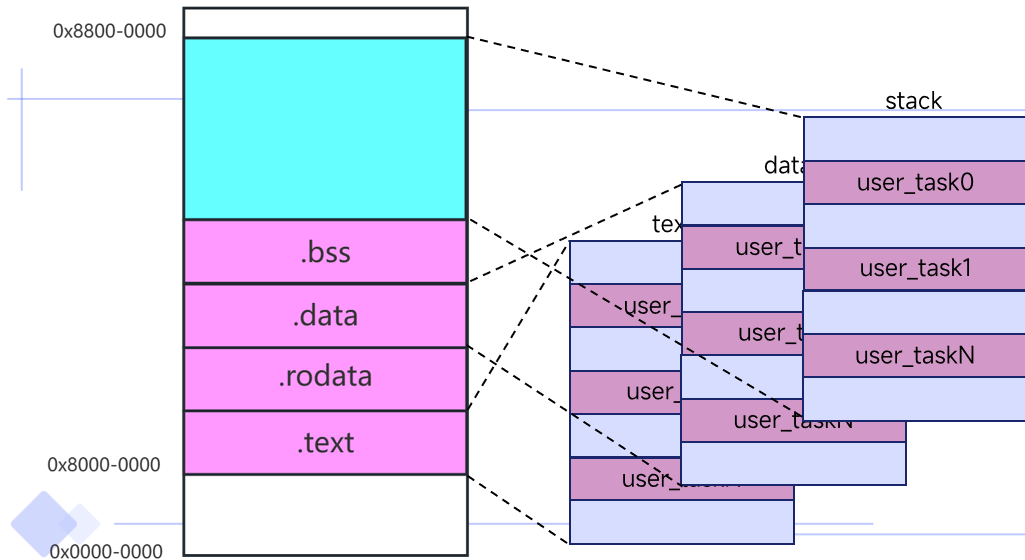
# 地址空间



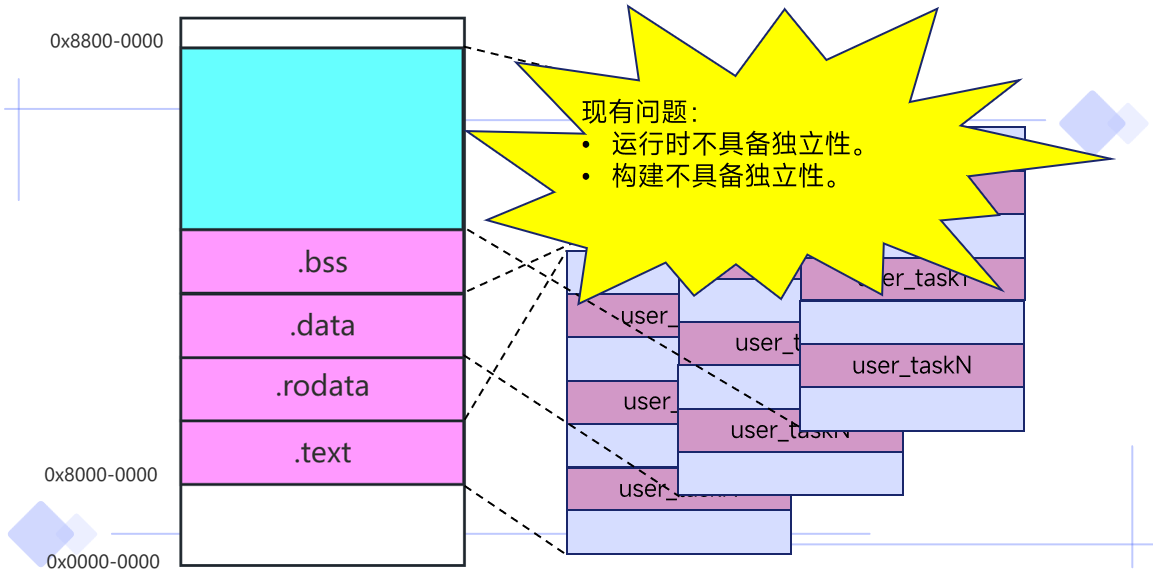
# 地址空间



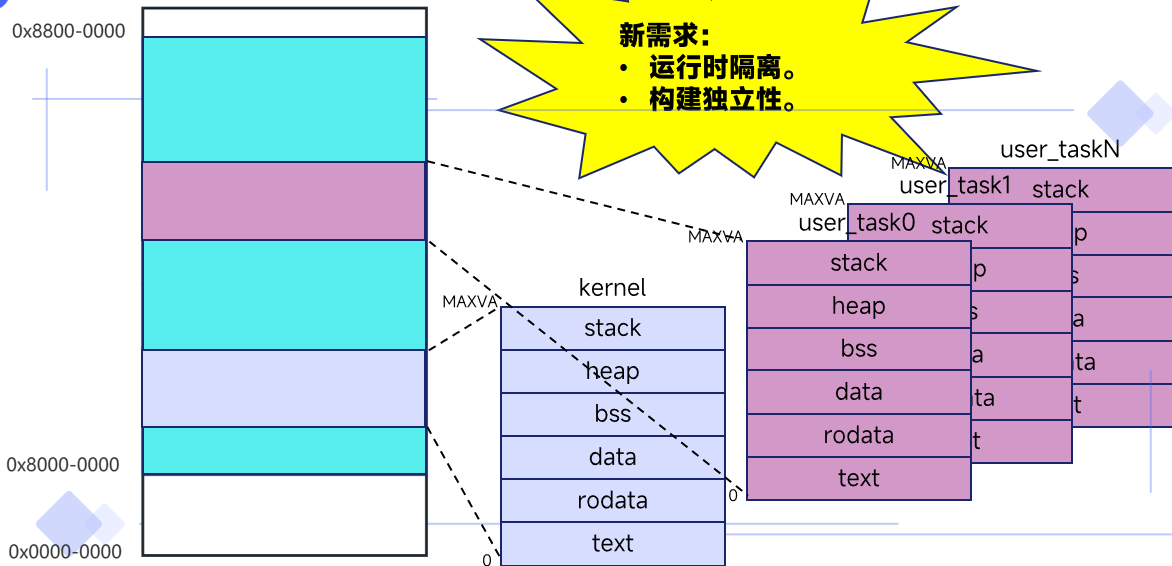
# 地址空间



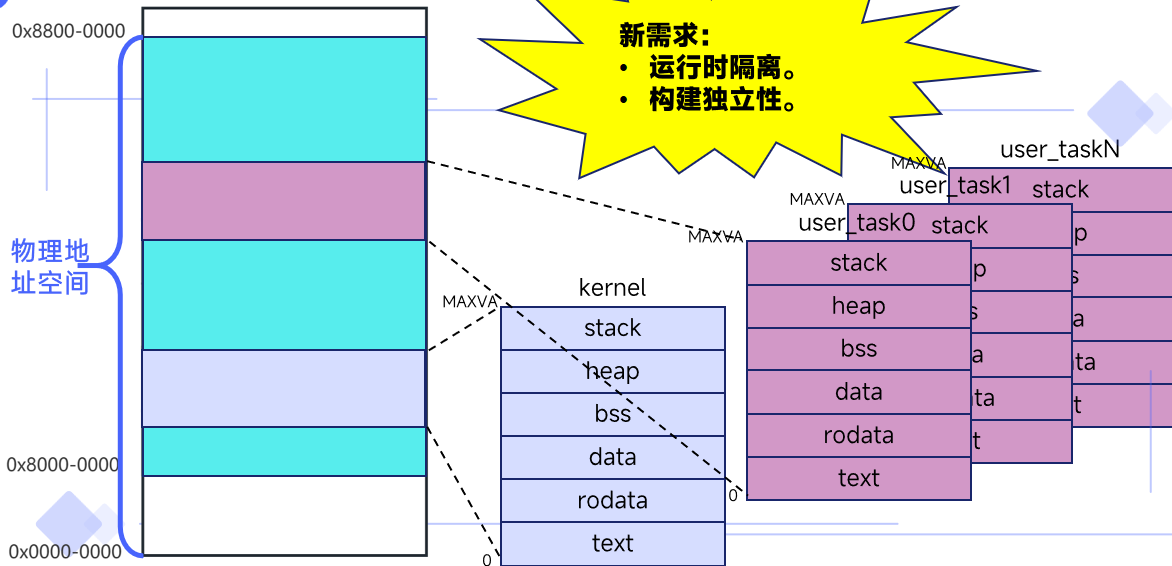
# 地址空间



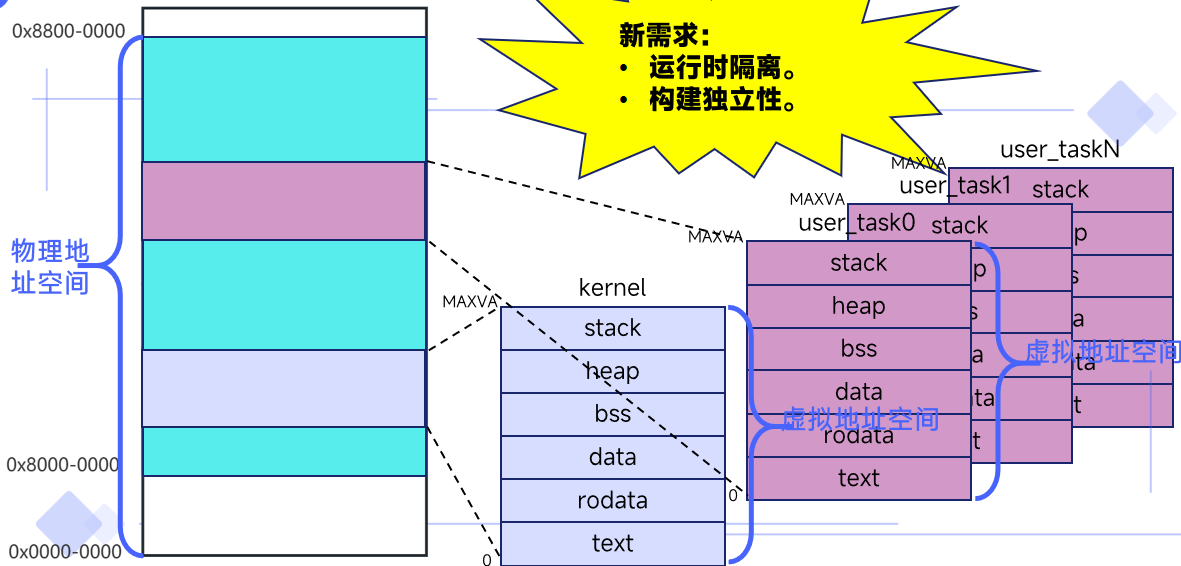
# 地址空间



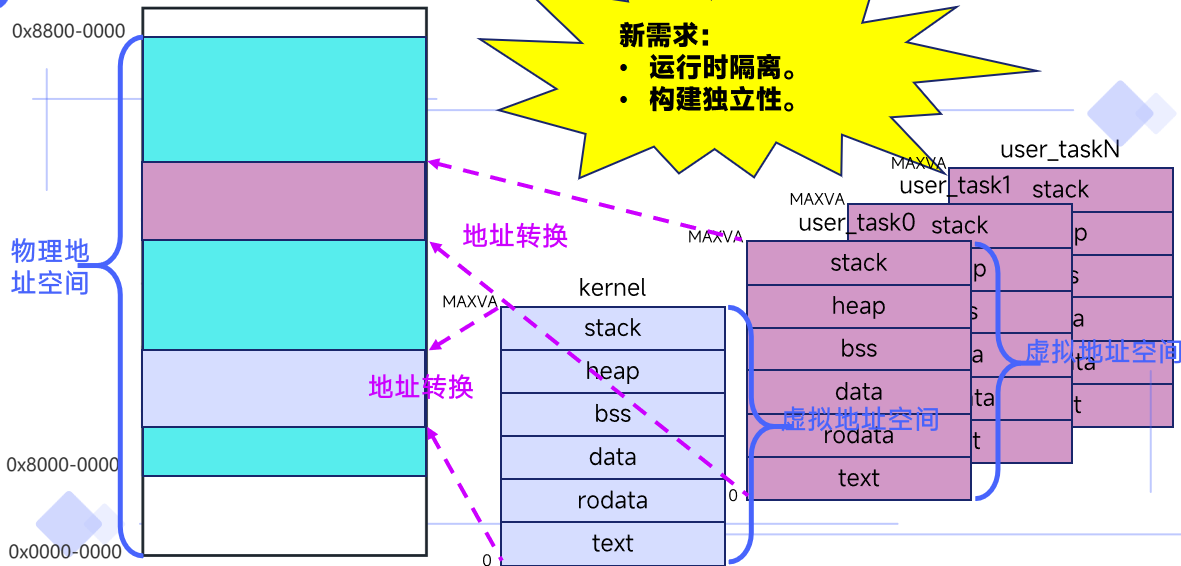
## 地址空间



# 地址空间

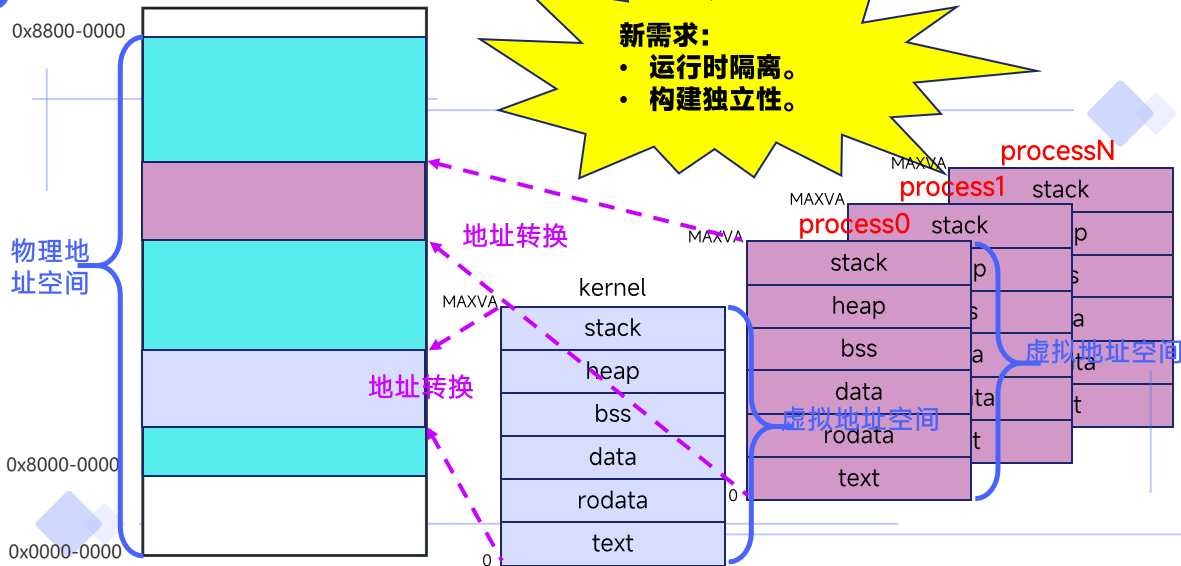


# 地址空间





# 地址空间





02

## 地址转换

---

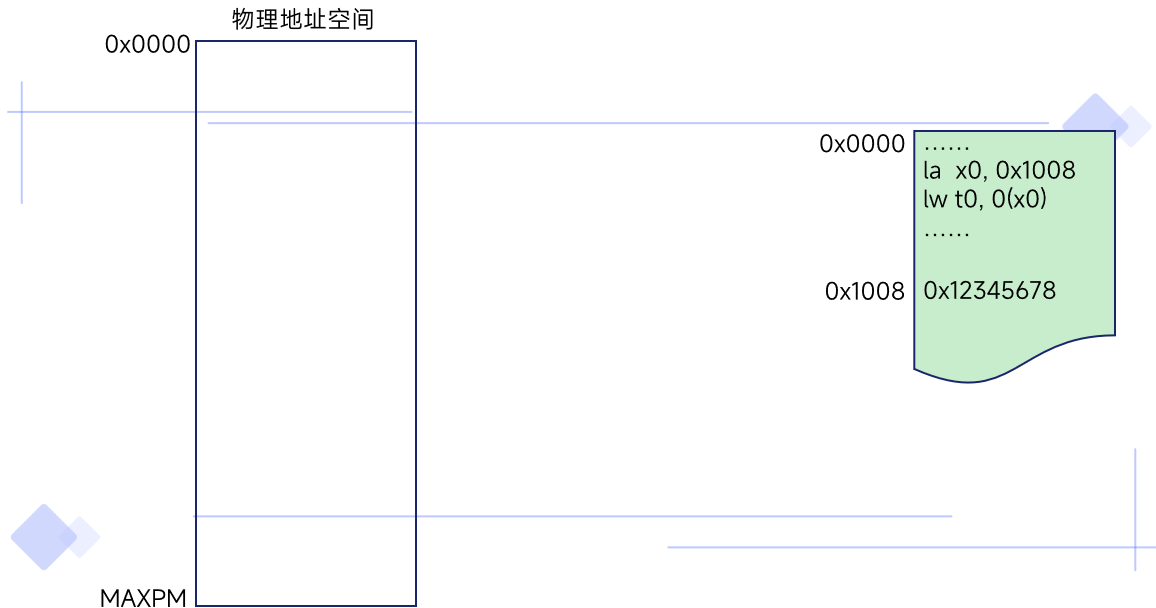
# 地址转换

- 重定位
- 分段
- 分页

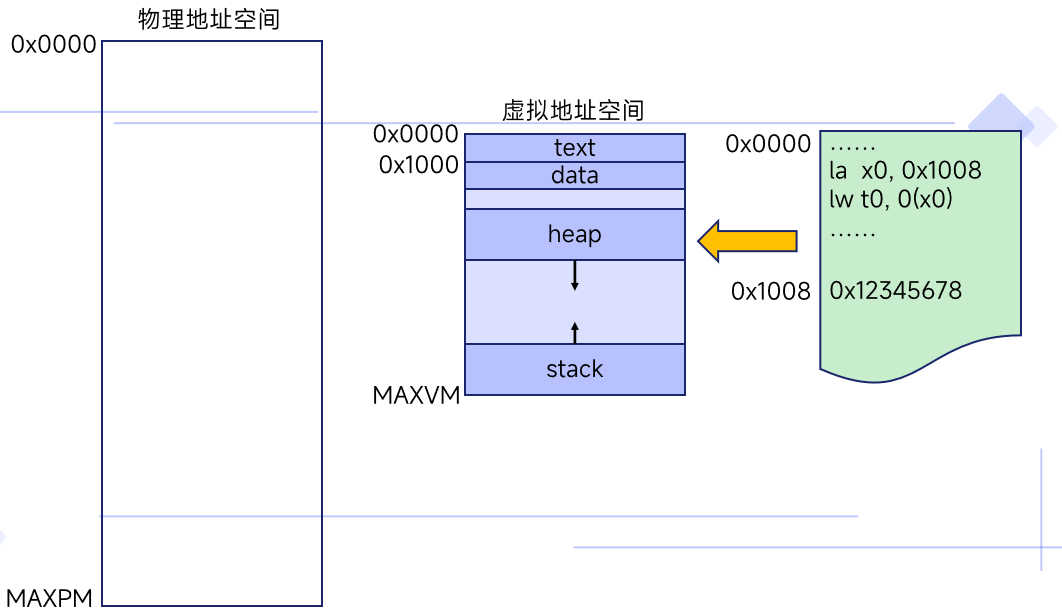
# 地址转换

- **重定位**
- 分段
- 分页

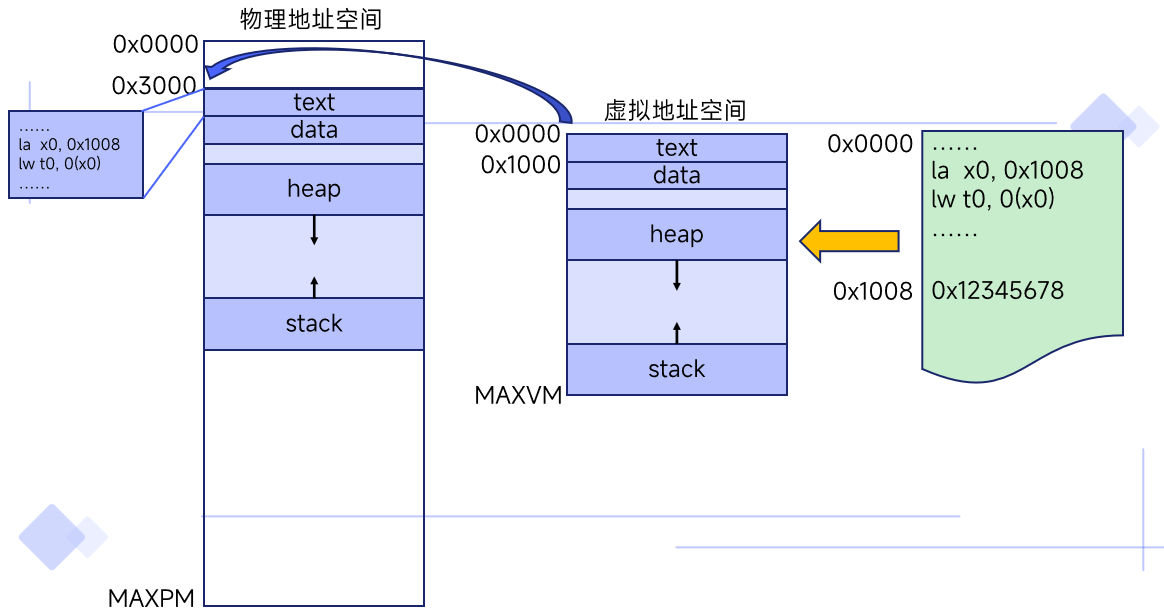
# 重定位



# 重定位



# 重定位



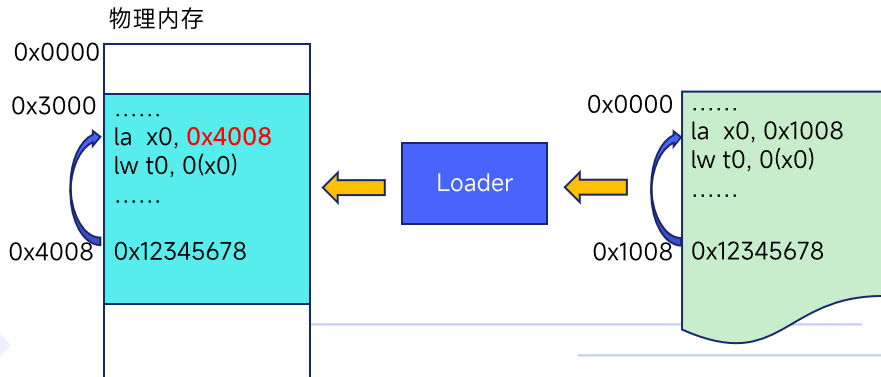
# 重定位

- 基于软件的静态重定位 (static relocation)
- 基于硬件的动态重定位 (dynamic relocation)



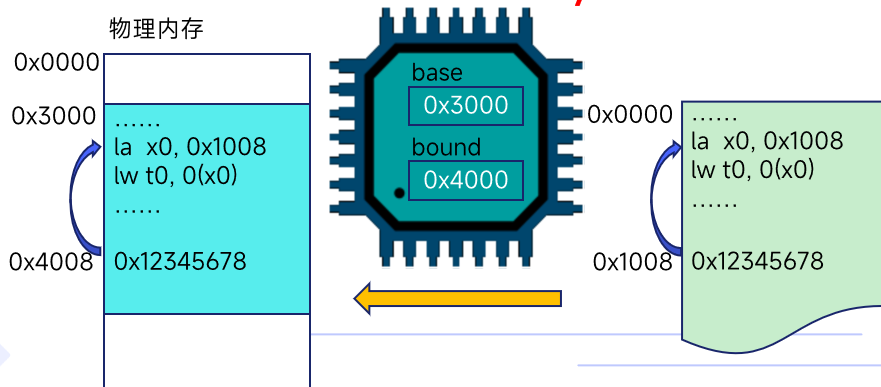
# 重定位

- 基于软件的静态重定位 (static relocation)
- 基于硬件的动态重定位 (dynamic relocation)



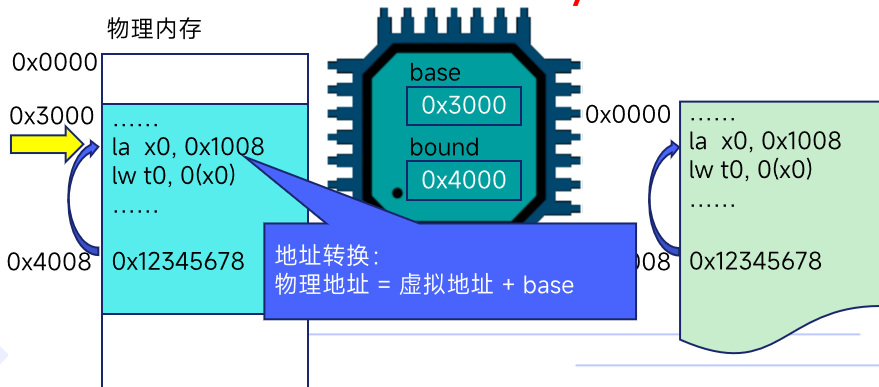
# 重定位

- 基于软件的静态重定位 (static relocation)
- 基于硬件的动态重定位 (dynamic relocation)



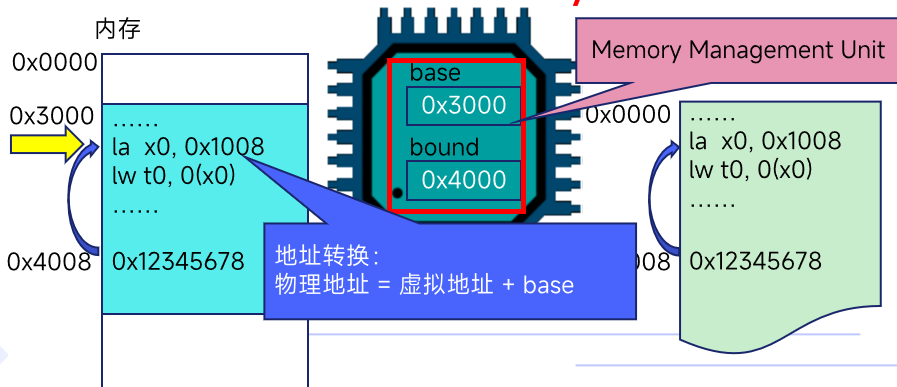
# 重定位

- 基于软件的静态重定位 (static relocation)
- 基于硬件的动态重定位 (dynamic relocation)

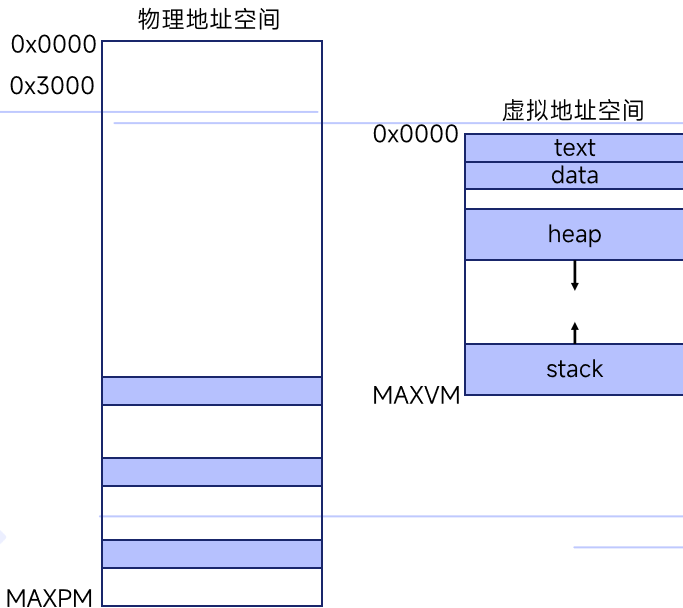


# 重定位

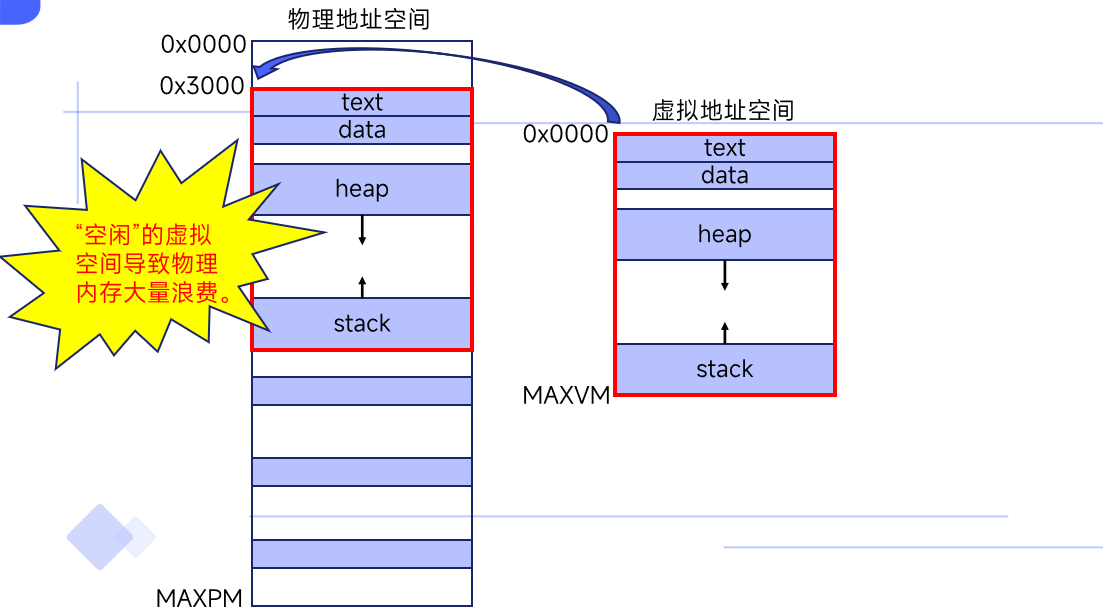
- 基于软件的静态重定位 (static relocation)
- 基于硬件的动态重定位 (dynamic relocation)



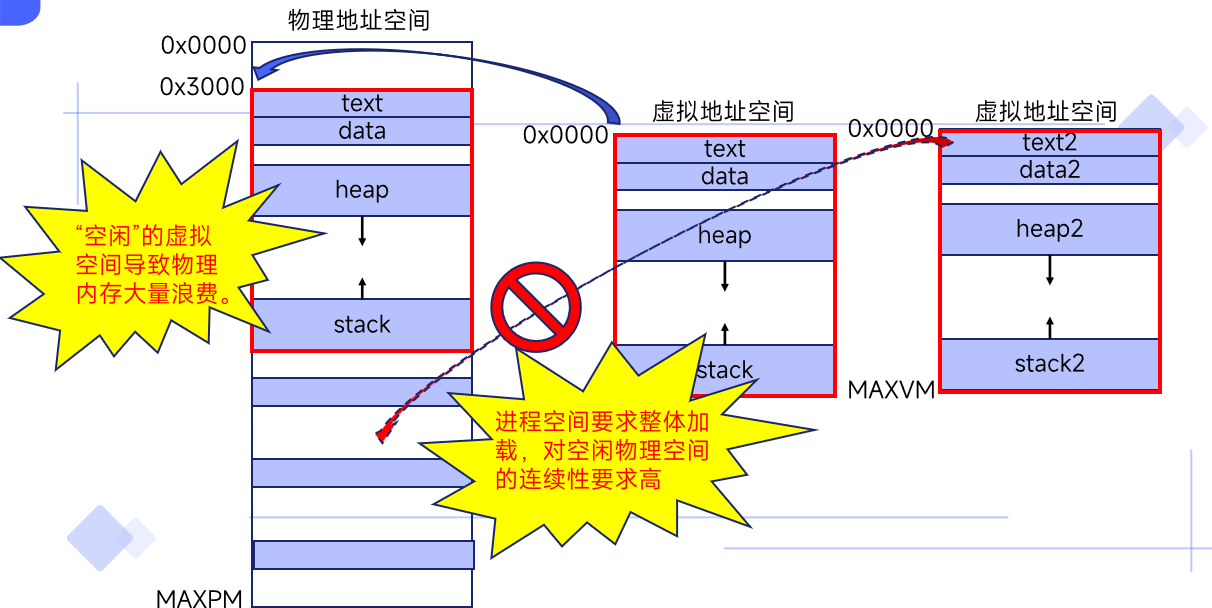
# 重定位



# 重定位



# 重定位

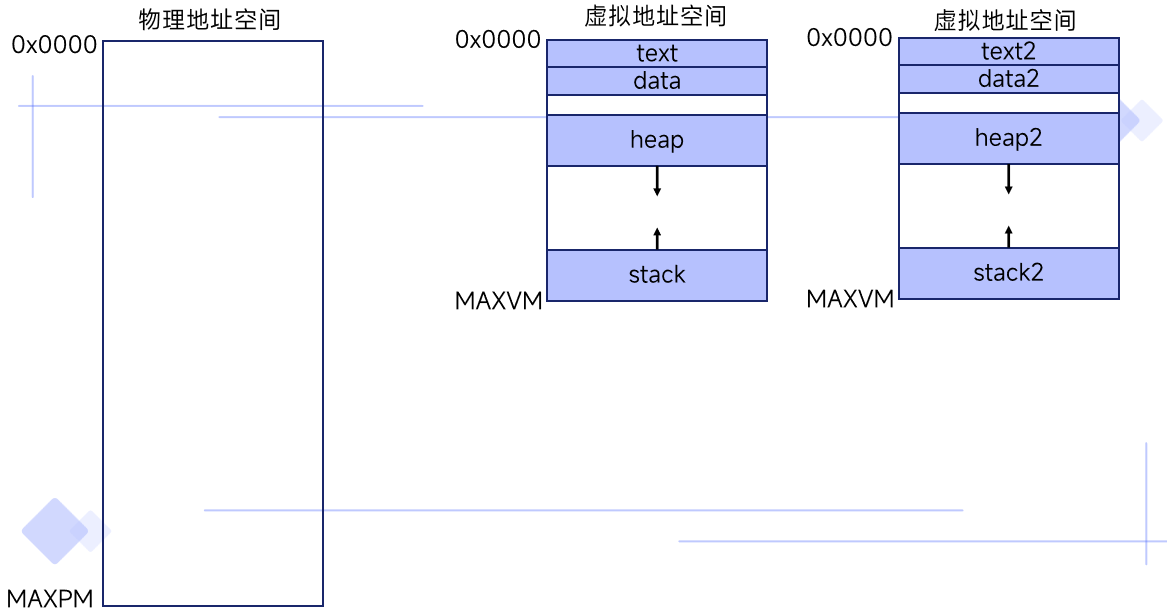


# 地址转换

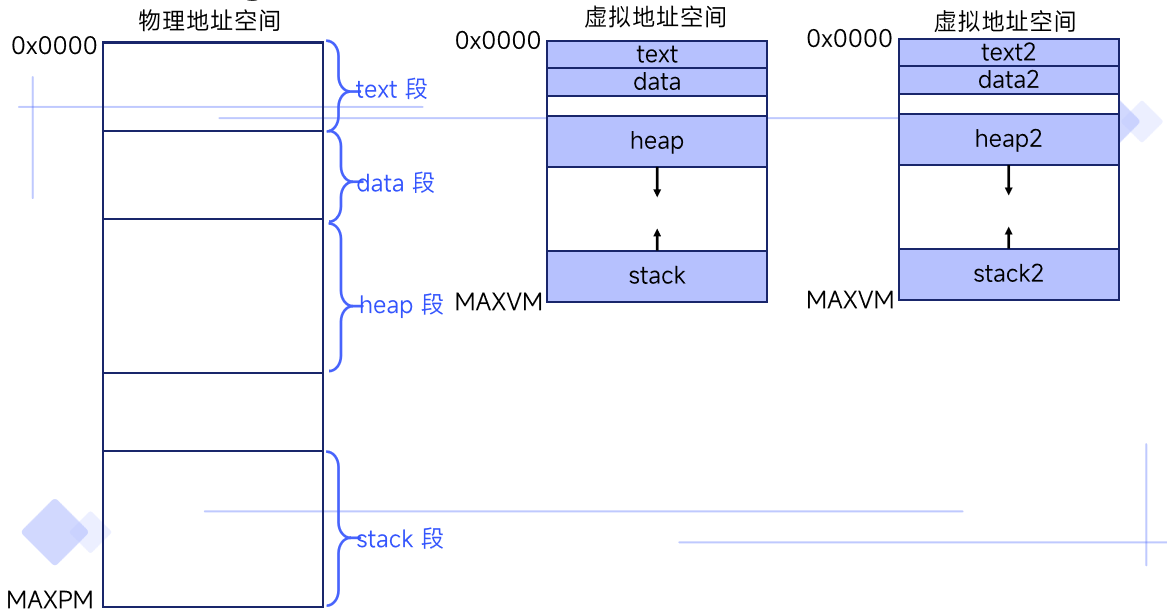
- 重定位
- **分段**
- 分页



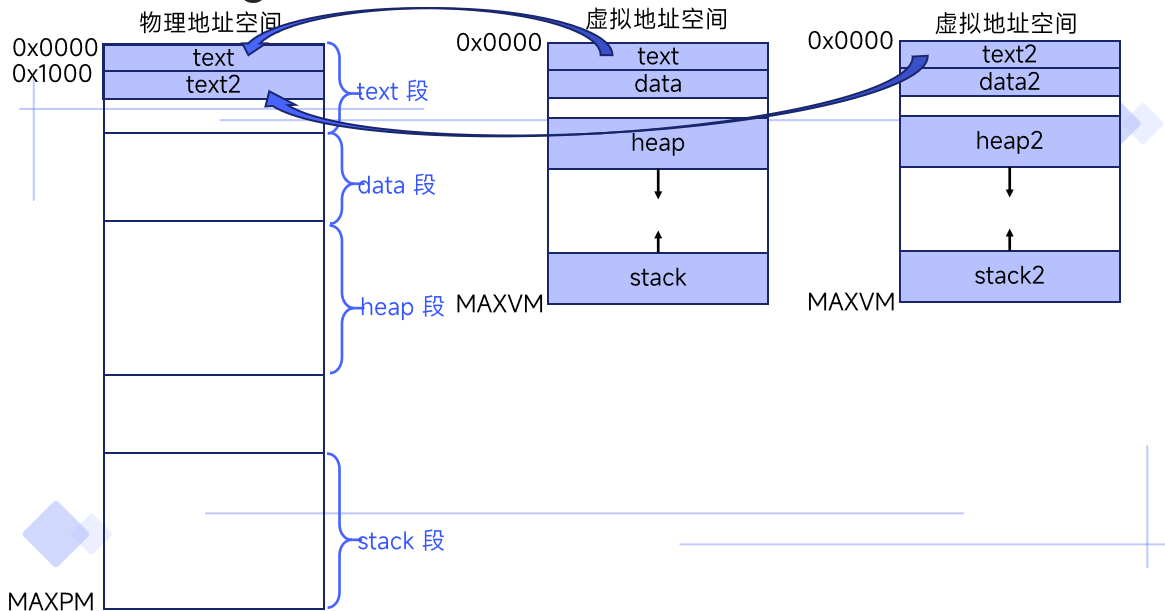
# 分段 ( segment )



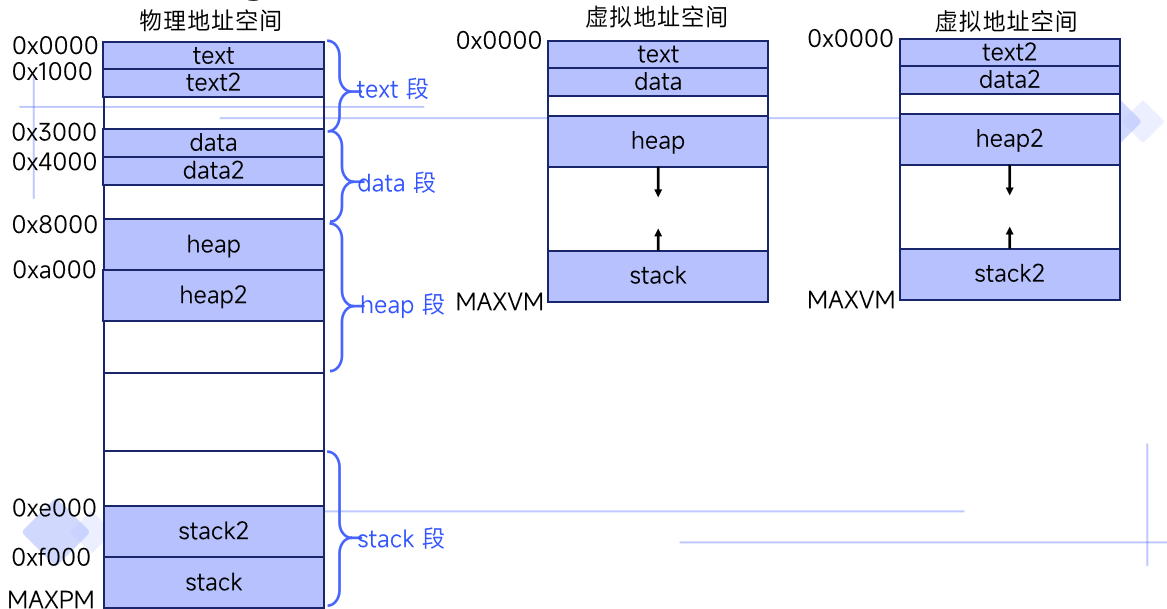
# 分段 ( segment )



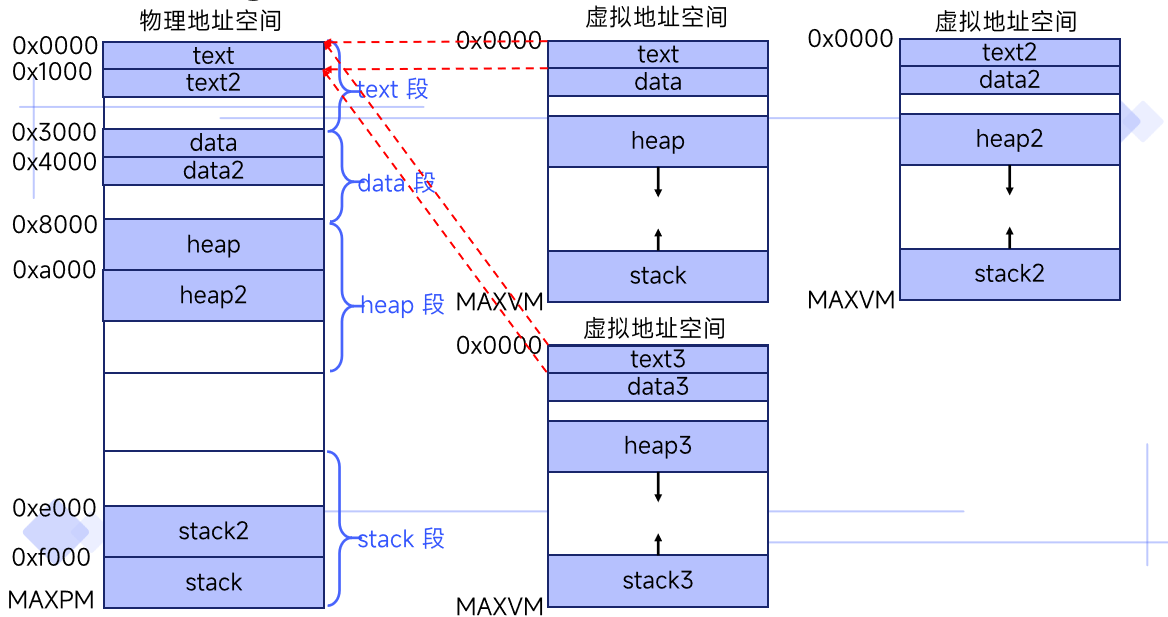
# 分段 ( segment )



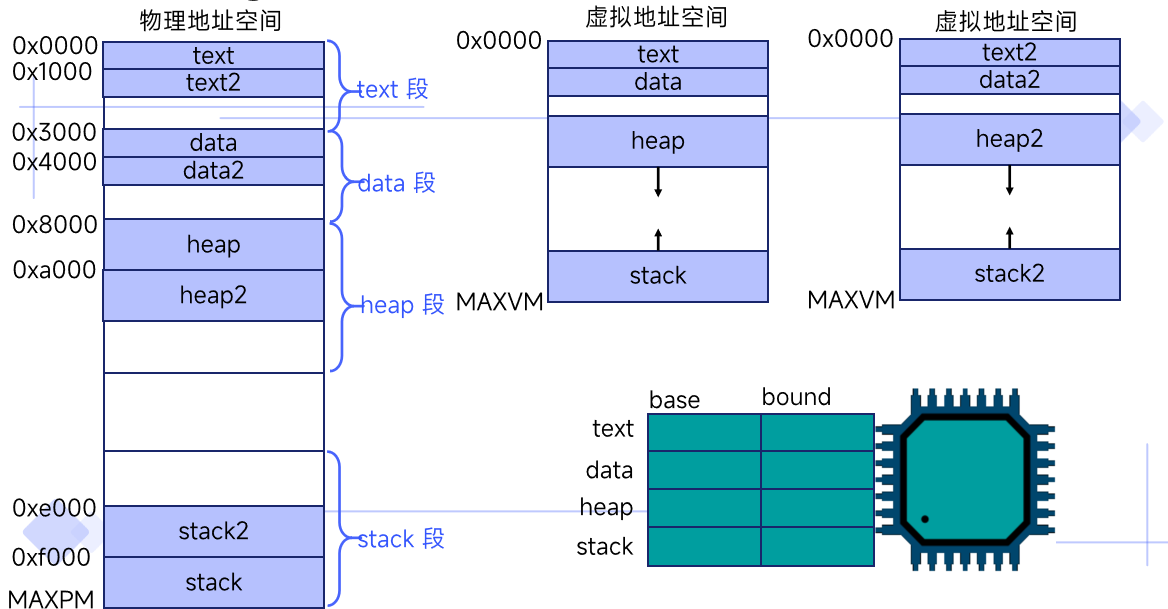
# 分段 ( segment )



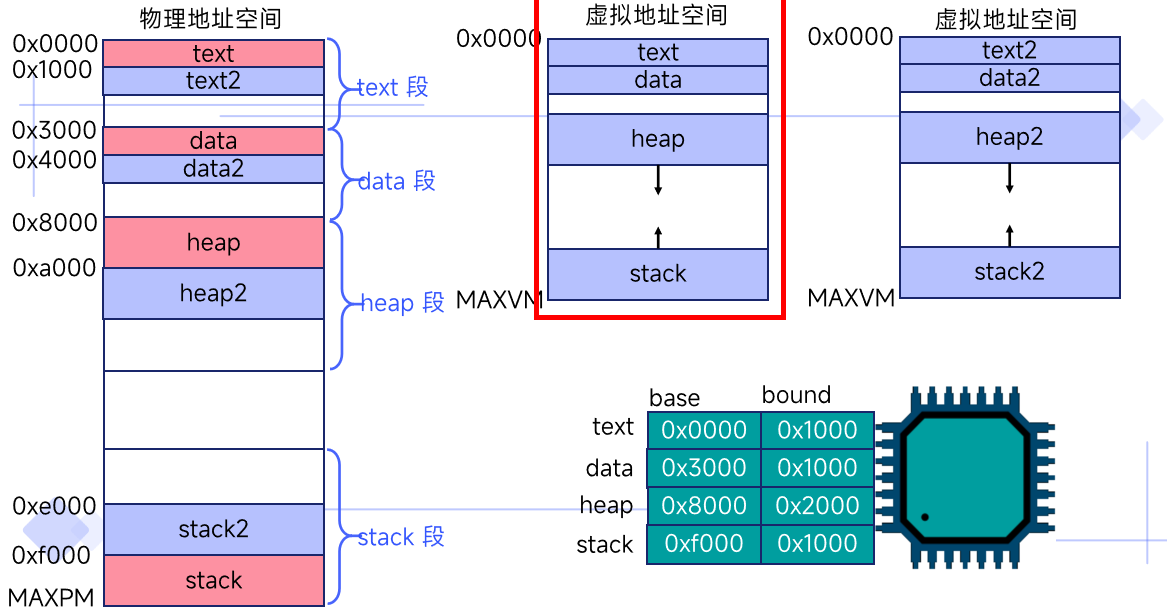
# 分段 ( segment )



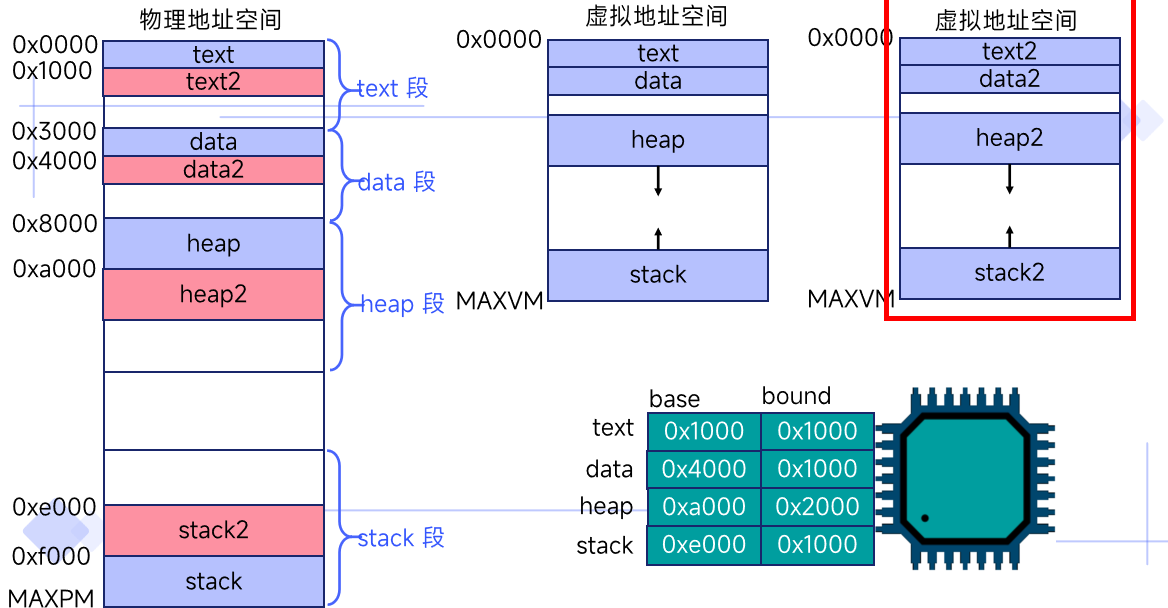
# 分段 ( segment )



# 分段 ( segment )

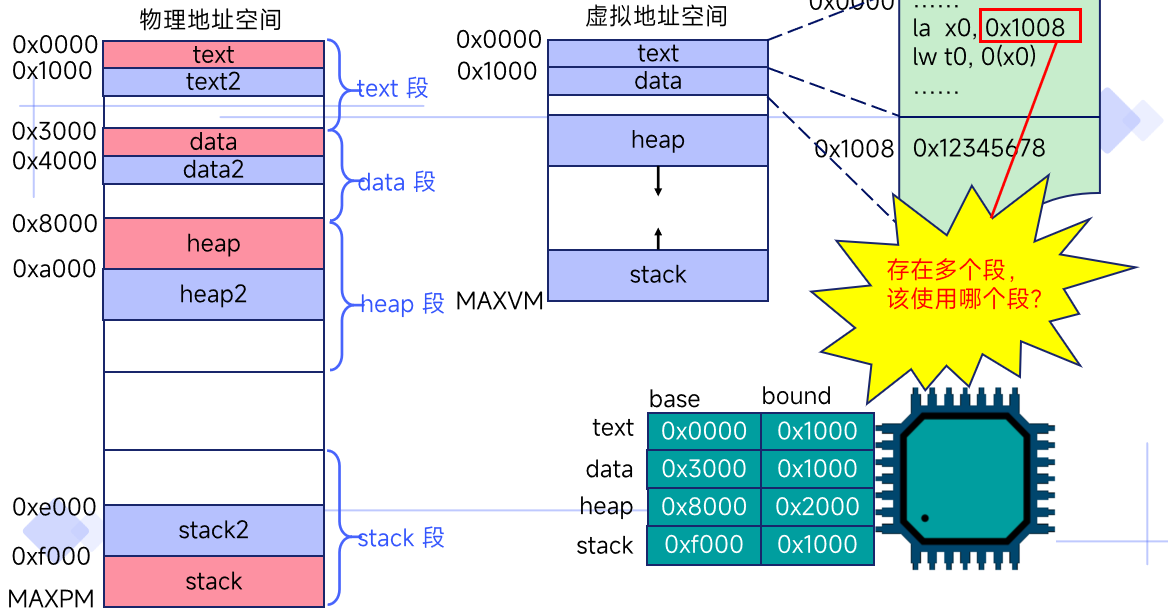


# 分段 ( segment )

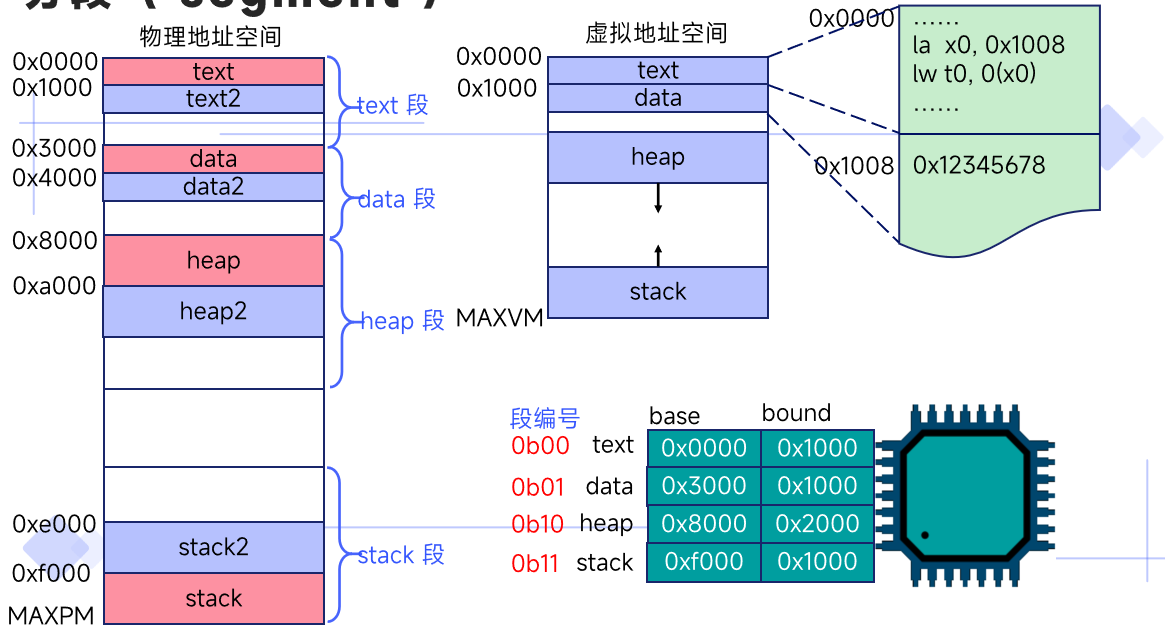




# 分段 ( segment )



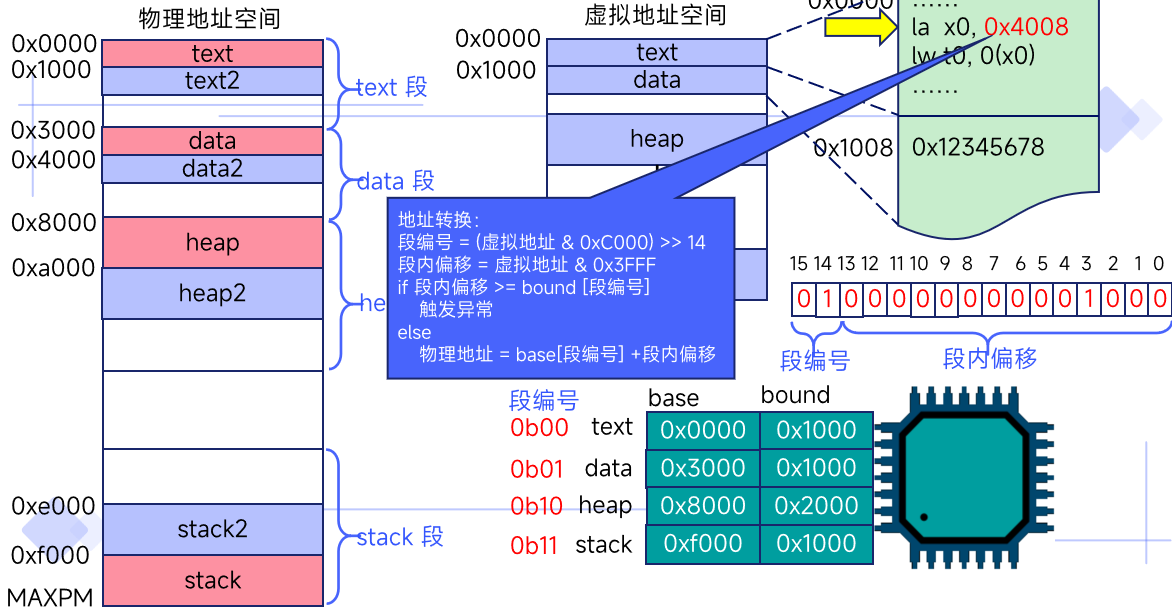
# 分段 ( segment )







## 分段 ( segment )

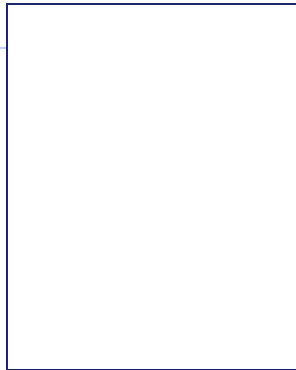


## 分段机制存在的问题

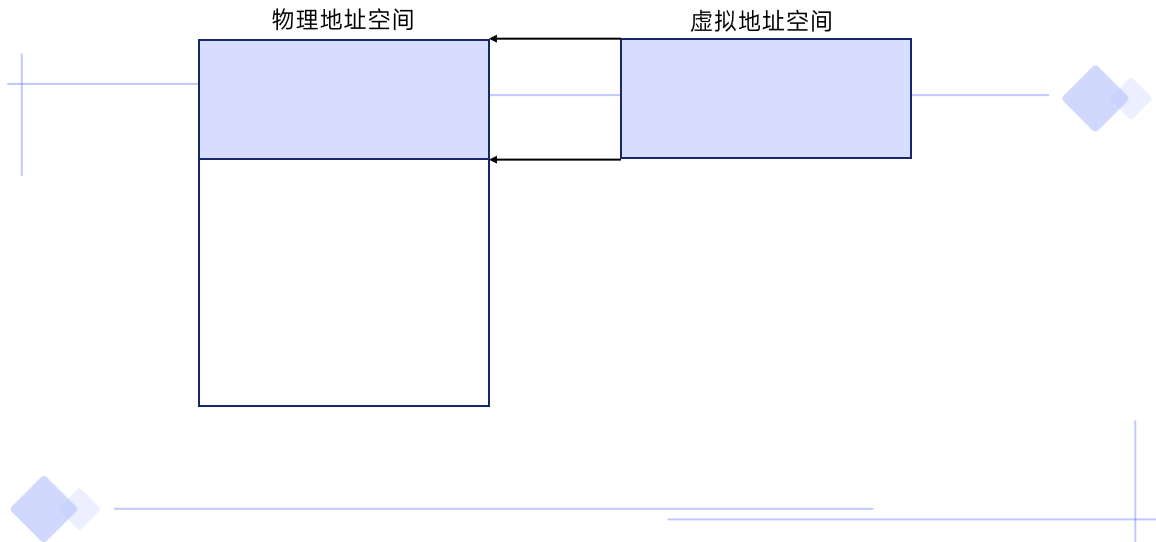
- 外部碎片 (External Fragmentation)
- 内存浪费 (Memory Waste)

# 分段（外部碎片 external fragmentation）

物理地址空间

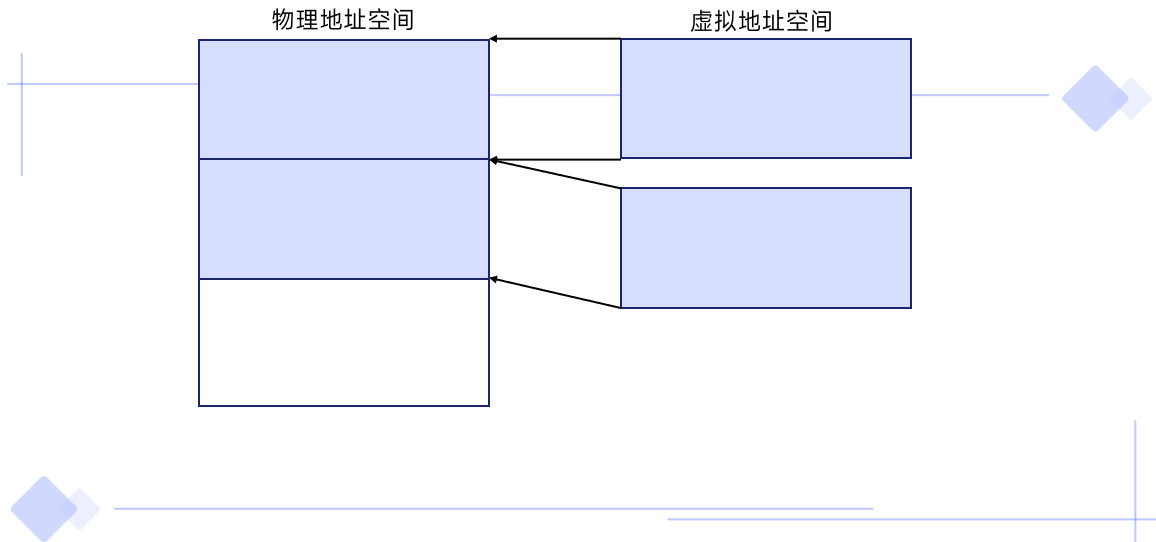


# 分段（外部碎片 external fragmentation）

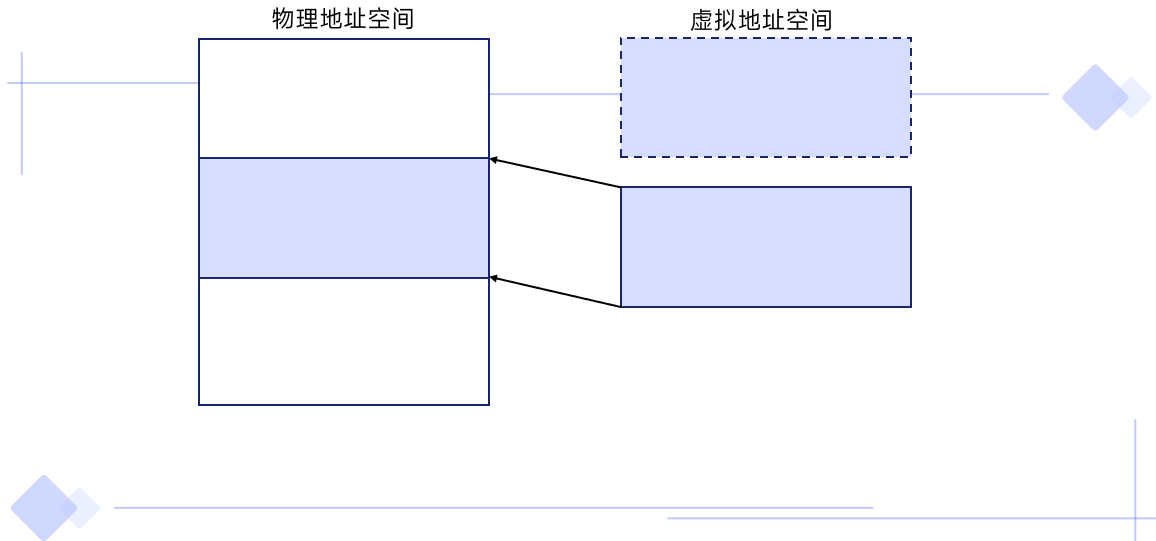




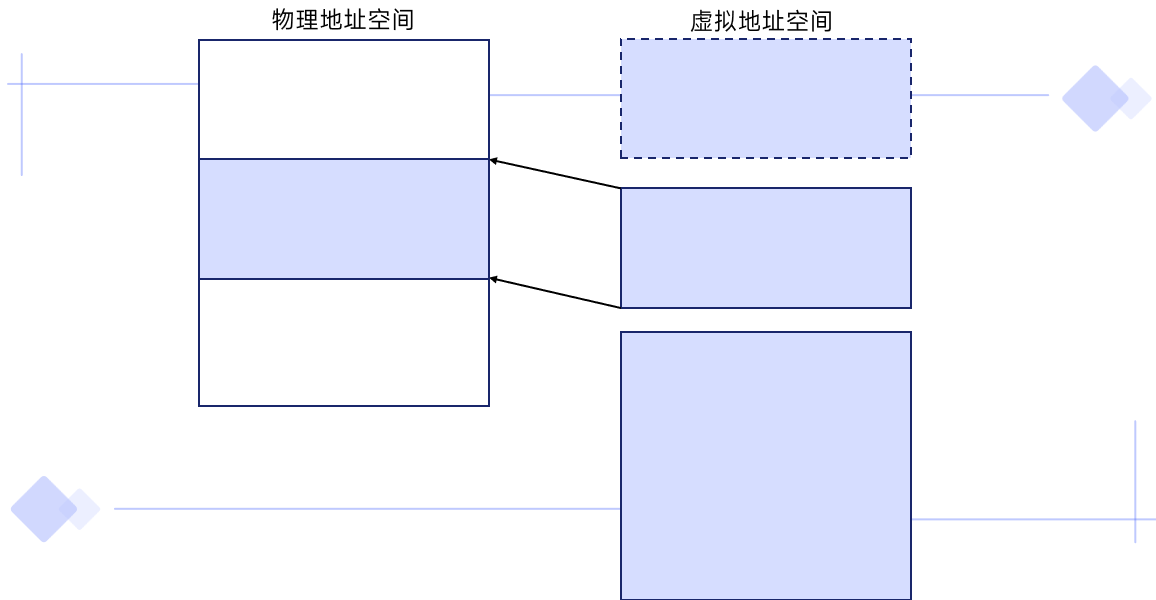
# 分段（外部碎片 external fragmentation）



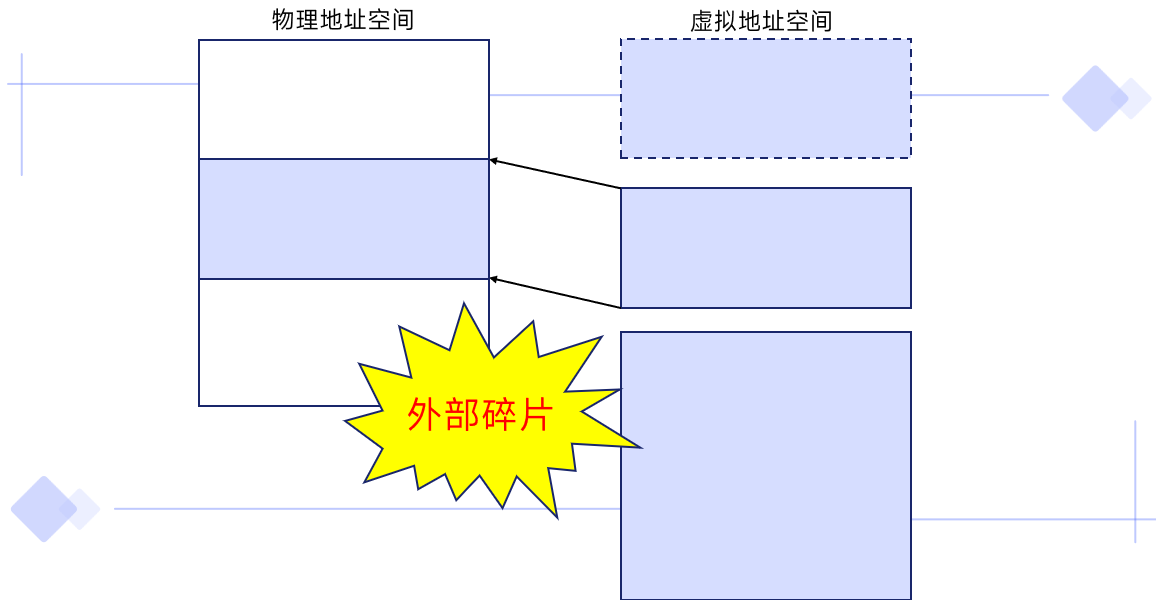
# 分段（外部碎片 external fragmentation）



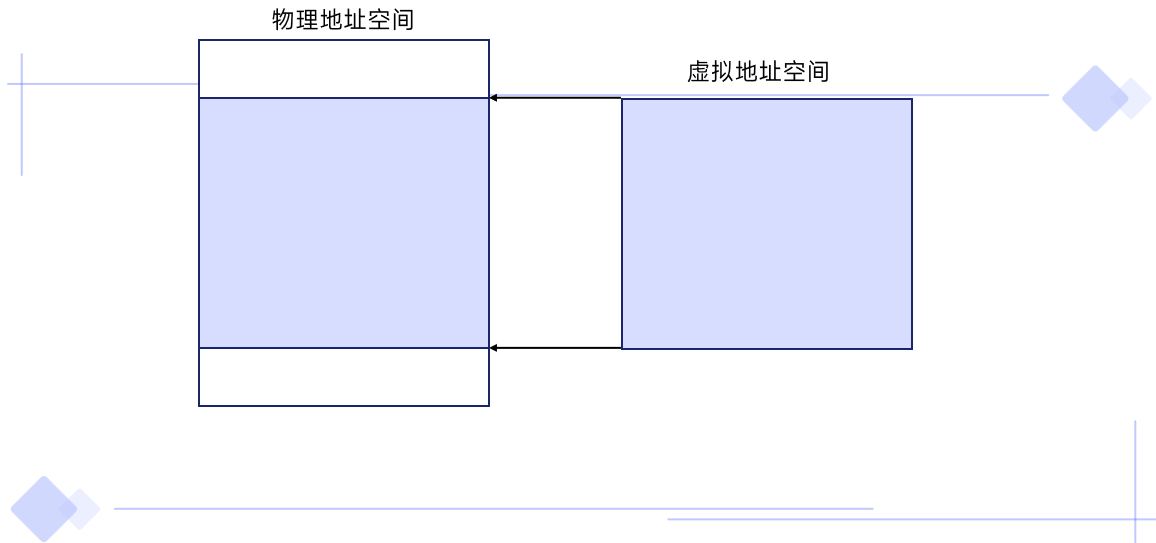
# 分段（外部碎片 external fragmentation）



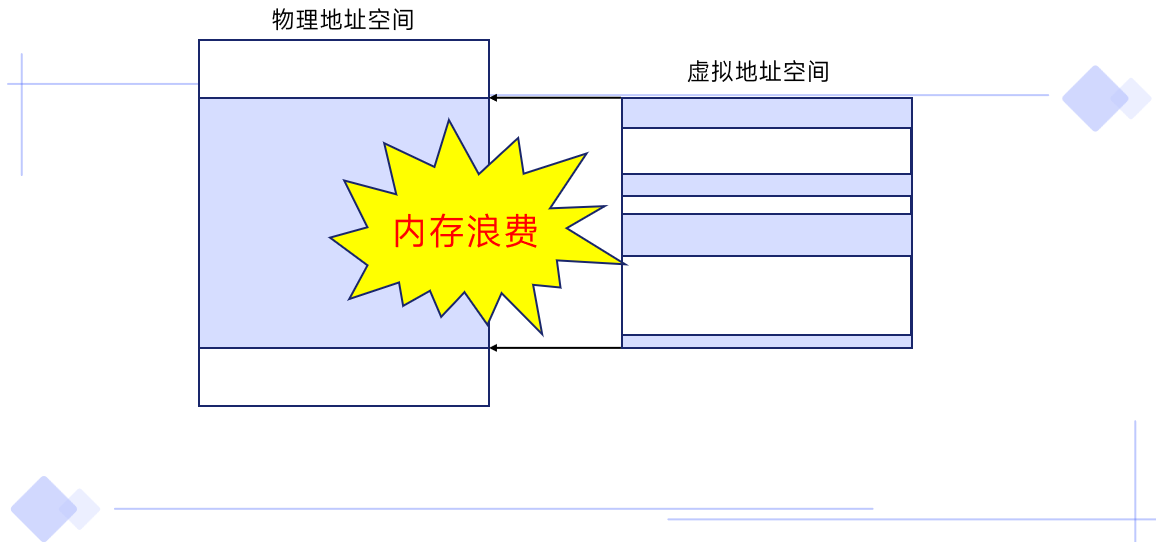
# 分段（外部碎片 external fragmentation）



# 分段（稀疏虚拟地址空间导致的内存浪费）



# 分段（稀疏虚拟地址空间导致的内存浪费）



# 地址转换

- 重定位
- 分段
- **分页**

## 分页

- 所谓分页，是指将进程的虚拟地址空间分割成固定大小的单元。每个单元称为“**页 (Page)**”。
- 对应的，我们将物理地址空间按照同样的单元大小分割，每个单元称为“**页帧 (Page Frame)**”。
- 在基于分页的虚拟内存管理技术中，硬件和软件配合通过 **页表** 将虚拟页映射到分散的物理页帧上，从而实现地址隔离并高效利用内存。



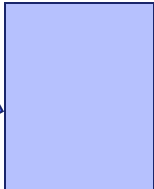
# 分页

分页可以有效解决“外部碎片”问题。

物理地址空间

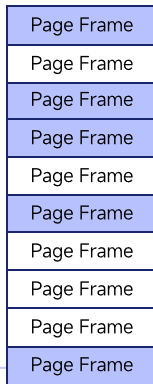


虚拟地址空间

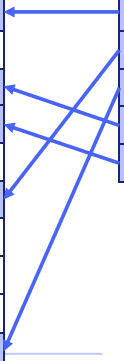
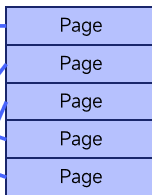


分段方式

物理地址空间



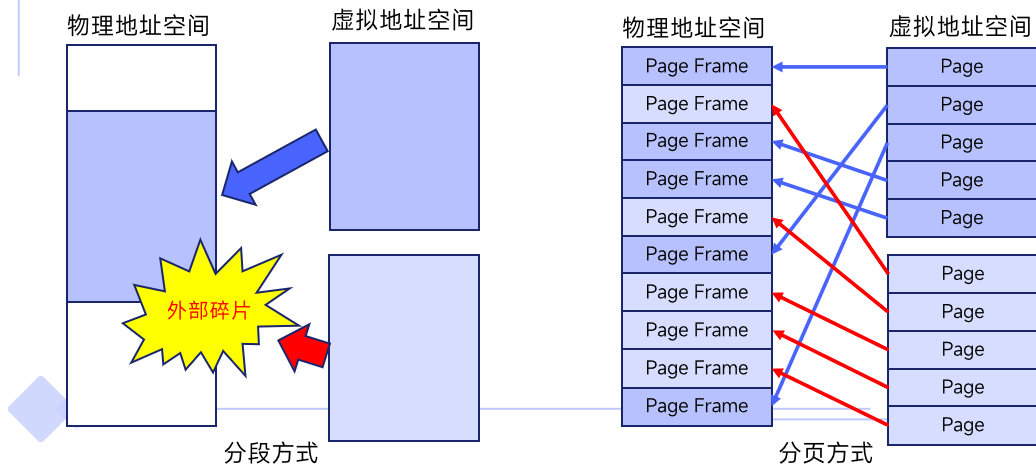
虚拟地址空间



分页方式

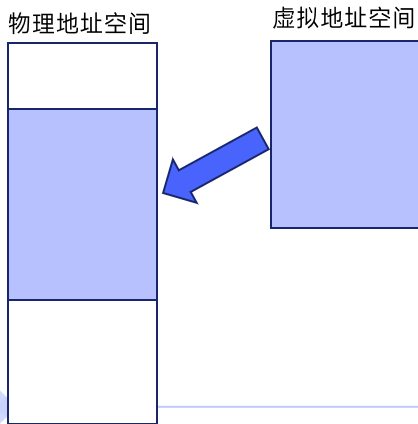
# 分页

分页可以有效解决“外部碎片”问题。

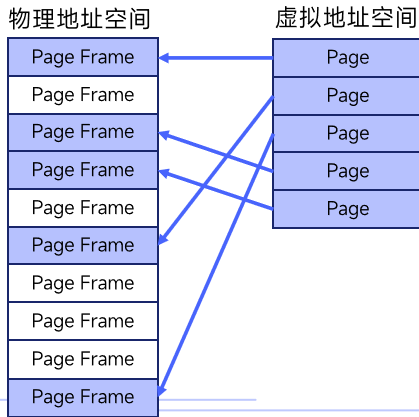


# 分页

分页可以有效解决稀疏虚拟地址空间导致的“内存浪费”。



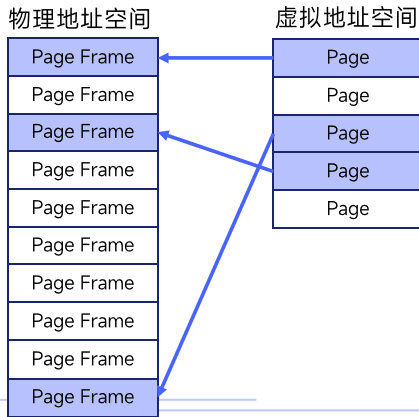
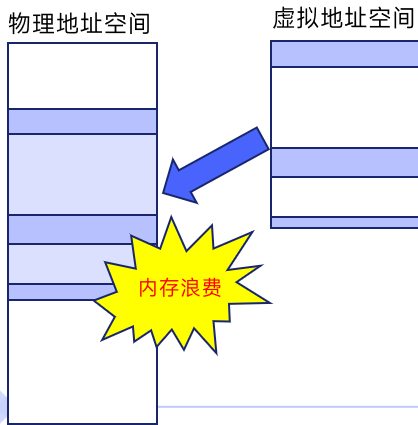
分段方式



分页方式

# 分页

分页可以有效解决稀疏虚拟地址空间导致的“内存浪费”。



# 页表

- 系统软件负责记录虚拟地址空间中每个页所对应的页帧的位置（映射关系），这个用于记录映射关系的数据结构就是“**页表（Page Table）**”。
- “页表”保存在内存中，每个进程一个。
- MMU 记录了当前进程的“页表”的位置。
- 启用“页表”功能后，MMU 负责通过查询页表，将指令中所使用的虚拟地址转换为物理地址（地址转换），完成最终的寻址操作。



**03**

# **RISC-V 页表**

---

# RISC-V 页表

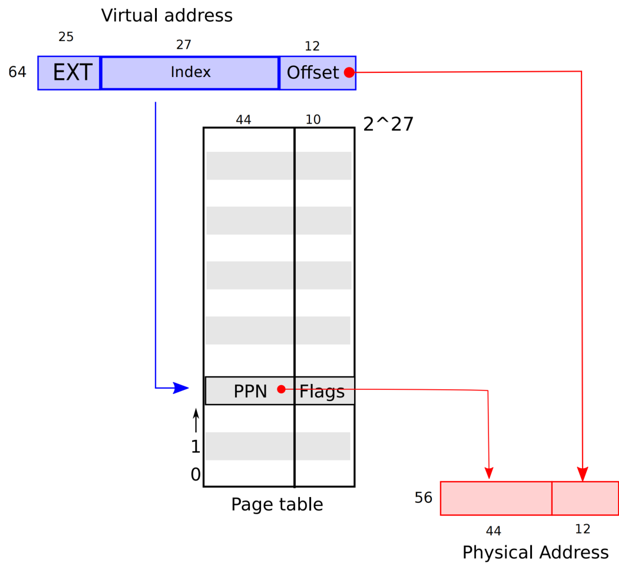
- 一级页表
- 三级页表

# RISC-V 页表

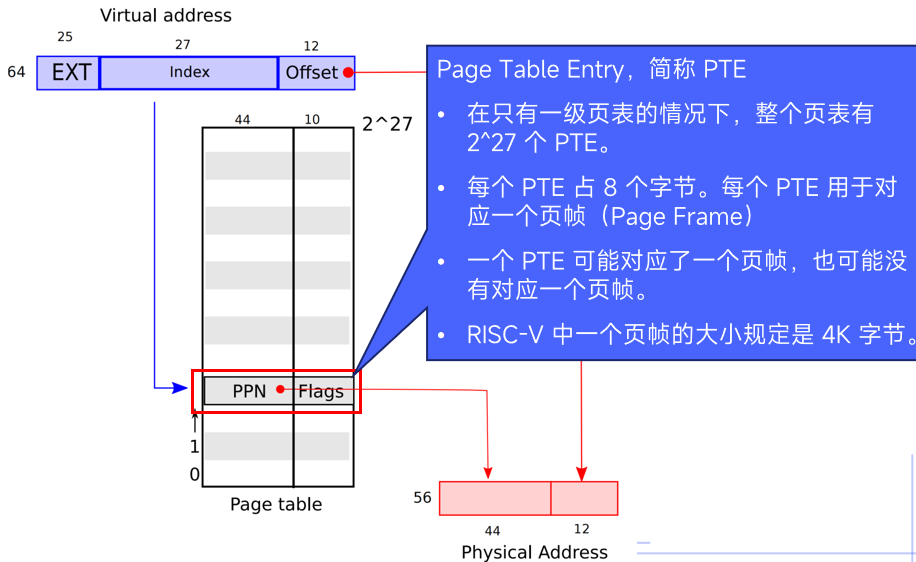
- **一级页表**
- 三级页表



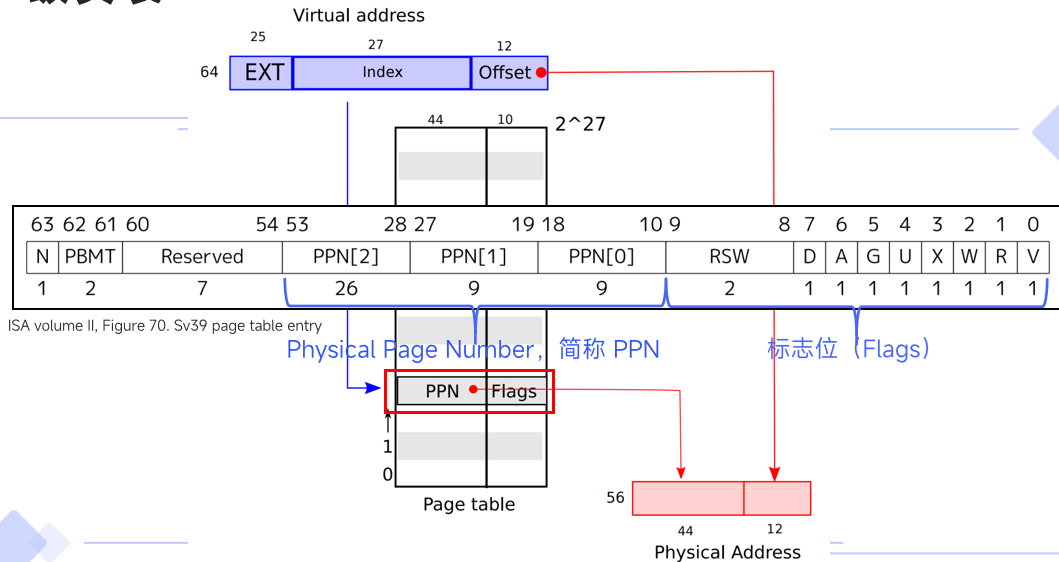
# 一级页表



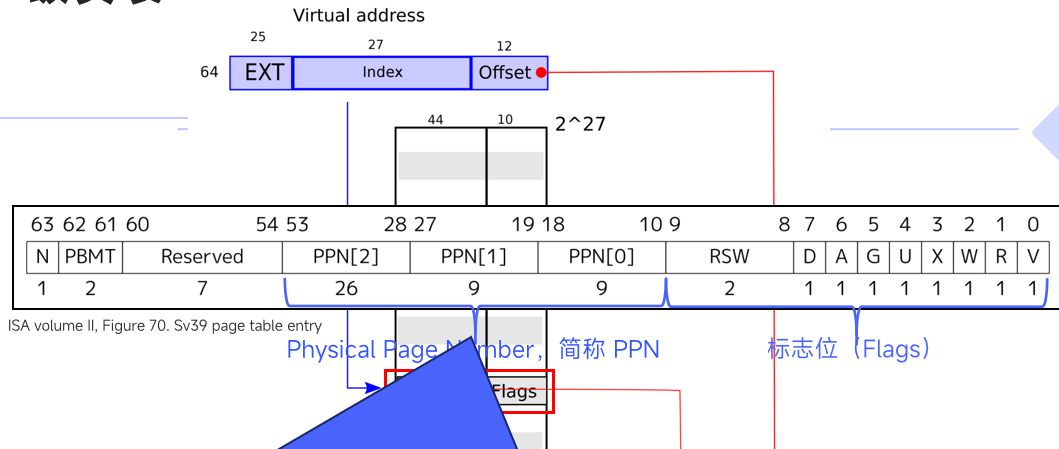
# 一级页表



# 一级页表

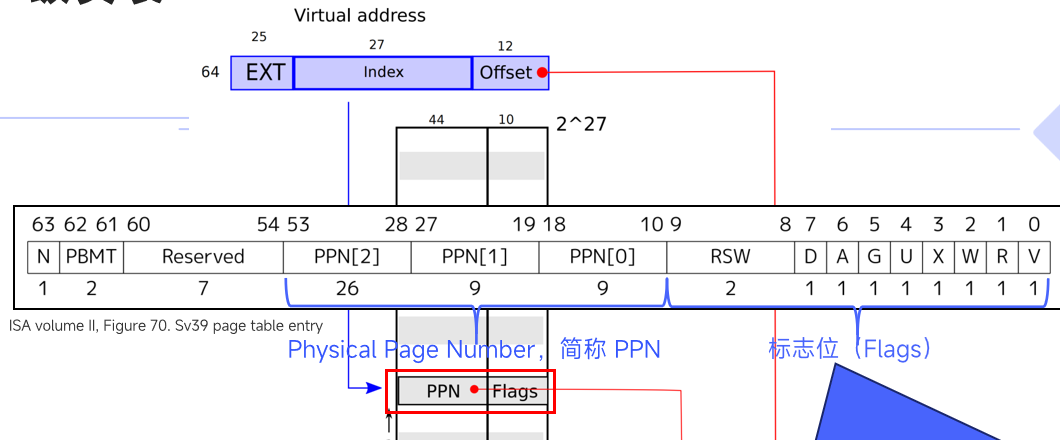


# 一级页表



理论上物理地址可表达的最大范围是： $2^{44} \times \text{sizeof(页帧)} = 2^{16} \text{ TB} = 64 \text{ PB}$   
但注意在 Sv39 下，可以访问的 PTE 的最大个数是  $2^{27}$  项，即实际可映射管理的物理内存大小为  $2^{27} \times \text{sizeof(页帧)} = 512 \text{ GB}$ ，达不到理论上物理地址可表达的最大值。

# 一级页表

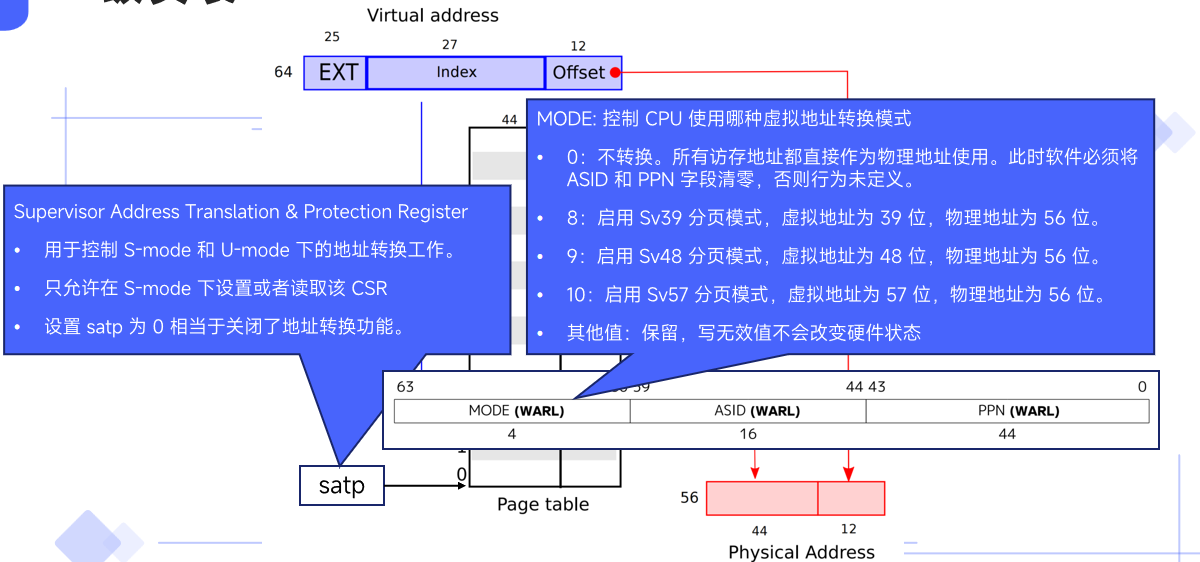


ISA volume II, Figure 70. Sv39 page table entry

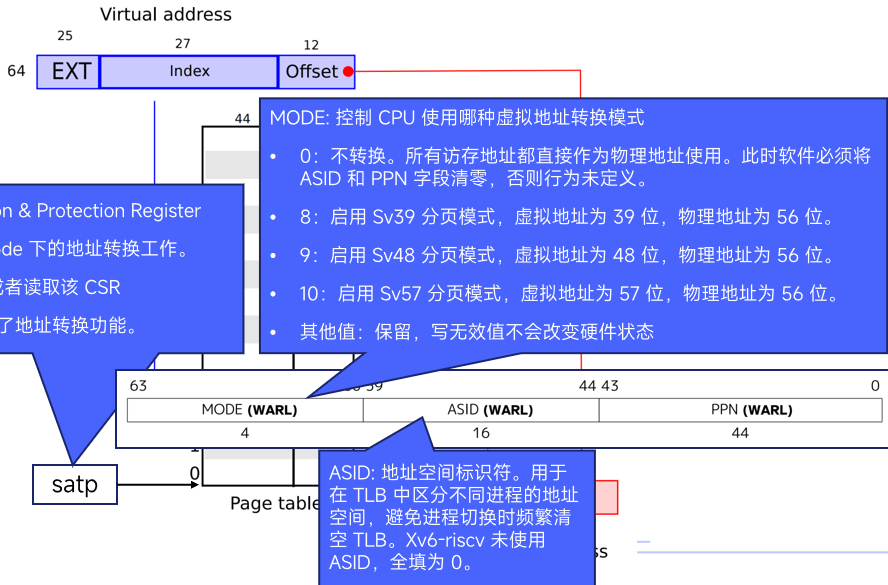
- V (Valid): 有效位。为 1 时表示该页表项有效; 为 0 时表示无效, 访问会触发缺页异常。
- R (Read): 读权限位。为 1 表示其映射的页可读。
- W (Write): 写权限位。为 1 表示其映射的页可写。
- X (Execute): 执行权限位。为 1 表示其映射的页中的指令可被执行。
- U (User): 用户模式访问位。为 1 时, U-mode 下可访问该页; 否则仅 S-mode 可访问。

- 用于控制 S-mode 和 U-mode 下的地址转换工作。
- 只允许在 S-mode 下设置或者读取该 CSR
- 设置 satp 为 0 相当于关闭了地址转换功能。

# 一级页表

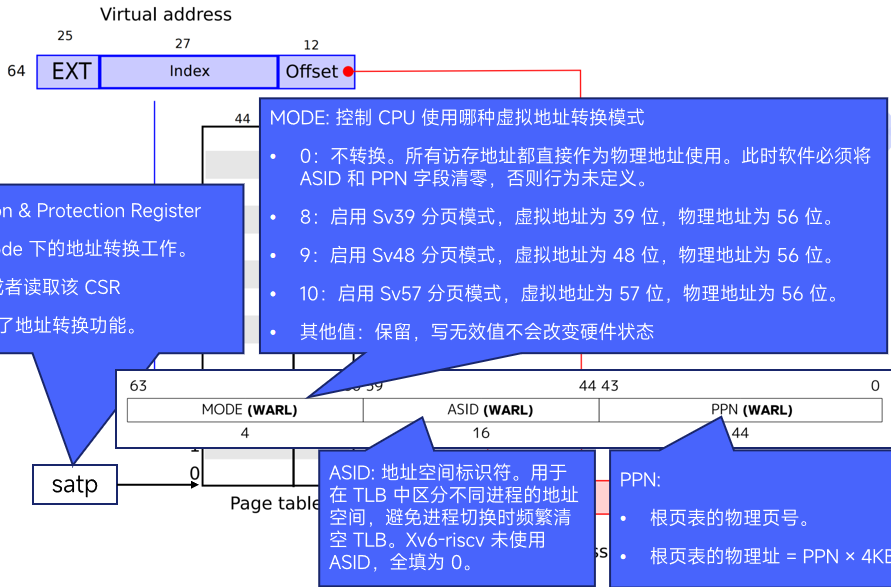


# 一级页表





# 一级页表



# 一级页表

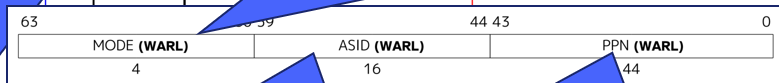
```
// use riscv's sv39 page table scheme.  
#define SATP_SV39 (8L << 60)  
  
#define MAKE_SATP(paetable) (SATP_SV39 | (((uint64)paetable) >> 12))
```

## Supervisor Address Translation & Protection Register

- 用于控制 S-mode 和 U-mode 下的地址转换工作。
- 只允许在 S-mode 下设置或者读取该 CSR
- 设置 satp 为 0 相当于关闭了地址转换功能。

MODE: 控制 CPU 使用哪种虚拟地址转换模式

- 0: 不转换。所有访存地址都直接作为物理地址使用。此时软件必须将 ASID 和 PPN 字段清零, 否则行为未定义。
- 8: 启用 Sv39 分页模式, 虚拟地址为 39 位, 物理地址为 56 位。
- 9: 启用 Sv48 分页模式, 虚拟地址为 48 位, 物理地址为 56 位。
- 10: 启用 Sv57 分页模式, 虚拟地址为 57 位, 物理地址为 56 位。
- 其他值: 保留, 写无效值不会改变硬件状态



satp

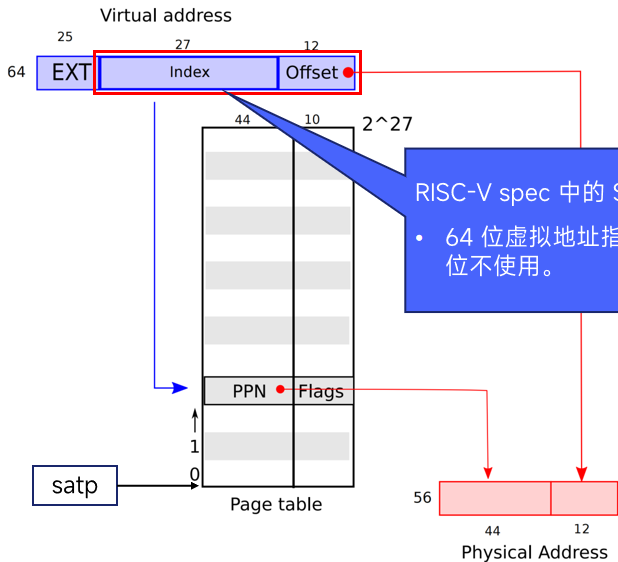
Page table

ASID: 地址空间标识符。用于在 TLB 中区分不同进程的地址空间, 避免进程切换时频繁清空 TLB。Xv6-riscv 未使用 ASID, 全填为 0。

PPN:

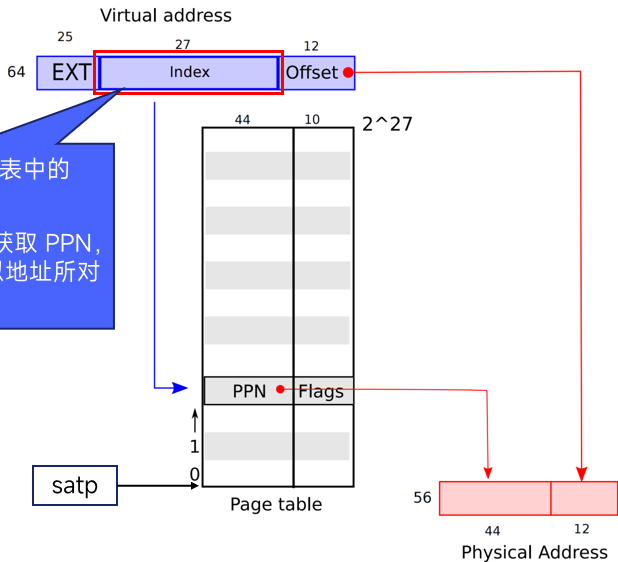
- 根页表的物理页号。
- 根页表的物理址 = PPN × 4KB

# 一级页表

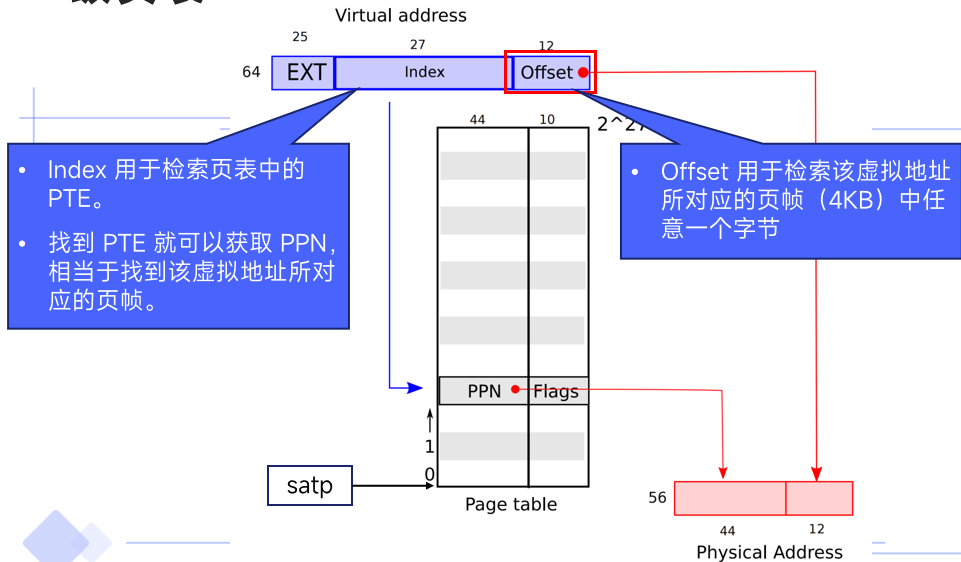


# 一级页表

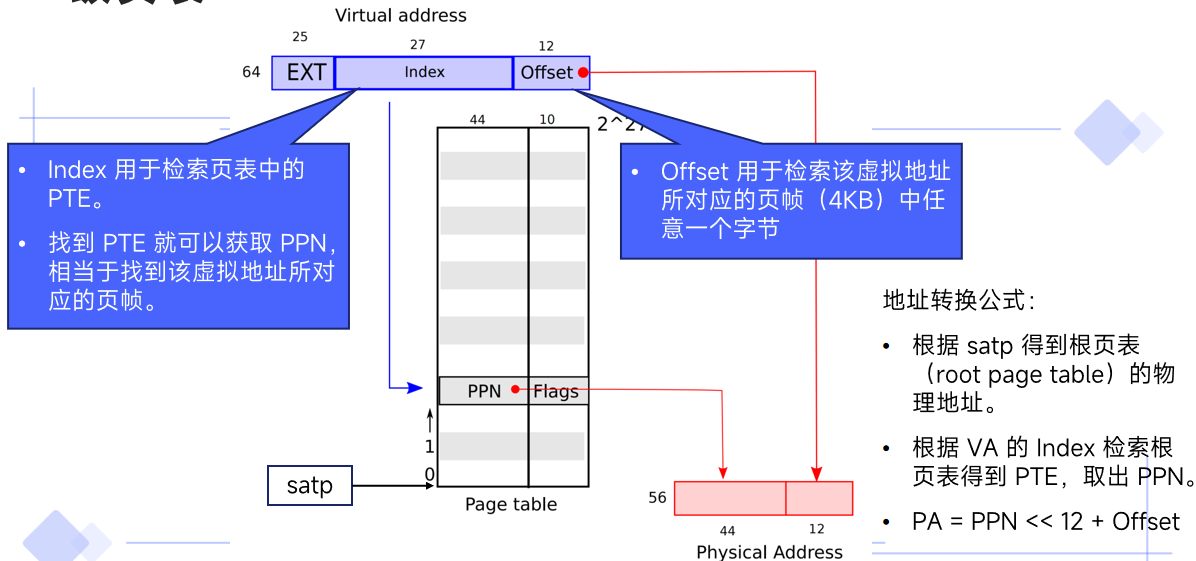
- Index 用于检索页表中的 PTE。
- 找到 PTE 就可以获取 PPN, 相当于找到该虚拟地址所对应的页帧。



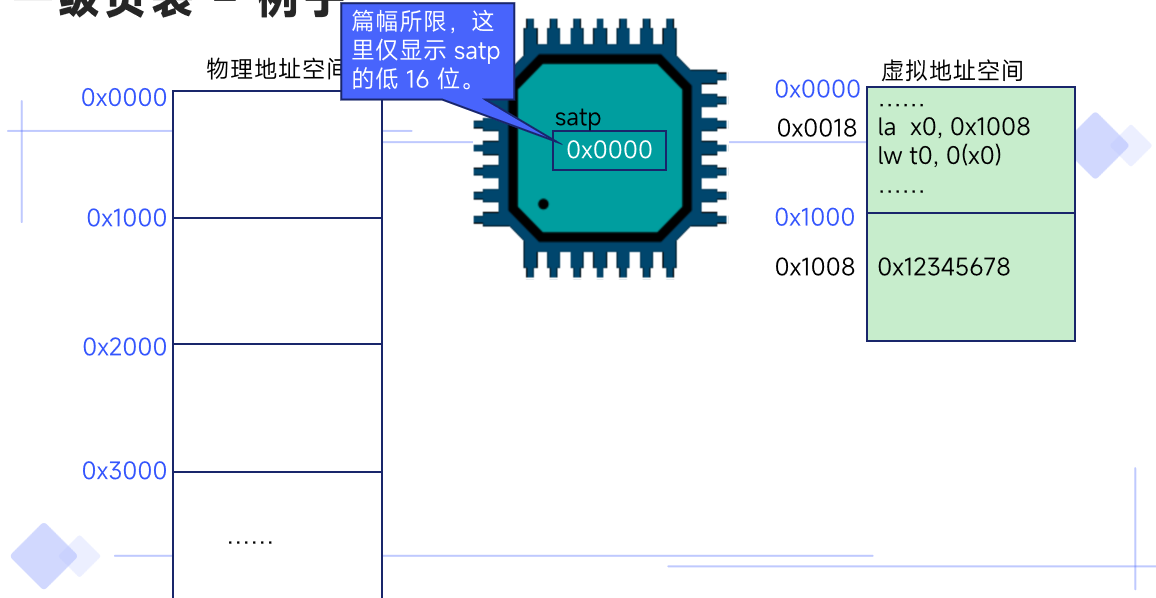
# 一级页表



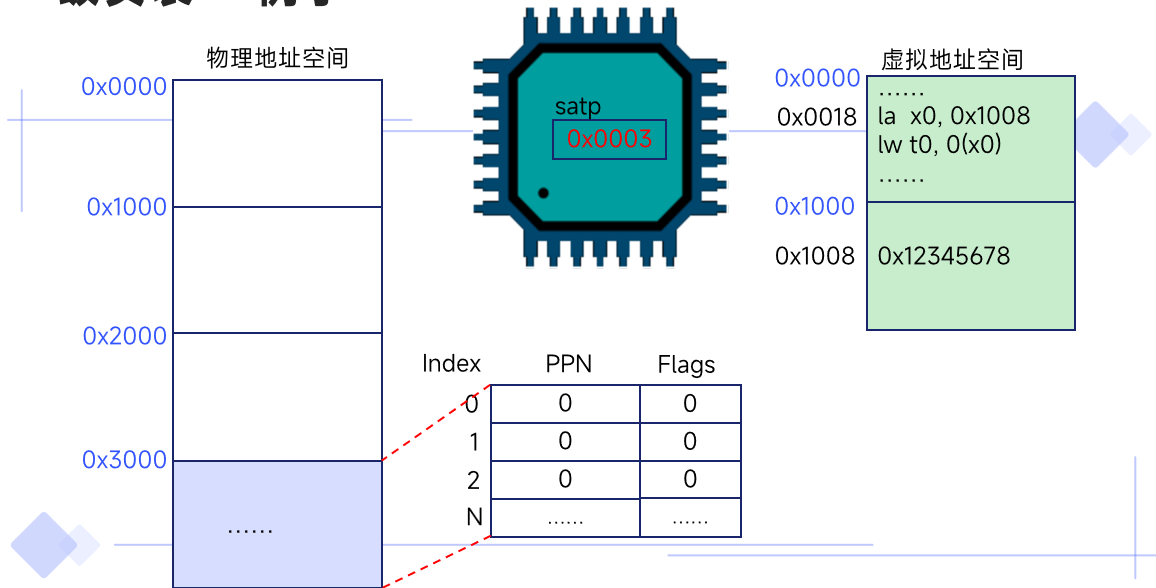
# 一级页表



# 一级页表 - 例子

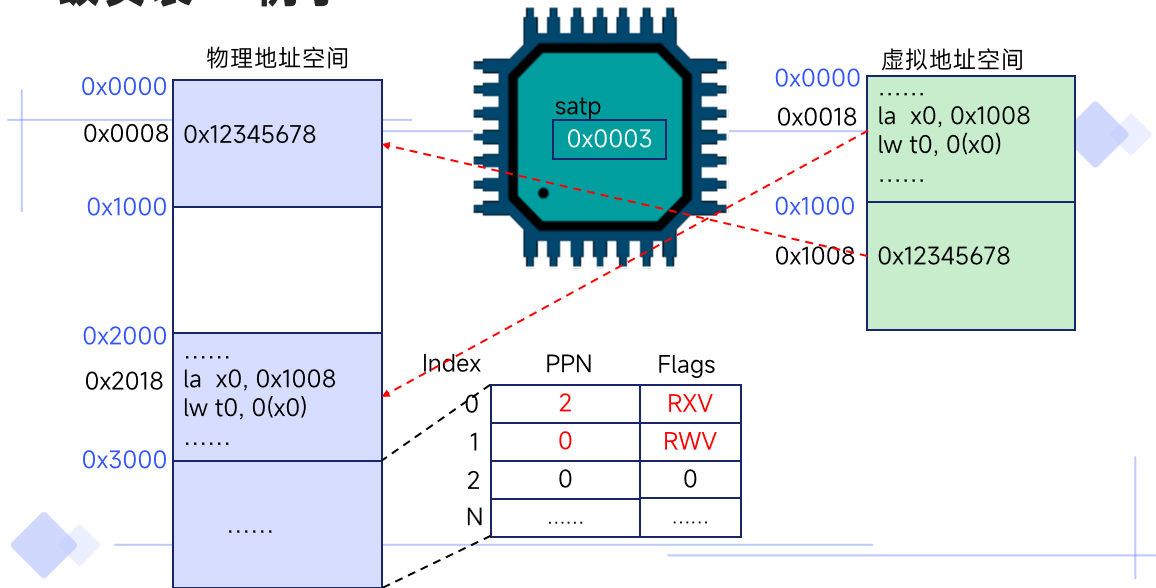


# 一级页表 - 例子

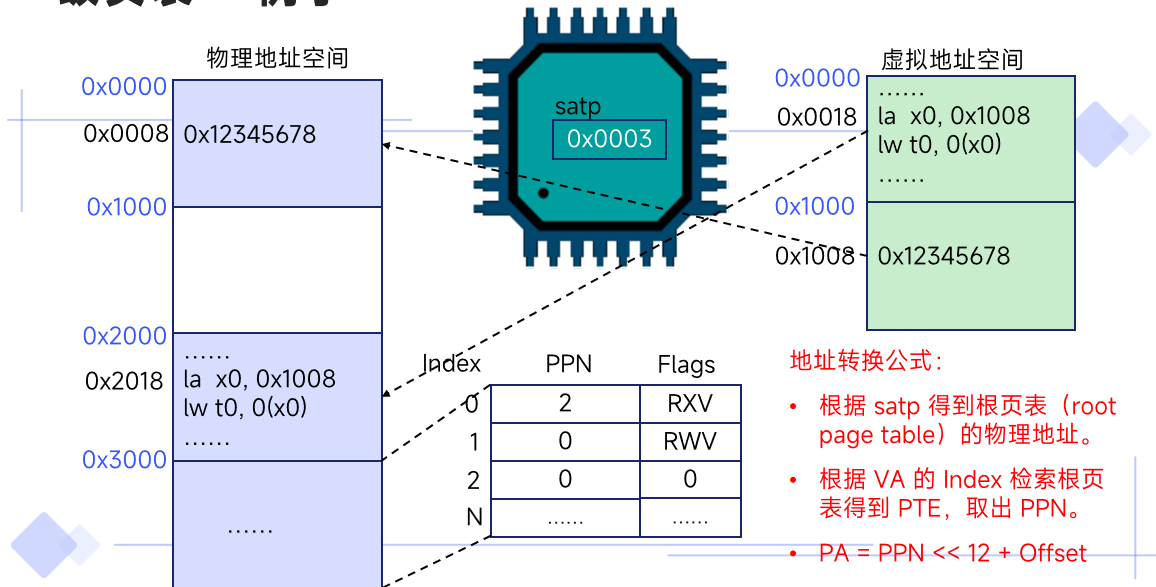




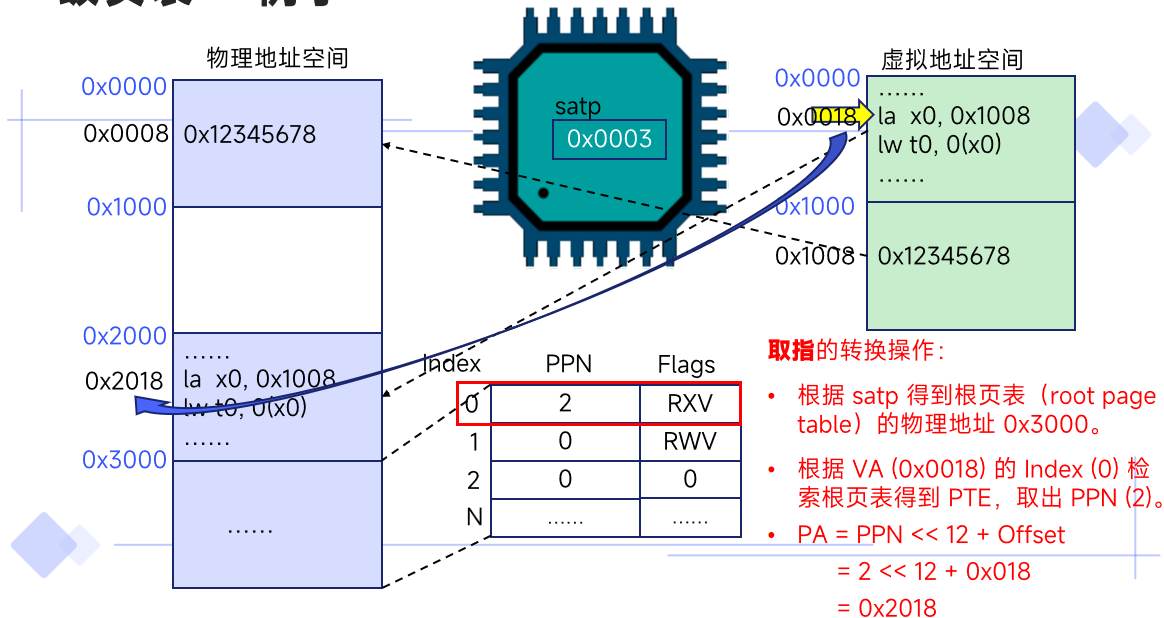
# 一级页表 - 例子



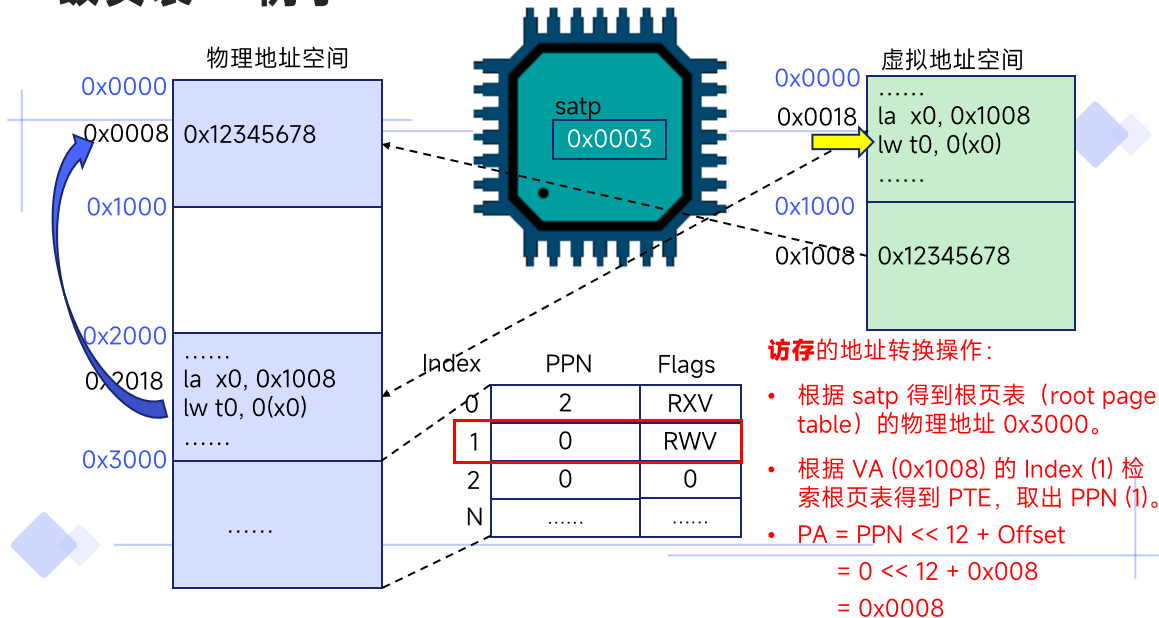
# 一级页表 - 例子



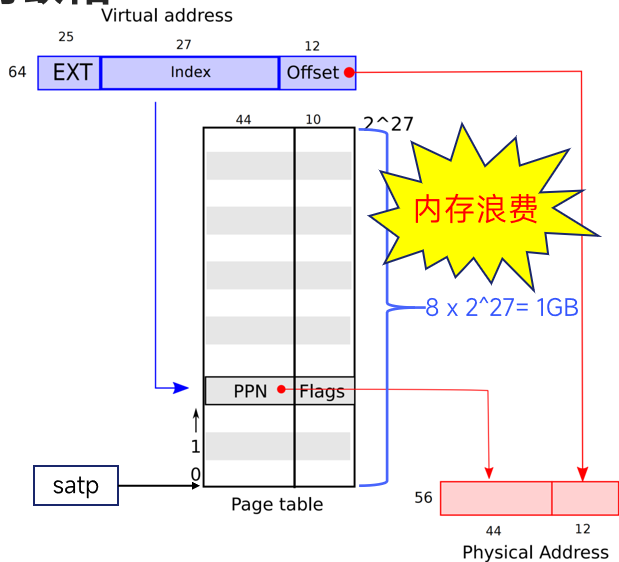
# 一级页表 - 例子



# 一级页表 - 例子



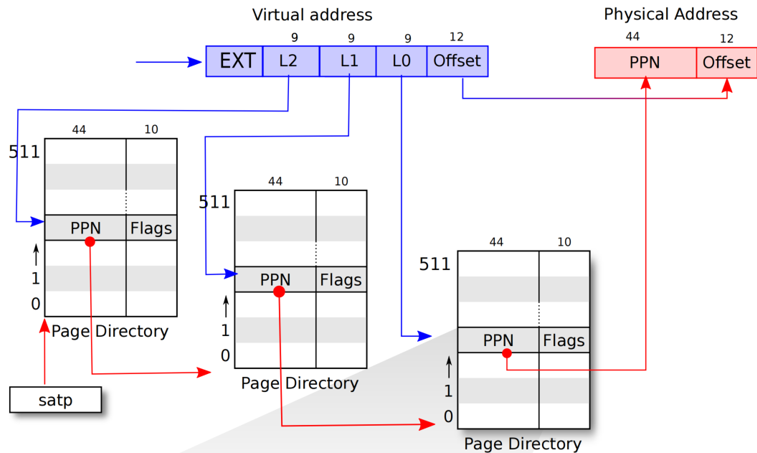
# 一级页表的缺陷



# RISC-V 页表

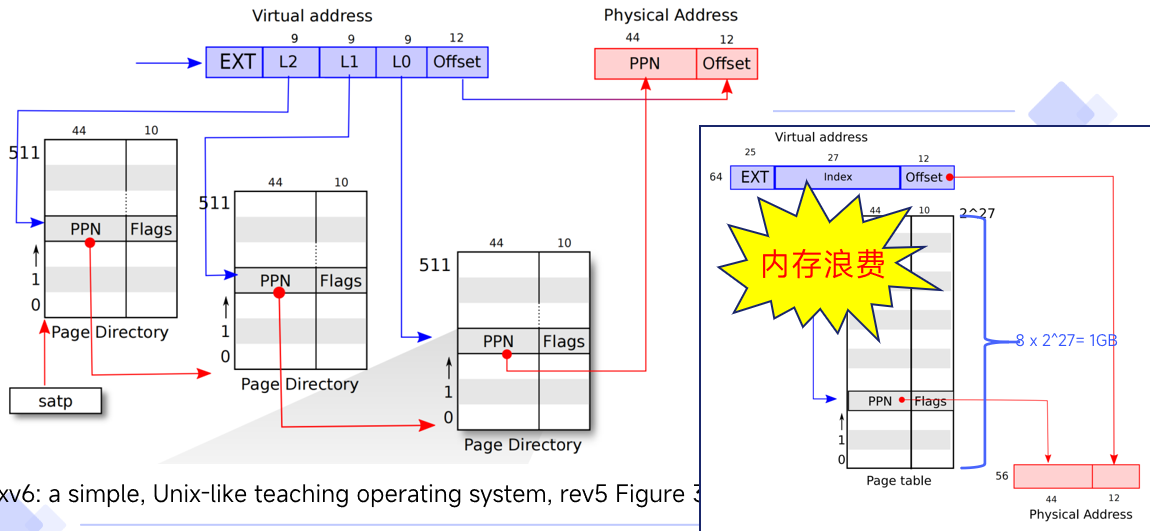
- 一级页表
- **三级页表**

# 三级页表



xv6: a simple, Unix-like teaching operating system, rev5 Figure 3.2

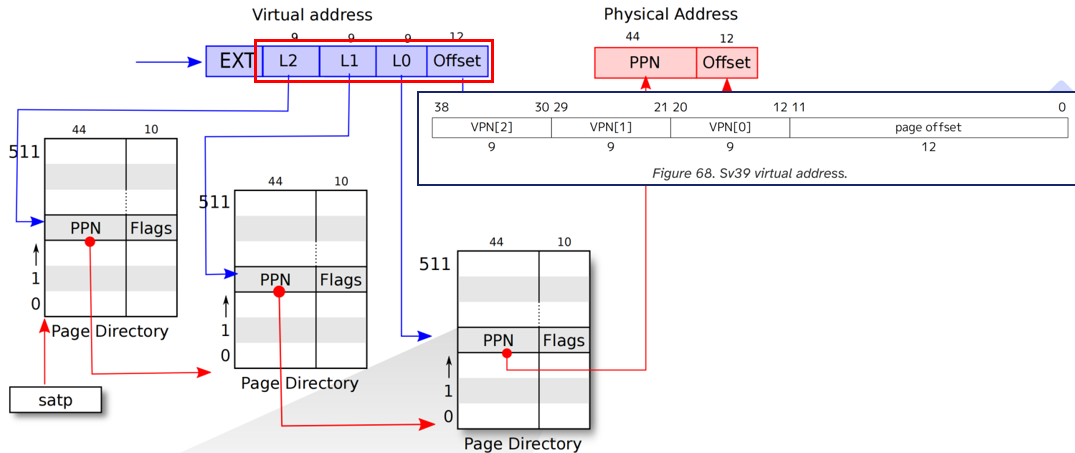
# 三级页表



xv6: a simple, Unix-like teaching operating system, rev5 Figure 3

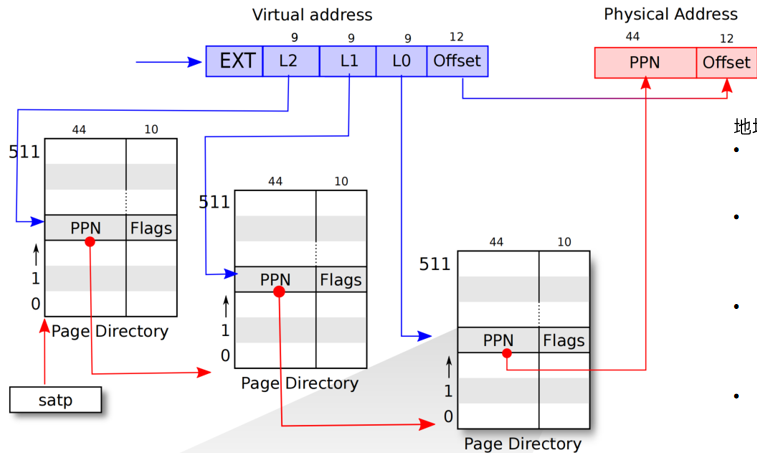


# 三级页表



xv6: a simple, Unix-like teaching operating system, rev5 Figure 3.2

# 三级页表

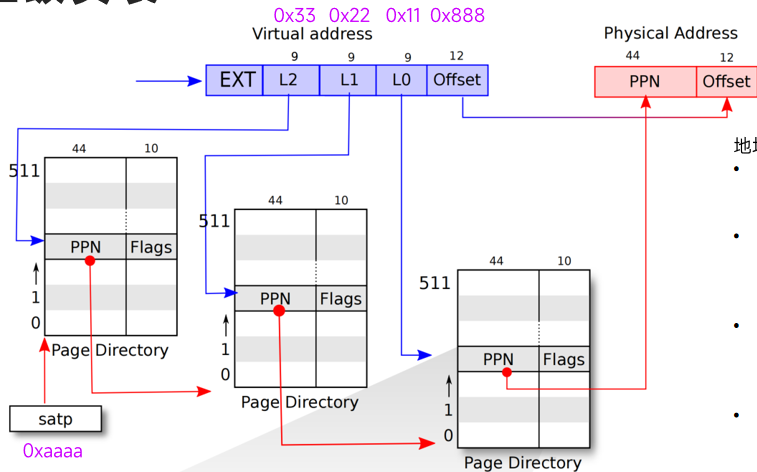


地址转换公式:

- 根据 `satp` 得到根页表所在页帧的 PPN。这是 L2 页表的 PA 的高 44 位，将其左移 12 位即得到 L2 页表对应的 PA。
- 根据 VA 中的 L2 检索 L2 页表得到 L2 页表中对应 PTE 的位置。取出 PTE 的 PPN, 这是 L1 页表的 PA 的高 44 位，将其左移 12 位即得到 L1 页表对应的 PA。
- 根据 VA 中的 L1 检索 L1 页表得到 L1 页表中对应 PTE 的位置。取出 PTE 的 PPN, 这是 L0 页表的 PA 的高 44 位，将其左移 12 位即得到 L0 页表对应的 PA。
- 根据 VA 中的 L0 检索 L0 页表得到 L0 页表中对应 PTE 的位置。取出 PTE 的 PPN, 这是叶子页帧的 PA 的高 44 位，将其左移 12 位后和 VA 的 Offset 相加即得到 VA 对应的 PA。

xv6: a simple, Unix-like teaching operating system, rev5 Figure 3.2

# 三级页表

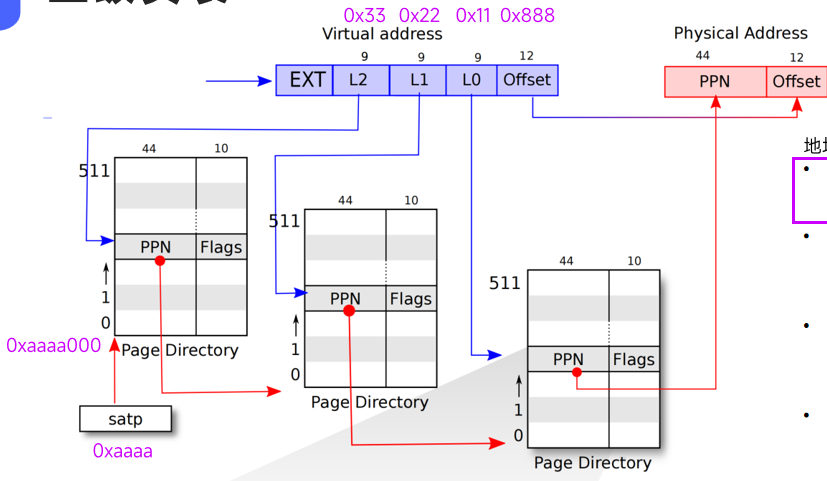


地址转换公式:

- 根据 satp 得到根页表所在页帧的 PPN。这是 L2 页表的 PA 的高 44 位, 将其左移 12 位即得到 L2 页表对应的 PA。
- 根据 VA 中的 L2 检索 L2 页表得到 L2 页表中对应 PTE 的位置。取出 PTE 的 PPN, 这是 L1 页表的 PA 的高 44 位, 将其左移 12 位即得到 L1 页表对应的 PA。
- 根据 VA 中的 L1 检索 L1 页表得到 L1 页表中对应 PTE 的位置。取出 PTE 的 PPN, 这是 L0 页表的 PA 的高 44 位, 将其左移 12 位即得到 L0 页表对应的 PA。
- 根据 VA 中的 L0 检索 L0 页表得到 L0 页表中对应 PTE 的位置。取出 PTE 的 PPN, 这是叶子页帧的 PA 的高 44 位, 将其左移 12 位后和 VA 的 Offset 相加即得到 VA 对应的 PA。

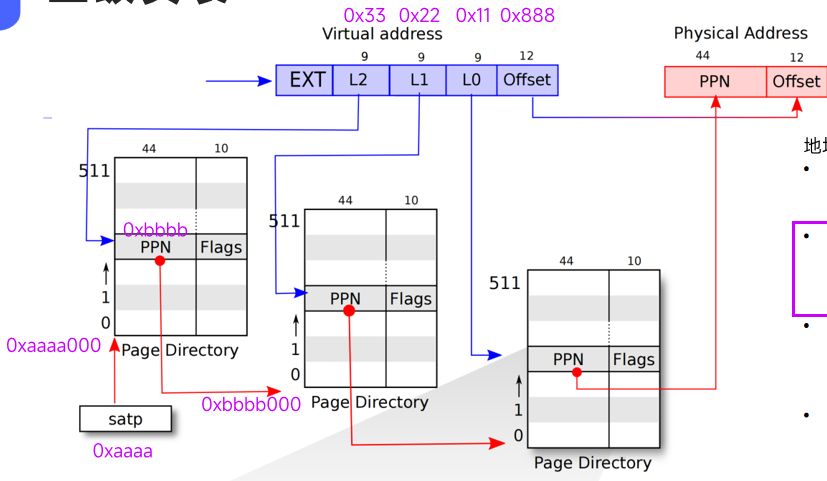
xv6: a simple, Unix-like teaching operating system, rev5 Figure 3.2

# 三级页表



xv6: a simple, Unix-like teaching operating system, rev5 Figure 3.2

# 三级页表

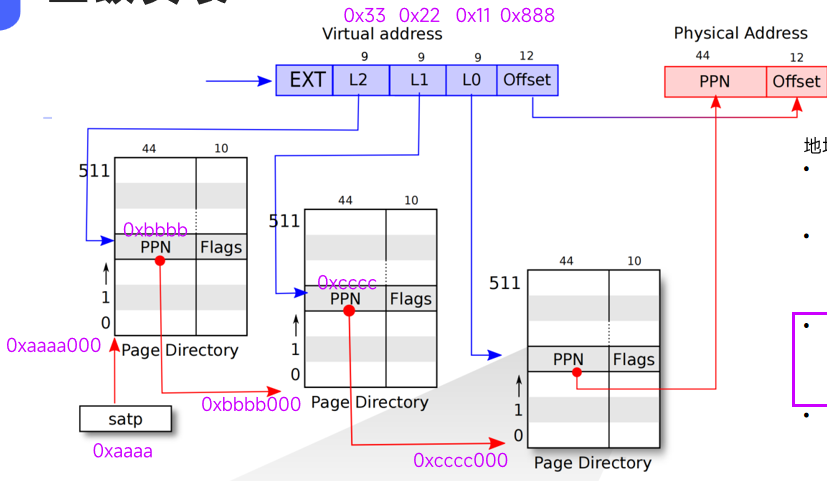


地址转换公式:

- 根据 satp 得到根页表所在页帧的 PPN。这是 L2 页表的 PA 的高 44 位, 将其左移 12 位即得到 L2 页表对应的 PA。
- 根据 VA 中的 L2 检索 L2 页表得到 L2 页表中对应 PTE 的位置。取出 PTE 的 PPN, 这是 L1 页表的 PA 的高 44 位, 将其左移 12 位即得到 L1 页表对应的 PA。
- 根据 VA 中的 L1 检索 L1 页表得到 L1 页表中对应 PTE 的位置。取出 PTE 的 PPN, 这是 L0 页表的 PA 的高 44 位, 将其左移 12 位即得到 L0 页表对应的 PA。
- 根据 VA 中的 L0 检索 L0 页表得到 L0 页表中对应 PTE 的位置。取出 PTE 的 PPN, 这是叶子页帧的 PA 的高 44 位, 将其左移 12 位后和 VA 的 Offset 相加即得到 VA 对应的 PA。

xv6: a simple, Unix-like teaching operating system, rev5 Figure 3.2

# 三级页表

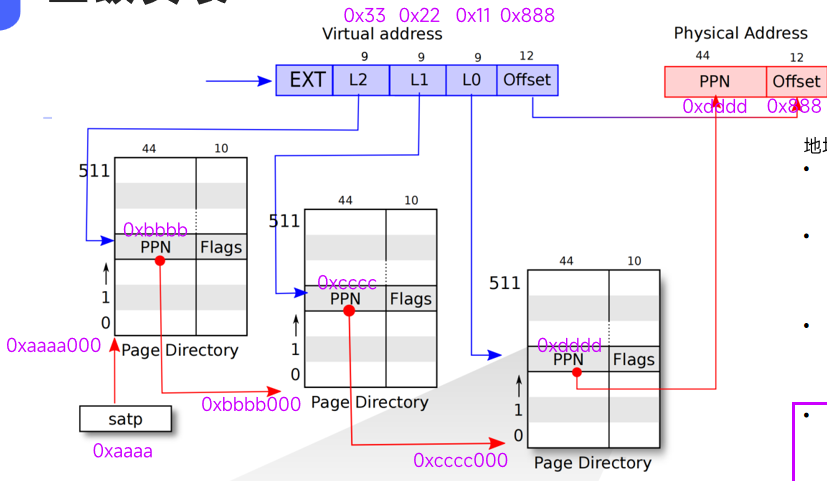


地址转换公式:

- 根据 satp 得到根页表所在页帧的 PPN。这是 L2 页表的 PA 的高 44 位, 将其左移 12 位即得到 L2 页表对应的 PA。
- 根据 VA 中的 L2 检索 L2 页表得到 L2 页表中对应 PTE 的位置。取出 PTE 的 PPN, 这是 L1 页表的 PA 的高 44 位, 将其左移 12 位即得到 L1 页表对应的 PA。
- 根据 VA 中的 L1 检索 L1 页表得到 L1 页表中对应 PTE 的位置。取出 PTE 的 PPN, 这是 L0 页表的 PA 的高 44 位, 将其左移 12 位即得到 L0 页表对应的 PA。
- 根据 VA 中的 L0 检索 L0 页表得到 L0 页表中对应 PTE 的位置。取出 PTE 的 PPN, 这是叶子页帧的 PA 的高 44 位, 将其左移 12 位后和 VA 的 Offset 相加即得到 VA 对应的 PA。

xv6: a simple, Unix-like teaching operating system, rev5 Figure 3.2

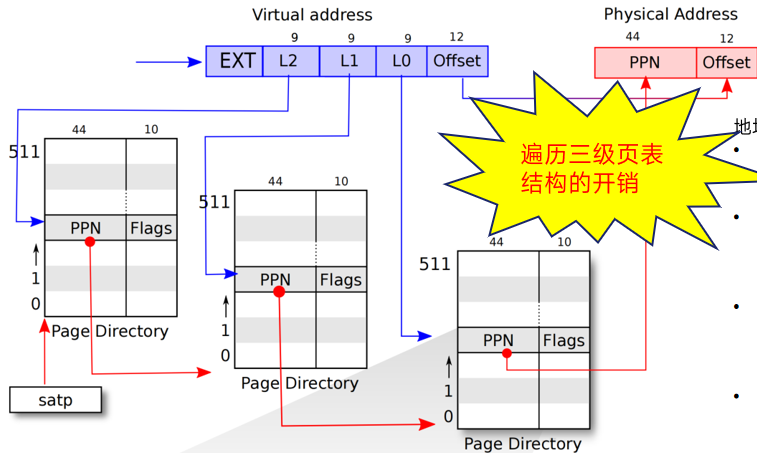
# 三级页表



地址转换公式:

- 根据 satp 得到根页表所在页帧的 PPN。这是 L2 页表的 PA 的高 44 位, 将其左移 12 位即得到 L2 页表对应的 PA。
- 根据 VA 中的 L2 检索 L2 页表得到 L2 页表中对应 PTE 的位置。取出 PTE 的 PPN, 这是 L1 页表的 PA 的高 44 位, 将其左移 12 位即得到 L1 页表对应的 PA。
- 根据 VA 中的 L1 检索 L1 页表得到 L1 页表中对应 PTE 的位置。取出 PTE 的 PPN, 这是 L0 页表的 PA 的高 44 位, 将其左移 12 位即得到 L0 页表对应的 PA。
- 根据 VA 中的 L0 检索 L0 页表得到 L0 页表中对应 PTE 的位置。取出 PTE 的 PPN, 这是叶子页帧的 PA 的高 44 位, 将其左移 12 位后和 VA 的 Offset 相加即得到 VA 对应的 PA。

# 三级页表



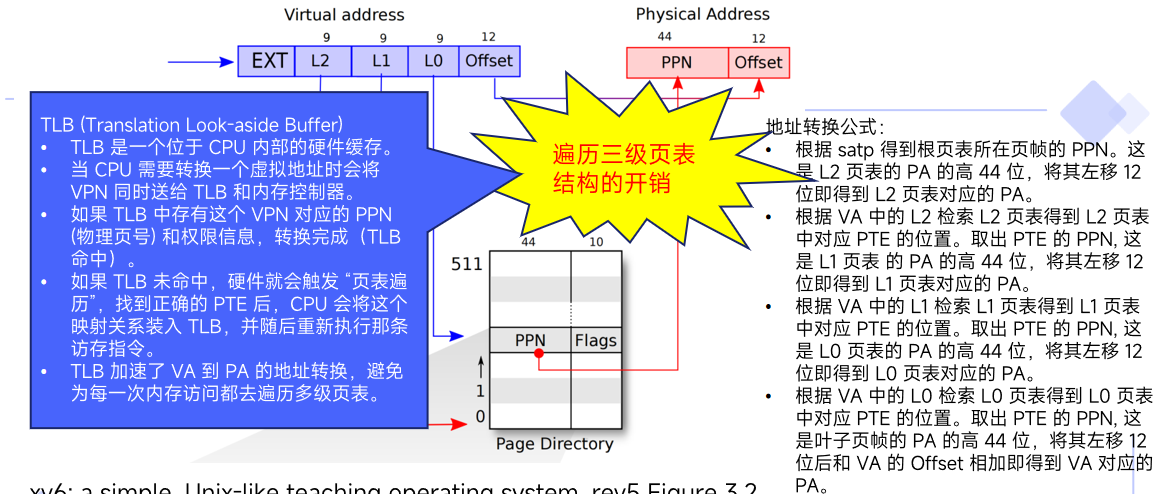
地址转换公式:

- 根据 **satp** 得到根页表所在页帧的 PPN。这是 L2 页表的 PA 的高 44 位，将其左移 12 位即得到 L2 页表对应的 PA。
- 根据 VA 中的 L2 检索 L2 页表得到 L2 页表中对应 PTE 的位置。取出 PTE 的 PPN, 这是 L1 页表的 PA 的高 44 位，将其左移 12 位即得到 L1 页表对应的 PA。
- 根据 VA 中的 L1 检索 L1 页表得到 L1 页表中对应 PTE 的位置。取出 PTE 的 PPN, 这是 L0 页表的 PA 的高 44 位，将其左移 12 位即得到 L0 页表对应的 PA。
- 根据 VA 中的 L0 检索 L0 页表得到 L0 页表中对应 PTE 的位置。取出 PTE 的 PPN, 这是叶子页帧的 PA 的高 44 位，将其左移 12 位后和 VA 的 Offset 相加即得到 VA 对应的 PA。

xv6: a simple, Unix-like teaching operating system, rev5 Figure 3.2



# 三级页表



xv6: a simple, Unix-like teaching operating system, rev5 Figure 3.2

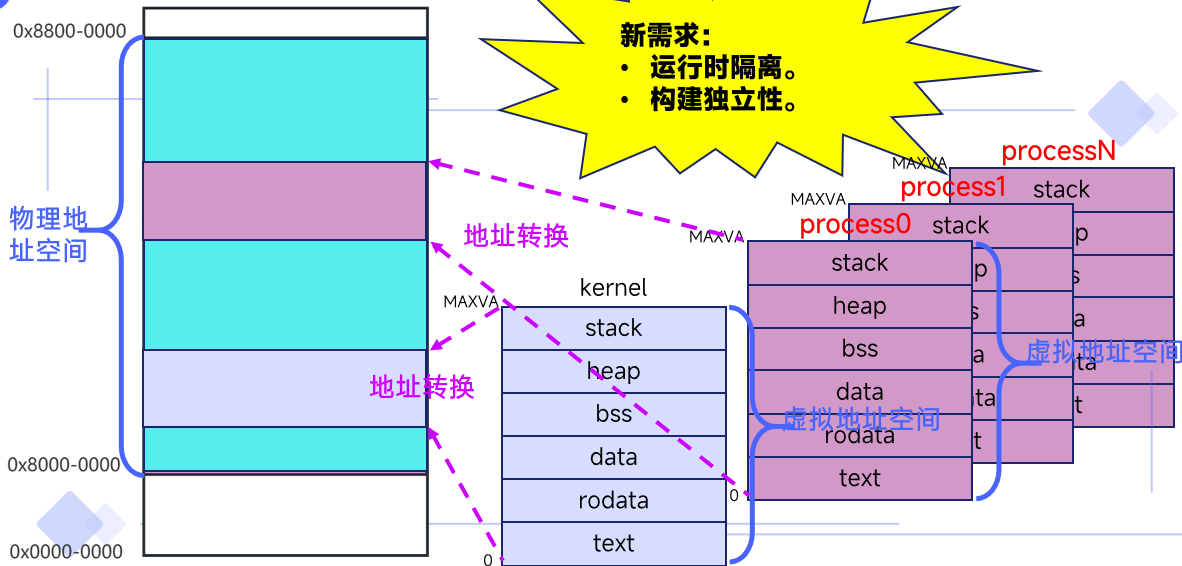


04

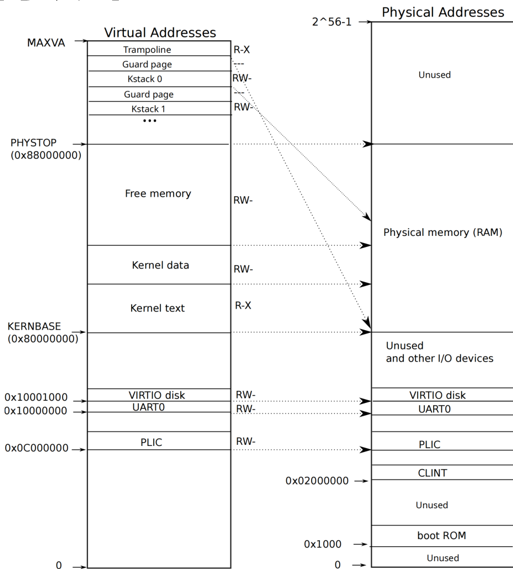
## xv6 的内核地址空间设计

---

# 地址空间

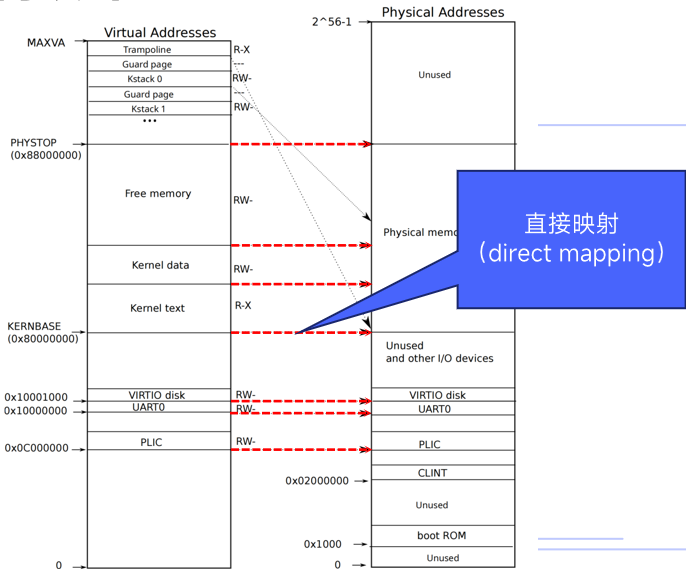


# 内核地址空间设计

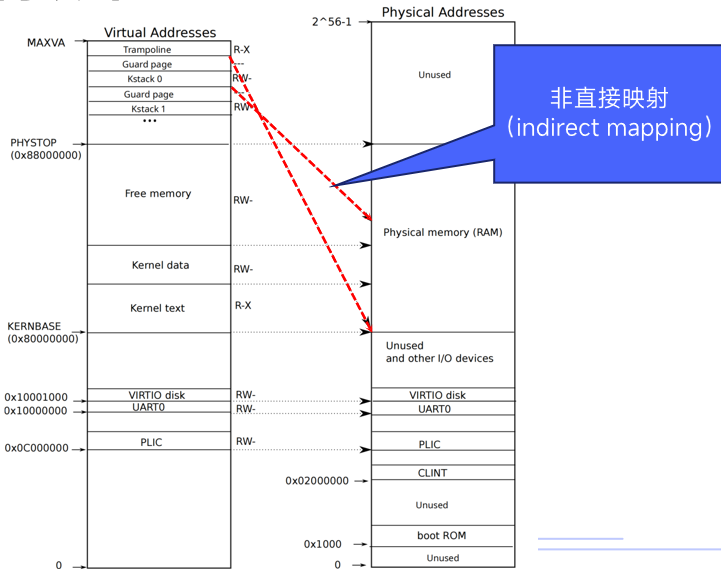


xv6: a simple, Unix-like teaching operating system, rev5 Figure 3.3

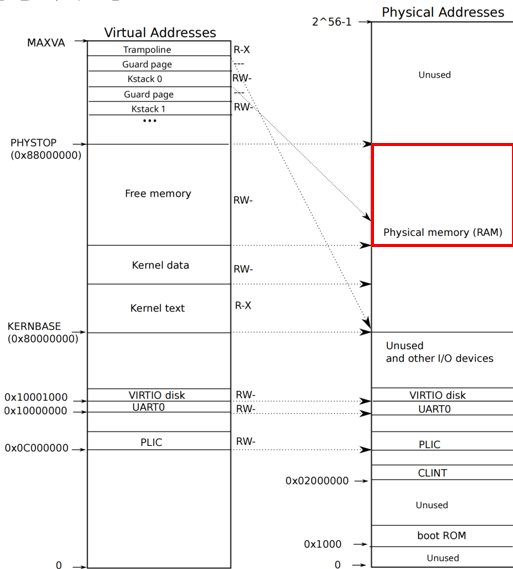
# 内核地址空间设计



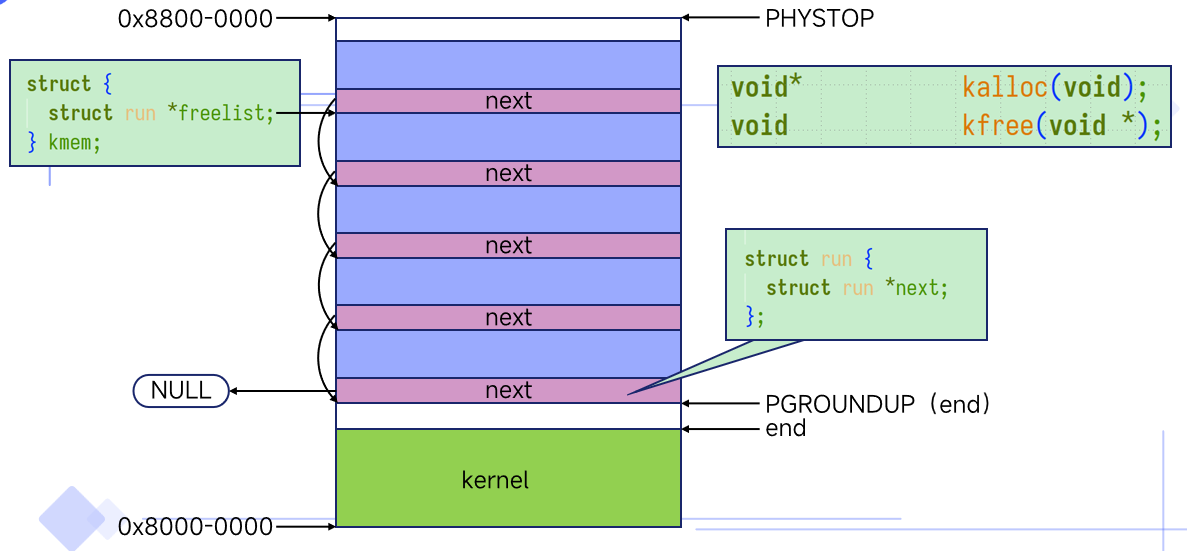
# 内核地址空间设计



# 内核地址空间设计



# 内核地址空间设计





# 内核地址空间设计 - 初始化

```
volatile static int started = 0;
void main()
{
    if(cpuid() == 0){
        // .....
        kinit();           // physical page allocator
        kvminit();         // create kernel page table
        kminithart();      // turn on paging
        // .....
        started = 1;
    } else {
        while(started == 0)
            ;
        kminithart();      // turn on paging
        // .....
    }
    scheduler();
}
```

```
pagetable_t kernel_pagetable;

// Initialize the kernel_pagetable, shared by all CPUs.
void kvminit(void)
{
    kernel_pagetable = kvmmake();
}

// Switch the current CPU's h/w page table register to
// the kernel's page table, and enable paging.
void kminithart()
{
    // wait for any previous writes to the page table memory to finish.
    sfence_vma();
    w_satp(MAKE_SATP(kernel_pagetable));
    // flush stale entries from the TLB.
    sfence_vma();
}
```

# 内核地址空间设计 - 初始化

```
volatile static int started = 0;
void main()
{
    if(cpuid() == 0){
        // .....
        kinit();           // physical page allocator
        kvminit();         // create kernel page table
        kvmminithart();    // turn on paging
        // .....
        started = 1;
    } else {
        while(started == 0)
            ;
        kvmminithart();    // turn on paging
        // .....
    }
    scheduler();
}
```

```
pagetable_t kernel_pagetable;

// Initialize the kernel_pagetable, shared by all CPUs.
void kvminit(void)
{
    kernel_pagetable = kvmmake();
}

// Switch the current CPU's h/w page table register to
// the kernel's page table, and enable paging.
void kvmminithart()
{
    // wait for any previous writes to the page table memory to finish.
    sfence_vma();
    w_satp(MAKE_SATP(kernel_pagetable));
    // flush stale entries from the TLB.
    sfence_vma();
}
```

# 内核地址空间设计 - 初始化

```
volatile static int started = 0;
void main()
{
    if(cpuid() == 0){
        // .....
        kinit();           // physical page allocator
        kvminit();         // create kernel page table
        kvmminithart();    // turn on paging
        // .....
        started = 1;
    } else {
        while(started == 0)
            ;
        kvmminithart();    // turn on paging
        // .....
    }
    scheduler();
}
```

```
pagetable_t kernel_pagetable;
```

```
// Initialize the kernel_pagetable, shared by all CPUs.
```

```
void kvminit(void)
```

```
{
```

```
    kernel_pagetable = kvmmake();
```

```
}
```

```
// Switch the current CPU's h/w page table register to
// the kernel's page table, and enable paging.
```

```
void kvmminithart()
```

```
{
```

```
    // wait for any previous writes to the page table memory to finish.
```

```
    sfence_vma();
```

```
    w_satp(MAKE_SATP(kernel_pagetable));
```

```
    // flush stale entries from the TLB.
```

```
    sfence_vma();
```

```
}
```

# 内核地址空间设计 - 初始化

```
volatile static int started = 0;
void main()
{
    if(cpuid() == 0){
        // .....
        kinit();          // physical page allocator
        kvminit();        // create kernel page table
        kvminithart();    // turn on paging
        // .....
        started = 1;
    } else {
        while(started == 0)
            ;
        kvminithart();    // turn on paging
        // .....
    }
    scheduler();
}
```

```
pagetable_t kernel_pagetable;

// Initialize the kernel_pagetable, shared by all CPUs.
void kvminit(void)
{
    kernel_pagetable = kvmmake();
}

// Switch the current CPU's h/w page table register to
// the kernel's page table, and enable paging.
void kvminithart()
{
    // wait for any previous writes to the page table memory to finish.
    sfence_vma();
    w_satp(MAKE_SATP(kernel_pagetable));
    // flush stale entries from the TLB.
    sfence_vma();
}
```

# 内核地址空间设计 - 初始化

```
volatile static int started = 0;
void main()
{
    if(cpuid() == 0){
        // .....
        kinit();          // physical page allocator
        kvmnit();          // create kernel page table
        kvmnithart();      // turn on paging
        // .....
        started = 1;
    } else {
        while(started == 0)
            ;
        kvmnithart();      // turn on paging
        // .....
    }
    scheduler();
}
```

```
pagetable_t kernel_pagetable;

// Initialize the kernel_pagetable, shared by all CPUs.
void kvmnit(void)
{
    kernel_pagetable = kvmmake();
}

static inline void sfence_vma()
{
    // the zero, zero means flush all TLB entries.
    asm volatile("sfence.vma zero, zero");
}

// wait for any previous writes to the page table memory to finish.
sfence_vma();
w_satp(MAKE_SATP(kernel_pagetable));
// flush stale entries from the TLB.
sfence_vma();
}
```

# 内核地址空间设计 - 初始化

```
volatile static int started = 0;
void main()
{
    if(cpuid() == 0){
        // .....
        kinit();          // physical page allocator
        kvminit();         // create kernel page table
        kvminithart();     // turn on paging
        // .....
        started = 1;
    } else {
        while(started == 0)
            ;
        kvminithart();     // turn on paging
        // .....
    }
    scheduler();
}
```

```
pagetable_t kernel_pagetable;
```

```
// Initialize the
```

```
void kvminit(void)
```

```
{
```

```
    kernel_pagetable
```

```
}
```

```
static inline
```

```
{
```

```
// Sw
```

```
// th
```

```
void
```

```
{
```

```
    // wait for any previous writes to the page table memory to finish.
```

```
    sfence_vma();
```

```
    w_satp(MAKE_SATP(kernel_pagetable));
```

```
    // flush stale entries from the TLB.
```

```
    sfence_vma();
```

```
}
```

SFENCE.VMA: supervisor memory-management fence

- 强制所有在 sfence.vma 之前发起的针对页表的写入 (Store) 操作生效;
- 根据参数 (rs1 和 rs2) 使 TLB 条目失效
- sfence.vma rs1, rs2

# 内核地址空间设计 - 初始化

```
volatile static int started = 0;
void main()
{
    if(cpuid() == 0){
        // .....
        kinit();          // physical page allocator
        kvminit();         // create kernel page table
        kvminithart();     // turn on paging
        // .....
        started = 1;
    } else {
        while(started == 0)
            ;
        kvminithart();    // turn on paging
        // .....
    }
    scheduler();
}
```

```
pagetable_t kernel_pagetable;
```

```
// Initialize the
```

```
void kvminit(void)
```

```
{
```

```
    kernel_pagetable
```

```
}
```

```
static inline
```

```
{
```

```
// Sw
```

```
// th
```

```
void
```

```
{
```

```
    // wait for any p
```

```
    sfence_vma();
```

```
    w_satp(MAKE_SATP(kernel_pagetable));
```

```
    // flush stale entries from the TLB.
```

```
    sfence_vma();
```

```
}
```

SFENCE.VMA: supervisor memory-management fence

- 强制所有在 sfence.vma 之前发起的针对页表的写入 (Store) 操作生效;
- 根据参数 (rs1 和 rs2) 使 TLB 条目失效
- sfence.vma rs1, rs2

确保之前的页表写操作完成, 并使得 TLB 缓存条目失效

# 内核地址空间设计 - 初始化

```
volatile static int started = 0;
void main()
{
    if(cpuid() == 0){
        // .....
        kinit();          // physical page allocator
        kvminit();         // create kernel page table
        kvminithart();     // turn on paging
        // .....
        started = 1;
    } else {
        while(started == 0)
            ;
        kvminithart();    // turn on paging
        // .....
    }
    scheduler();
}
```

```
pagetable_t kernel_pagetable;
```

```
// Initialize the
```

```
void kvminit(void)
```

```
{
```

```
    kernel_pagetable
```

```
}
```

```
static inline
```

```
{
```

```
// Sw
```

```
// th
```

```
void
```

```
{
```

```
    // wait for any p
```

```
    sfence_vma();
```

```
    w_satp(MAKE_SATP(kernel_pagetable));
```

```
    // flush stale entries from the TLB.
```

```
    sfence_vma();
```

```
}
```

SFENCE.VMA: supervisor memory-management fence

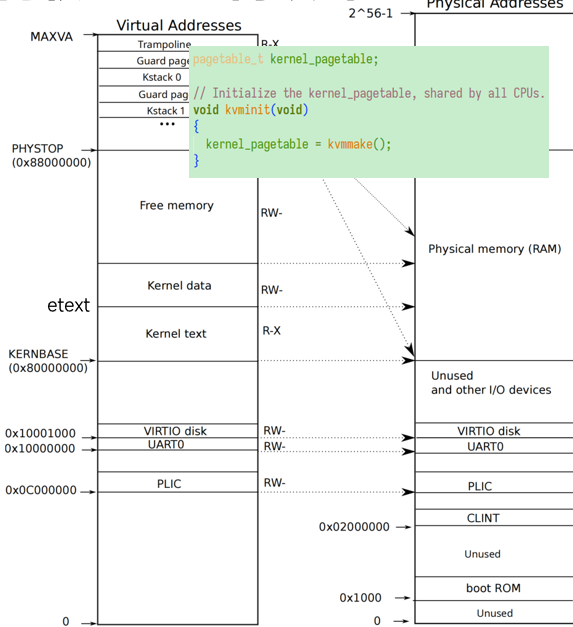
- 强制所有在 sfence.vma 之前发起的针对页表的写入 (Store) 操作生效;
- 根据参数 (rs1 和 rs2) 使 TLB 条目失效
- sfence.vma rs1, rs2

确保之前的页表写操作完成, 并使得 TLB 缓存条目失效

清空切换瞬间可能产生的残留或无效映射, 确保后续的地址翻译工作在一个完全干净可靠的状态下。



# 内核地址空间设计 - 初始化



```

// Make a direct-map page table for the kernel.
pagetable_t kvmmake(void)
{
    pagetable_t kpgtbl;

    kpgtbl = (pagetable_t) kalloc();
    memset(kpgtbl, 0, PGSIZE);

    // uart registers
    kvmmap(kpgtbl, UART0, UART0, PGSIZE, PTE_R | PTE_W);

    // virtio mmio disk interface
    kvmmap(kpgtbl, VIRTIO0, VIRTIO0, PGSIZE, PTE_R | PTE_W);

    // PLIC
    kvmmap(kpgtbl, PLIC, PLIC, 0x4000000, PTE_R | PTE_W);

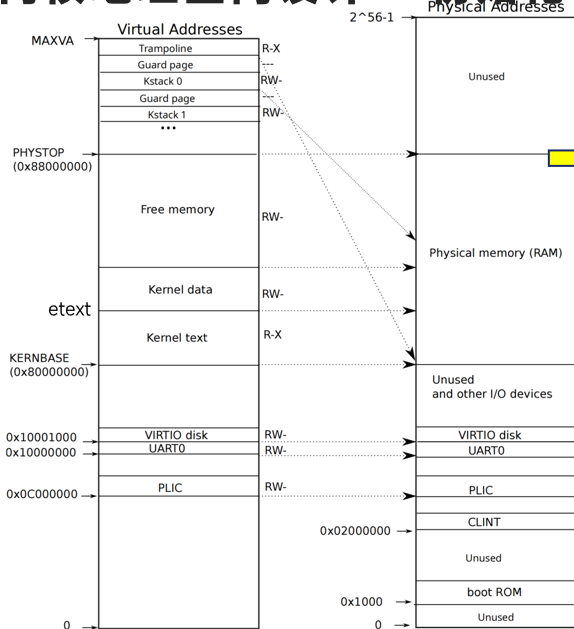
    // map kernel text executable and read-only.
    kvmmap(kpgtbl, KERNBASE, KERNBASE, (uint64)etext-KERNBASE, PTE_R | PTE_X);

    // map kernel data and the physical RAM we'll make use of.
    kvmmap(kpgtbl, (uint64)etext, (uint64)etext, PHYSTOP-(uint64)etext, PTE_R | PTE_W);

    // allocate and map a kernel stack for each process.
    proc_mapstacks(kpgtbl);

    return kpgtbl;
}
    
```

# 内核地址空间设计 - 初始化



```
// Make a direct-map page table for the kernel.
pagetable_t kvmmake(void)
{
    pagetable_t kpgtbl;

    kpgtbl = (pagetable_t) kalloc();
    memset(kpgtbl, 0, PGSIZE);

    // uart registers
    kvmmap(kpgtbl, UART0, UART0, PGSIZE, PTE_R | PTE_W);

    // virtio mmio disk interface
    kvmmap(kpgtbl, VIRTIO0, VIRTIO0, PGSIZE, PTE_R | PTE_W);

    // PLIC
    kvmmap(kpgtbl, PLIC, PLIC, 0x4000000, PTE_R | PTE_W);

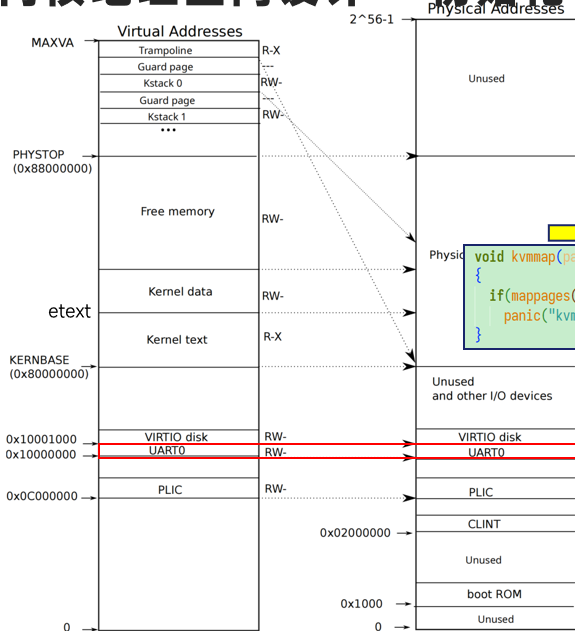
    // map kernel text executable and read-only.
    kvmmap(kpgtbl, KERNBASE, KERNBASE, (uint64)etext-KERNBASE, PTE_R | PTE_X);

    // map kernel data and the physical RAM we'll make use of.
    kvmmap(kpgtbl, (uint64)etext, (uint64)etext, PHYSTOP-(uint64)etext, PTE_R | PTE_W);

    // allocate and map a kernel stack for each process.
    proc_mapstacks(kpgtbl);

    return kpgtbl;
}
```

# 内核地址空间设计 - 初始化



```
// Make a direct-map page table for the kernel.
```

```
pagetable_t kvmmake(void)
```

```
{
```

```
    pagetable_t kpgtbl;
```

```
    kpgtbl = (pagetable_t) kalloc();
```

```
    memset(kpgtbl, 0, PGSIZE);
```

```
    // uart registers
```

```
    kvmmap(kpgtbl, UART0, UART0, PGSIZE, PTE_R | PTE_W);
```

```
void kvmmap(pagetable_t kpgtbl, uint64 va, uint64 pa, uint64 sz, int perm)
```

```
{
```

```
    if(mappages(kpgtbl, va, sz, pa, perm) != 0)
```

```
        panic("kvmmap");
```

```
}
```

```
// map kernel text executable and read-only.
```

```
kvmmap(kpgtbl, KERNBASE, KERNBASE, (uint64)etext-KERNBASE, PTE_R | PTE_X);
```

```
// map kernel data and the physical RAM we'll make use of.
```

```
kvmmap(kpgtbl, (uint64)etext, (uint64)etext, PHYSTOP-(uint64)etext, PTE_R | PTE_W);
```

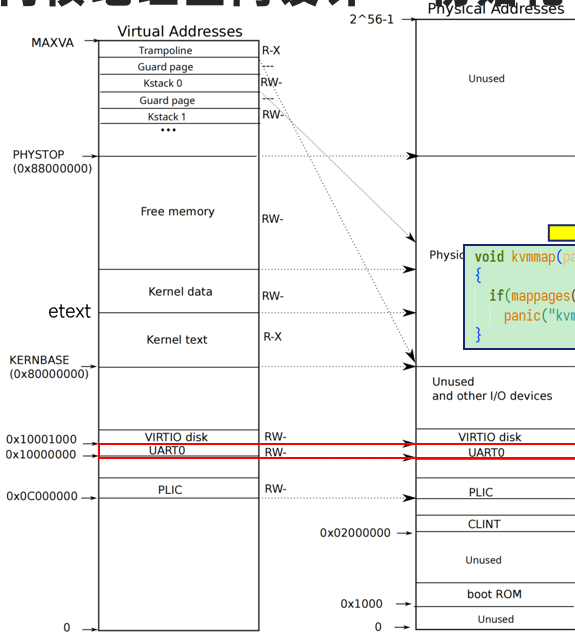
```
// allocate and map a kernel stack for each process.
```

```
proc_mapstacks(kpgtbl);
```

```
    return kpgtbl;
```

```
}
```

# 内核地址空间设计 - 初始化



```
// Make a direct-map page table for the kernel.
```

```
pagetable_t kvmmake(void)
```

```
{  
    pagetable_t kpgtbl;
```

```
    kpgtbl = (pagetable_t) kalloc();
```

```
    memset(kpgtbl, 0, PGSIZE);
```

```
    // uart registers
```

```
    kvmmap(kpgtbl, UART0, UART0, PGSIZE, PTE_R | PTE_W);
```

```
void kvmmap(pagetable_t kpgtbl, uint64 va, uint64 pa, uint64 sz, int perm)  
{  
    if(mappages(kpgtbl, va, sz, pa, perm) != 0)  
        panic("kvmmap");  
}
```

```
// map kernel text e
```

```
kvmmap(kpgtbl, KERNE
```

```
// map kernel data a
```

```
kvmmap(kpgtbl, (uint
```

```
// allocate and map
```

```
proc_mapstacks(kpgtbl
```

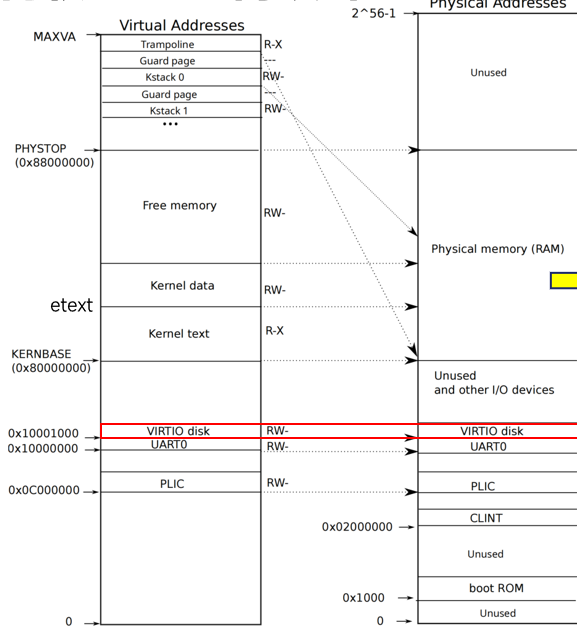
```
return kpgtbl;
```

```
}
```

- kpgtbl: kernel page table
- va: virtual address
- pa: physical address
- sz: size
- perm: permission

在指定页表 (pagetable) 中将虚拟地址 va 起始的一段大小为 size 的地址空间映射到物理地址 pa 处, 同时根据指定的 perm 设置这段地址空间的访问权限。

# 内核地址空间设计 - 初始化



```
// Make a direct-map page table for the kernel.
pagetable_t kvmmake(void)
{
    pagetable_t kpgtbl;

    kpgtbl = (pagetable_t) kalloc();
    memset(kpgtbl, 0, PGSIZE);

    // uart registers
    kvmmap(kpgtbl, UART0, UART0, PGSIZE, PTE_R | PTE_W);

    // virtio mmio disk interface
    kvmmap(kpgtbl, VIRTIO0, VIRTIO0, PGSIZE, PTE_R | PTE_W);

    // PLIC
    kvmmap(kpgtbl, PLIC, PLIC, 0x4000000, PTE_R | PTE_W);

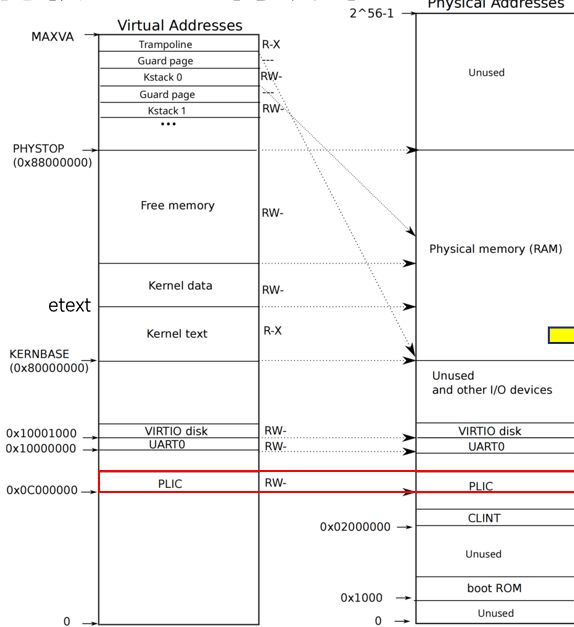
    // map kernel text executable and read-only.
    kvmmap(kpgtbl, KERNBASE, KERNBASE, (uint64)etext-KERNBASE, PTE_R | PTE_X);

    // map kernel data and the physical RAM we'll make use of.
    kvmmap(kpgtbl, (uint64)etext, (uint64)etext, PHYSTOP-(uint64)etext, PTE_R | PTE_W);

    // allocate and map a kernel stack for each process.
    proc_mapstacks(kpgtbl);

    return kpgtbl;
}
```

# 内核地址空间设计 - 初始化



```
// Make a direct-map page table for the kernel.
pagetable_t kvmmake(void)
{
    pagetable_t kpgtbl;

    kpgtbl = (pagetable_t) kalloc();
    memset(kpgtbl, 0, PGSIZE);

    // uart registers
    kvmmap(kpgtbl, UART0, UART0, PGSIZE, PTE_R | PTE_W);

    // virtio mmio disk interface
    kvmmap(kpgtbl, VIRTIO0, VIRTIO0, PGSIZE, PTE_R | PTE_W);

    // PLIC
    kvmmap(kpgtbl, PLIC, PLIC, 0x4000000, PTE_R | PTE_W);

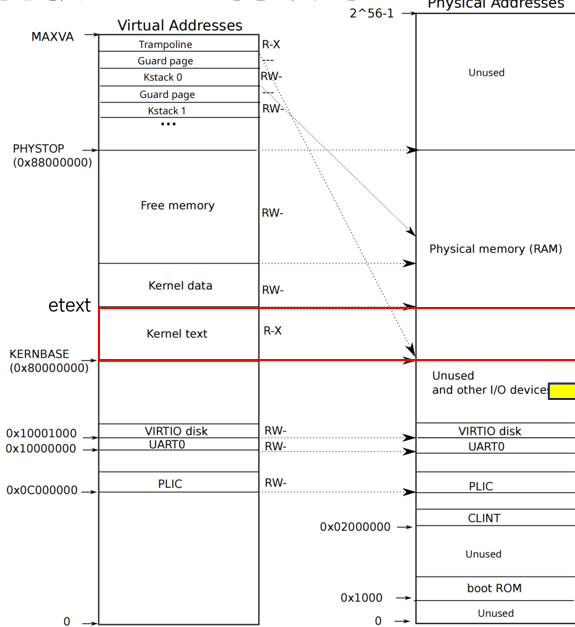
    // map kernel text executable and read-only.
    kvmmap(kpgtbl, KERNBASE, KERNBASE, (uint64)etext-KERNBASE, PTE_R | PTE_X);

    // map kernel data and the physical RAM we'll make use of.
    kvmmap(kpgtbl, (uint64)etext, (uint64)etext, PHYSTOP-(uint64)etext, PTE_R | PTE_W);

    // allocate and map a kernel stack for each process.
    proc_mapstacks(kpgtbl);

    return kpgtbl;
}
```

# 内核地址空间设计 - 初始化



```
// Make a direct-map page table for the kernel.
```

```
pagetable_t kvmmake(void)
```

```
{
    pagetable_t kpgtbl;
```

```
    kpgtbl = (pagetable_t) kalloc();
    memset(kpgtbl, 0, PGSIZE);
```

```
    // uart registers
```

```
    kvmmap(kpgtbl, UART0, UART0, PGSIZE, PTE_R | PTE_X);
```

```
    // virtio mmio disk interface
```

```
    kvmmap(kpgtbl, VIRTIO0, VIRTIO0, PGSIZE, PTE_R | PTE_X);
```

```
    // PLIC
```

```
    kvmmap(kpgtbl, PLIC, PLIC, 0x4000000, PTE_R | PTE_X);
```

```
    // map kernel text executable and read-only.
```

```
    kvmmap(kpgtbl, KERNBASE, KERNBASE, (uint64)etext-KERNBASE, PTE_R | PTE_X);
```

```
    // map kernel data and the physical RAM we'll make use of.
```

```
    kvmmap(kpgtbl, (uint64)etext, (uint64)etext, PHYSTOP-(uint64)etext, PTE_R | PTE_W);
```

```
    // allocate and map a kernel stack for each process.
```

```
    proc_mapstacks(kpgtbl);
```

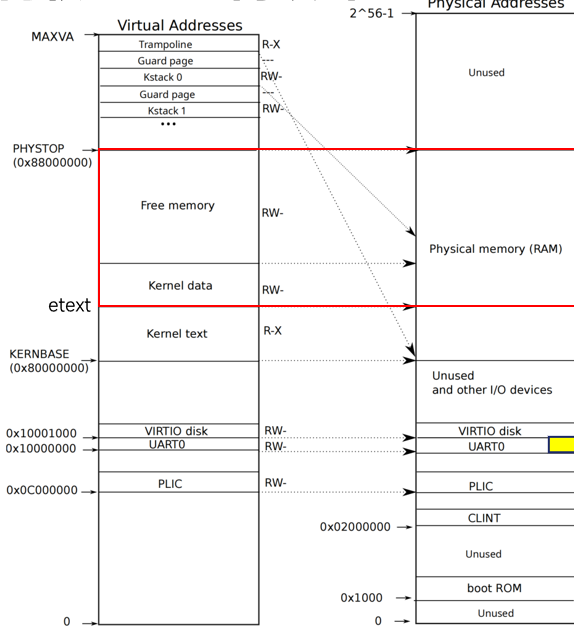
```
    return kpgtbl;
```

```
}
```

kernel/kernel.ld

```
.text : {
    kernel/entry.o(_entry)
    *(.text .text.*)
    . = ALIGN(0x1000);
    PROVIDE(etext = .);
}
```

# 内核地址空间设计 - 初始化



```
// Make a direct-map page table for the kernel.
pagetable_t kvmmake(void)
{
    pagetable_t kpgtbl;

    kpgtbl = (pagetable_t) kalloc();
    memset(kpgtbl, 0, PGSIZE);

    // uart registers
    kvmmap(kpgtbl, UART0, UART0, PGSIZE, PTE_R | PTE_W);

    // virtio mmio disk interface
    kvmmap(kpgtbl, VIRTIO0, VIRTIO0, PGSIZE, PTE_R | PTE_W);

    // PLIC
    kvmmap(kpgtbl, PLIC, PLIC, 0x4000000, PTE_R | PTE_W);

    // map kernel text executable and read-only.
    kvmmap(kpgtbl, KERNBASE, KERNBASE, (uint64)etext-KERNBASE, PTE_R | PTE_X);

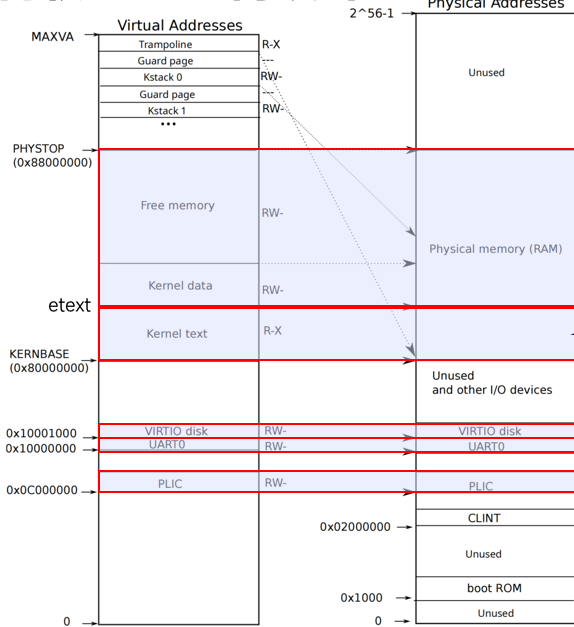
    // map kernel data and the physical RAM we'll make use of.
    kvmmap(kpgtbl, (uint64)etext, (uint64)etext, PHYSTOP-(uint64)etext, PTE_R | PTE_W);

    // allocate and map a kernel stack for each process.
    proc_mapstacks(kpgtbl);

    return kpgtbl;
}
```



# 内核地址空间设计 - 初始化



```
// Make a direct-map page table for the kernel.
```

```
pagetable_t kvmmake(void)
```

```
{
```

```
    pagetable_t kpgtbl;
```

```
    kpgtbl = (pagetable_t) kalloc();
```

```
    memset(kpgtbl, 0, PGSIZE);
```

```
    // uart registers
```

```
    kvmmap(kpgtbl, UART0, UART0, PGSIZE, PTE_R | PTE_W);
```

```
    // virt
```

```
    kvmmap
```

```
    //
```

```
    //
```

```
    // map kernel text executable and read-only.
```

```
    kvmmap(kpgtbl, KERNBASE, KERNBASE, (uint64)etext-KERNBASE, PTE_R | PTE_X);
```

```
    // map kernel data and the physical RAM we'll make use of.
```

```
    kvmmap(kpgtbl, (uint64)etext, (uint64)etext, PHYSTOP-(uint64)etext, PTE_R | PTE_W);
```

```
    // allocate and map a kernel stack for each process.
```

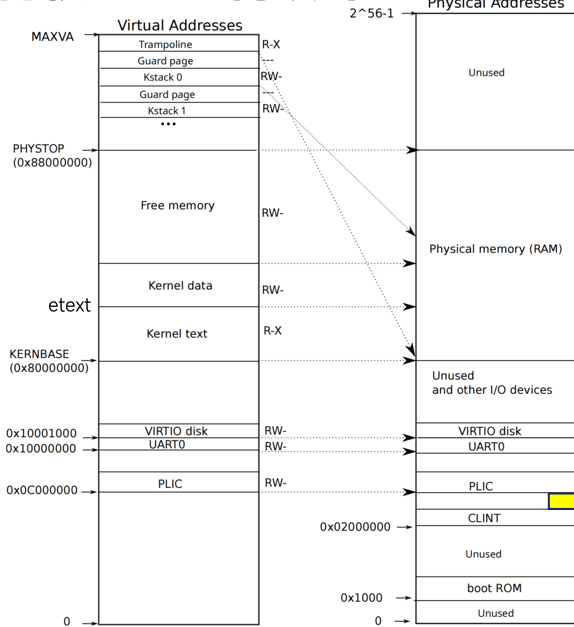
```
    proc_mapstacks(kpgtbl);
```

```
    return kpgtbl;
```

```
}
```

内核将所有物理 RAM 和设备寄存器映射到与物理地址相等的虚拟地址。这称为“直接映射 (direct mapping)”

# 内核地址空间设计 - 初始化



```
// Make a direct-map page table for the kernel.
pagetable_t kvmmake(void)
{
    pagetable_t kpgtbl;

    kpgtbl = (pagetable_t) kalloc();
    memset(kpgtbl, 0, PGSIZE);

    // uart registers
    kvmmap(kpgtbl, UART0, UART0, PGSIZE, PTE_R | PTE_W);

    // virtio mmio disk interface
    kvmmap(kpgtbl, VIRTIO0, VIRTIO0, PGSIZE, PTE_R | PTE_W);

    // PLIC
    kvmmap(kpgtbl, PLIC, PLIC, 0x4000000, PTE_R | PTE_W);

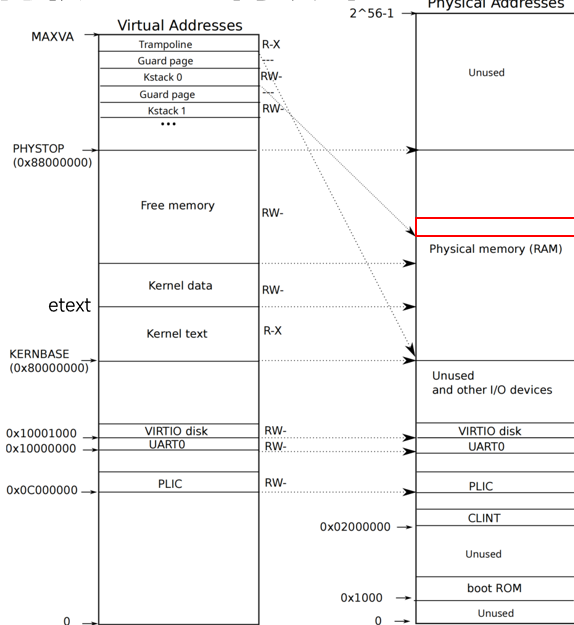
    // map kernel text executable and read-only.
    kvmmap(kpgtbl, KERNBASE, KERNBASE, (uint64)etext-KERNBASE, PTE_R | PTE_X);

    // map kernel data and the physical RAM we'll make use of.
    kvmmap(kpgtbl, (uint64)etext, (uint64)etext, PHYSTOP-(uint64)etext, PTE_R | PTE_W);

    // allocate and map a kernel stack for each process.
    proc_mapstacks(kpgtbl);

    return kpgtbl;
}
```

# 内核地址空间设计 - 初始化



```
// Make a direct-map page table for the kernel.
pagetable_t kvmmake(void)
{
    pagetable_t kpgtbl;

    kpgtbl = (pagetable_t) kalloc();
    memset(kpgtbl, 0, PGSIZE);

    // uart registers
    kvmmap(kpgtbl, UART0, UART0, PGSIZE, PTE_R | PTE_W);

    // virtio mmio disk interface
    kvmmap(kpgtbl, VIRTIO0, VIRTIO0, PGSIZE, PTE_R | PTE_W);

    // PLIC
    kvmmap(kpgtbl, PLIC, PLIC, 0x4000000, PTE_R | PTE_W);

    // map kernel text exec
    kvmmap(kpgtbl, KERNBASE, KERNBASE, 0x80000000, PTE_R | PTE_W);

    // map kernel data and
    kvmmap(kpgtbl, 0, 0, 0, PTE_R | PTE_W);

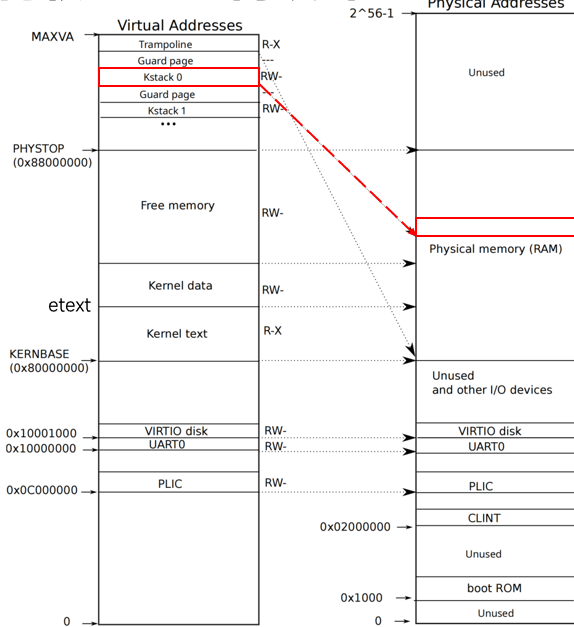
    // allocate and map a k
    proc_mapstacks(kpgtbl);

    return kpgtbl;
}
```

```
void proc_mapstacks(pagetable_t kpgtbl)
{
    struct proc *p;

    for(p = proc; p < &proc[NPROC]; p++) {
        char *pa = kalloc();
        if(pa == 0)
            panic("kalloc");
        uint64 va = KSTACK((int) (p - proc));
        kvmmap(kpgtbl, va, (uint64)pa, PGSIZE, PTE_R | PTE_W);
    }
}
```

# 内核地址空间设计 - 初始化



```
// Make a direct-map page table for the kernel.
pagetable_t kvmmake(void)
{
    pagetable_t kpgtbl;

    kpgtbl = (pagetable_t) kalloc();
    memset(kpgtbl, 0, PGSIZE);

    // uart registers
    kvmmap(kpgtbl, UART0, UART0, PGSIZE, PTE_R | PTE_W);

    // virtio mmio disk interface
    kvmmap(kpgtbl, VIRTIO0, VIRTIO0, PGSIZE, PTE_R | PTE_W);

    // PLIC
    kvmmap(kpgtbl, PLIC, PLIC, 0x4000000, PTE_R | PTE_W);

    // map kernel text exec
    kvmmap(kpgtbl, KERNBASE, KERNBASE, 0x80000000, PTE_R | PTE_W);

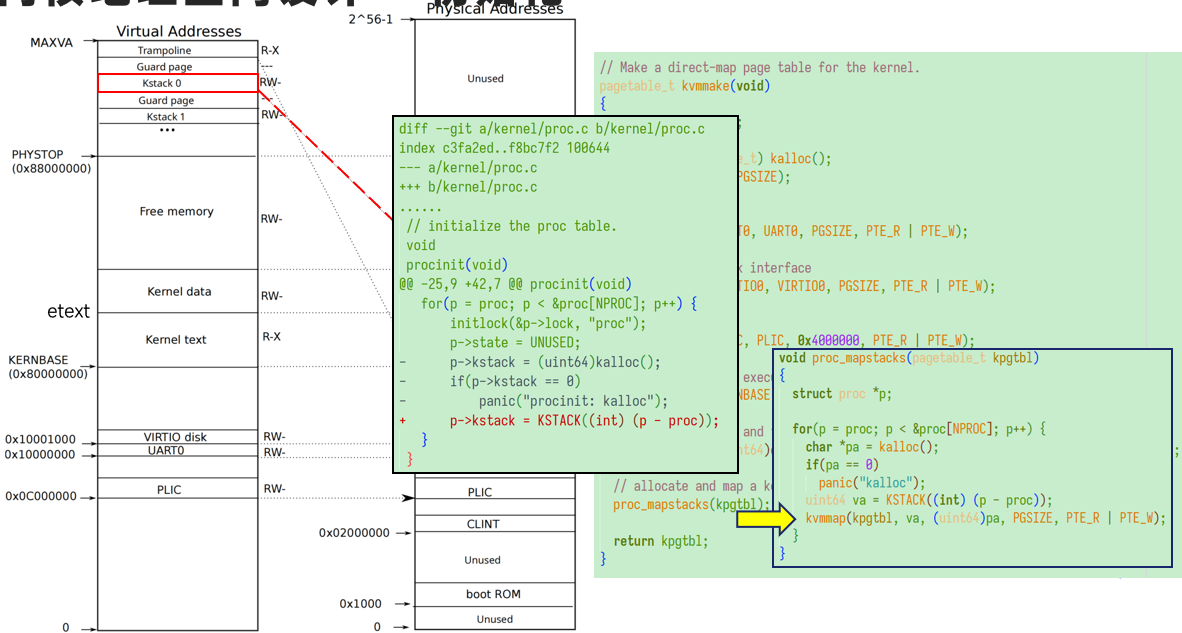
    // map kernel data and
    kvmmap(kpgtbl, (uint64)0, (uint64)0, PGSIZE, PTE_R | PTE_W);

    // allocate and map a k
    proc_mapstacks(kpgtbl);

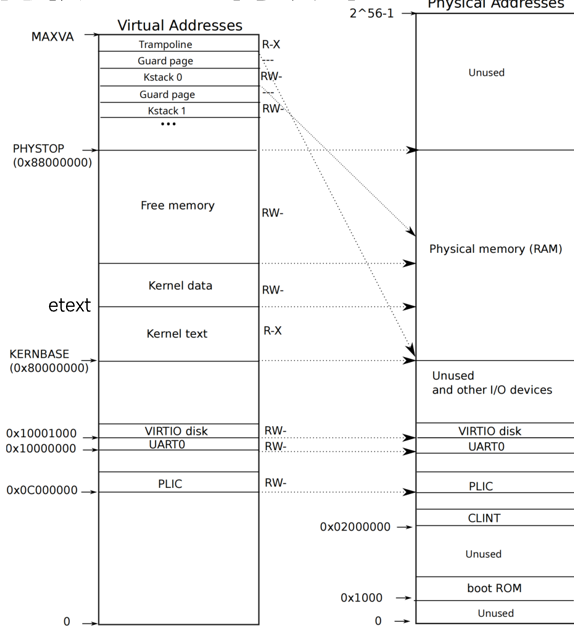
    return kpgtbl;
}

void proc_mapstacks(pagetable_t kpgtbl)
{
    struct proc *p;
    #define TRAMPOLINE (MAXVA - PGSIZE)
    for( // map kernel stacks beneath the trampoline,
        ch // each surrounded by invalid guard pages.
        if #define KSTACK(p) (TRAMPOLINE - ((p)+1)* 2*PGSIZE)
            uint64 va = KSTACK((int) (p - proc));
            kvmmap(kpgtbl, va, (uint64)pa, PGSIZE, PTE_R | PTE_W);
    }
}
```

# 内核地址空间设计 - 初始化



# 内核地址空间设计 - 初始化



```
// Make a direct-map page table for the kernel.
pagetable_t kvmmake(void)
{
    pagetable_t kpgtbl;

    kpgtbl = (pagetable_t) kalloc();
    memset(kpgtbl, 0, PGSIZE);

    // uart registers
    kvmmap(kpgtbl, UART0, UART0, PGSIZE, PTE_R | PTE_W);

    // virtio mmio disk interface
    kvmmap(kpgtbl, VIRTIO0, VIRTIO0, PGSIZE, PTE_R | PTE_W);

    // PLIC
    kvmmap(kpgtbl, PLIC, PLIC, 0x4000000, PTE_R | PTE_W);

    // map kernel text executable and read-only.
    kvmmap(kpgtbl, KERNBASE, KERNBASE, (uint64)etext-KERNBASE, PTE_R | PTE_X);

    // map kernel data and the physical RAM we'll make use of.
    kvmmap(kpgtbl, (uint64)etext, (uint64)etext, PHYSTOP-(uint64)etext, PTE_R | PTE_W);

    // allocate and map a kernel stack for each process.
    proc_mapstacks(kpgtbl);

    return kpgtbl;
}
```

# 内核地址空间设计 - 初级

- pagetable: page table
- va: virtual address
- sz: size
- pa: physical address
- perm: permission

```
int mappages(
    pagetable_t pagetable,
    uint64 va, uint64 size, uint64 pa, int perm)
{
    uint64 a, last;
    pte_t *pte;
    if((va % PGSIZE) != 0)
        panic("mappages: va not aligned");
    if((size % PGSIZE) != 0)
        panic("mappages: size not aligned");
    if(size == 0)
        panic("mappages: size");

    a = va;
    last = va + size - PGSIZE;
    for(;;){
        if((pte = walk(pagetable, a, 1)) == 0)
            return -1;
        if(*pte & PTE_V)
            panic("mappages: remap");
        *pte = PA2PTE(pa) | perm | PTE_V;
        if(a == last)
            break;
        a += PGSIZE;
        pa += PGSIZE;
    }
    return 0;
}
```

在指定页表 (pagetable) 中将虚拟地址 va 起始的一段大小为 size 的地址空间映射到物理地址 pa 处, 同时根据指定的 perm 设置这段地址空间的访问权限。

# 内核地址空间设计 - 初

```
int mappages(  
    pagetable_t pagetable,  
    uint64 va, uint64 size, uint64 pa, int perm)  
{  
    uint64 a, last;  
    pte_t *pte;  
    if((va % PGSIZE) != 0)  
        panic("mappages: va not aligned");  
    if((size % PGSIZE) != 0)  
        panic("mappages: size not aligned");  
    if(size == 0)  
        panic("mappages: size");  
  
    a = va;  
    last = va + size - PGSIZE;  
    for(;;){  
        if((pte = walk(pagetable, a, 1)) == 0)  
            return -1;  
        if(*pte & PTE_V)  
            panic("mappages: remap");  
        *pte = PA2PTE(pa) | perm | PTE_V;  
        if(a == last)  
            break;  
        a += PGSIZE;  
        pa += PGSIZE;  
    }  
    return 0;  
}
```

- pagetable: page table
- va: virtual address
- sz: size
- pa: physical address
- perm: permission

在指定页表 (pagetable) 中将虚拟地址 va 起始的一段大小为 size 的地址空间映射到物理地址 pa 处, 同时根据指定的 perm 设置这段地址空间的访问权限。

一些必要的检查:

- va 必须 4K 字节对齐
- size 必须 >=0 且必须是 4K 的整数倍



# 内核地址空间设计 - 初探

```
int mappages(  
    pagetable_t pagetable,  
    uint64 va, uint64 size, uint64 pa, int perm)  
{  
    uint64 a, last;  
    pte_t *pte;  
    if((va % PGSIZE) != 0)  
        panic("mappages: va not aligned");  
    if((size % PGSIZE) != 0)  
        panic("mappages: size not aligned");  
    if(size == 0)  
        panic("mappages: size");  
  
    a = va;  
    last = va + size - PGSIZE;  
    for(;;){  
        if((pte = walk(pagetable, a, 1)) == 0)  
            return -1;  
        if(*pte & PTE_V)  
            panic("mappages: remap");  
        *pte = PA2PTE(pa) | perm | PTE_V;  
        if(a == last)  
            break;  
        a += PGSIZE;  
        pa += PGSIZE;  
    }  
    return 0;  
}
```

- pagetable: page table
- va: virtual address
- sz: size
- pa: physical address
- perm: permission

在指定页表 (pagetable) 中将虚拟地址 va 起始的一段大小为 size 的地址空间映射到物理地址 pa 处, 同时根据指定的 perm 设置这段地址空间的访问权限。

一些必要的检查:

- va 必须 4K 字节对齐
- size 必须 >=0 且必须是 4K 的整数倍

在虚拟地址空间里, 在 [va, va + size] 范围中, 以 4KB 为单位, 逐项寻找每一项在页表中对应的 PTE, 如果找不到就创建新的 PTE。

找到的 PTE 需要确保没有被映射过, 因为不允许重复映射。如果一切正常, 则对找到 (或者新创建) 的 PTE 设置其值 (即建立映射关系)。

# 内核地址空间设计 - 初探

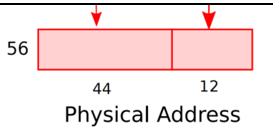
```
int mappages(  
    pagetable_t pagetable,  
    uint64 va, uint64 size, uint64 pa, int perm)  
{  
    uint64 a, last;  
    pte_t *pte;  
    if((va % PGSIZE) != 0)  
        panic("mappages: va not aligned");  
    if((size % PGSIZE) != 0)  
        panic("mappages: size not aligned");  
    if(size == 0)  
        panic("mappages: size");  
  
    a = va;  
    last = va + size - PGSIZE;  
    for(;;){  
        if((pte = walk(pagetable, a, 1)) == 0)  
            return -1;  
        if(*pte & PTE_V)  
            panic("mappages: remap");  
        *pte = PA2PTE(pa) | perm | PTE_V;  
        if(a == last)  
            break;  
        a += PGSIZE;  
        pa += PGSIZE;  
    }  
    return 0;  
}
```

- pagetable: page table
- va: virtual address
- sz: size
- pa: physical address
- perm: permission

在指定页表 (pagetable) 中将虚拟地址 va 起始的一段大小为 size 的地址空间映射到物理地址 pa 处, 同时根据指定的 perm 设置这段地址空间的访问权限。

一些必要的检查:

- va 必须 4K 字节对齐
- size 必须 >=0 且必须是 4K 的整数倍



在虚拟地址空间里, 在 [va, va + size) 范围内, 逐项寻找每一项在页表中的映射, 如果找到, 就创建新的 PTE。

```
// shift a physical address to the right place for a PTE.  
#define PA2PTE(pa) (((uint64)pa) >> 12) << 10
```

找到的 PTE 需要确保没有被映射过, 因为不允许重复映射。如果一切正常, 则对找到 (或者新创建) 的 PTE 设置其值 (即建立映射关系)。

# 内核地址空间设计 - 虚拟地址

```
int mappages(  
    pagetable_t pagetable,  
    uint64 va, uint64 size, uint64 pa, int perm)  
{  
    uint64 a, last;  
    pte_t *pte;  
    if((va % PGSIZE) != 0)  
        panic("mappages: va not aligned");  
    if((size % PGSIZE) != 0)  
        panic("mappages: size not aligned");  
    if(size == 0)  
        panic("mappages: size");  
  
    a = va;  
    last = va + size - PGSIZE;  
    for(;;){  
        if((pte = walk(pagetable, a, 1)) == 0)  
            return -1;  
        if(*pte & PTE_V)  
            panic("mappages: remap");  
        *pte = PA2PTE(pa) | perm | PTE_V;  
        if(a == last)  
            break;  
        a += PGSIZE;  
        pa += PGSIZE;  
    }  
    return 0;  
}
```

在找  
小  
定

一

•

•

• pagetable: page table

```
pte_t *walk(pagetable_t pagetable, uint64 va, int alloc)  
{  
    if(va >= MAXVA)  
        panic("walk");  
  
    for(int level = 2; level > 0; level--) {  
        pte_t *pte = &pagetable[PX(level, va)];  
        if(*pte & PTE_V) {  
            pagetable = (pagetable_t)PTE2PA(*pte);  
        } else {  
            if(!alloc || (pagetable = (pde_t*)kalloc()) == 0)  
                return 0;  
            memset(pagetable, 0, PGSIZE);  
            *pte = PA2PTE(pagetable) | PTE_V;  
        }  
    }  
    return &pagetable[PX(0, va)];  
}
```

在虚拟地址空间里，在 [va, va + size] 范围中，以 4KB 为单位，逐项寻找每一项在页表中对应的 PTE，如果找不到就创建新的 PTE。

找到的 PTE 需要确保没有被映射过，因为不允许重复映射。如果一切正常，则对找到（或者新创建）的 PTE 设置其值（即建立映射关系）。

# 内核地址空间设计 - 初

```
int mappages(  
    pagetable_t pagetable,  
    uint64 va, uint64 size, uint64 pa, int perm)  
{  
    uint64 a, last;  
    pte_t *pte;  
    if((va % PGSIZE) != 0)  
        panic("mappages: va not aligned");  
    if((size % PGSIZE) != 0)  
        panic("mappages: size not aligned");  
    if(size == 0)  
        panic("mappages: size");  
  
    a = va;  
    last = va + size - PGSIZE;  
    for(;;){  
        if((pte = walk(pagetable, a, 1)) == 0)  
            return -1;  
        if(*pte & PTE_V)  
            panic("mappages: remap");  
        *pte = PA2PTE(pa) | perm | PTE_V;  
        if(a == last)  
            break;  
        a += PGSIZE;  
        pa += PGSIZE;  
    }  
    return 0;  
}
```

• pagetable: page table

```
pte_t *walk(pagetable_t pagetable, uint64 va, int alloc)  
{  
    if(va >= MAXVA)  
        panic("walk");  
  
    for(int level = 2; level > 0; level--) {  
        pte_t *pte = &pagetable[PX(level, va)];  
        if(*pte & PTE_V) {  
            pagetable = (pagetable_t)PTE2PA(*pte);  
        } else {  
            if(!alloc || (pagetable = (pde_t*)kalloc()) == 0)  
                return 0;  
            memset(pagetable, 0, PGSIZE);  
            *pte = PA2PTE(pagetable) | PTE_V;  
        }  
    }  
    return &pagetable[PX(0, va)];  
}
```

在核  
小分  
定的

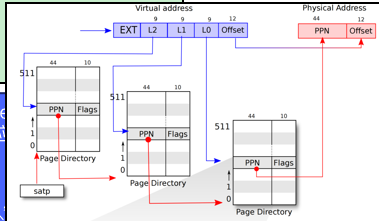
—

•

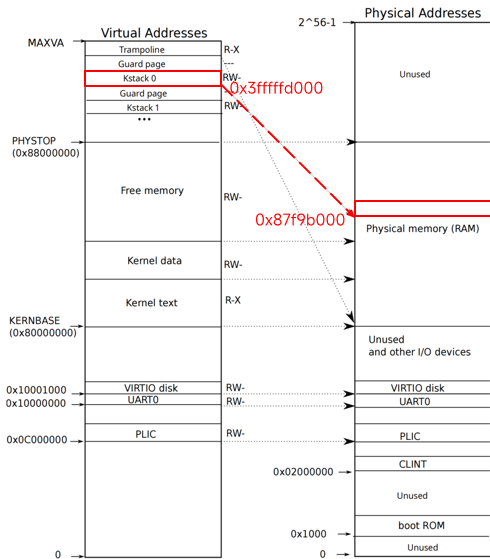
•

在虚拟地址空间里，在 [va, va + size) 单位，逐项寻找每一项在页表中对应，就创建新的 PTE。

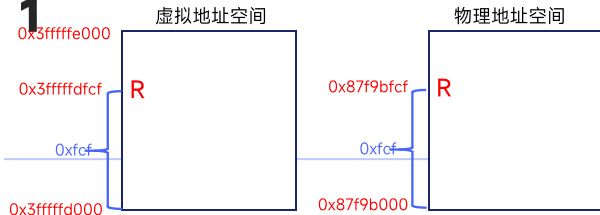
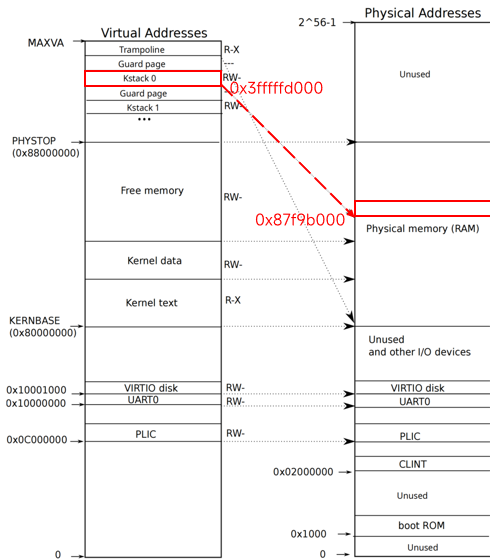
找到的 PTE 需要确保没有被映射过，如果一切正常，则对找到（或者新创建）的 PTE 设置其值（即建立映射关系）。



# 内核地址空间设计 - 实验 1



# 内核地址空间设计 - 实验 1

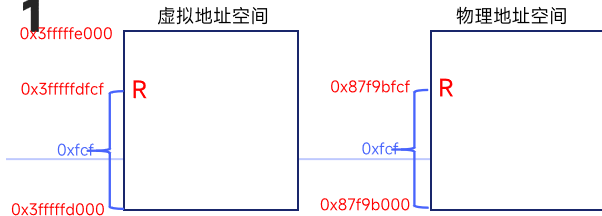


# 内核地址空间设计 - 实验 1

```
diff --git a/kernel/proc.c b/kernel/proc.c
index f8bc7f2..eaabc65 100644
--- a/kernel/proc.c
+++ b/kernel/proc.c
@@ -29,6 +29,8 @@ proc_mapstacks(pagetable_t kpgtbl)
    panic("kalloc");
    uint64 va = KSTACK((int) (p - proc));
    kvmmap(kpgtbl, va, (uint64)pa, PGSIZE, PTE_R | PTE_W);
+   if (p == proc)
+       printf("stack: va [%p] --> pa [%p]\n", (void *)va, pa);
}
```

```
diff --git a/kernel/user.c b/kernel/user.c
index e4ec41f..b84d49d 100644
--- a/kernel/user.c
+++ b/kernel/user.c
@@ -13,6 +13,8 @@ static void task_delay(volatile int count)

void user_task0(void)
{
+   volatile char c = 'R';
+   printf("%c @[%p]\n", c, &c);
    while (1) {
        task_delay(DELAY);
    }
```



xv6 kernel is booting

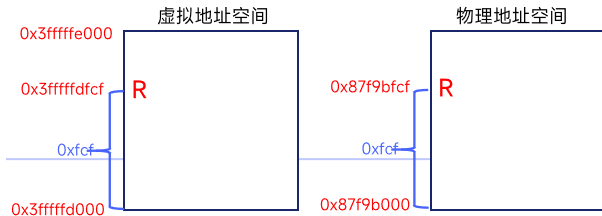
```
stack: va [0x00000003fffffd000] --> pa [0x0000000087f9b000]
hart 2 starting
hart 1 starting
$ R @[0x00000003fffffd0cf]
```

# 内核地址空间 - 实验 1

```
diff --git a/kernel/proc.c b/kernel/proc.c
index f8bc7f2..eaabc65 100644
--- a/kernel/proc.c
+++ b/kernel/proc.c
@@ -29,6 +29,8 @@ proc_mapstacks(pagetable_t kpgtbl)
    panic("kalloc");
    uint64 va = KSTACK((int) (p - proc));
    kvmmap(kpgtbl, va, (uint64)pa, PGSIZE, PTE_R | PTE_W);
+   if (p == proc)
+       printf("stack: va [%p] --> pa [%p]\n", (void *)va, pa);
}
```

```
diff --git a/kernel/user.c b/kernel/user.c
index e4ec41f..b84d49d 100644
--- a/kernel/user.c
+++ b/kernel/user.c
@@ -13,6 +13,8 @@ static void task_delay(volatile int count)

void user_task0(void)
{
+   volatile char c = 'R';
+   printf("%c @[%p]\n", c, &c);
    while (1) {
        task_delay(DELAY);
    }
```



```
xv6 kernel is booting
```

```
stack: va [0x0000003fffffd000] --> pa [0x0000000087f9b000]
hart 2 starting
hart 1 starting
$ R @[0x0000003fffffd000]
```

```
(gdb) b kernel/user.c:18
Breakpoint 1 at 0x8000195c: file kernel/user.c, line 18.
(gdb) c
Continuing.
[Switching to Thread 1.2]
```

```
Thread 2 hit Breakpoint 1, task_delay (count=1000) at kernel/user.c:8
8         count *= 50000;
(gdb) monitor xp /lcb 0x87f9bfcf
0000000087f9bfcf: 'R'
```



## 内核地址空间设计 - 实验 2

```
diff --git a/kernel/main.c b/kernel/main.c
index 7254bbc..0922622 100644
```

```
--- a/kernel/main.c
```

```
+++ b/kernel/main.c
```

```
@@ -18,10 +18,14 @@ main()
```

```
    printf("\n");
```

```
    kinit();          // physical page all
```

```
    kvminit();        // create kernel pag
```

```
+ *(int *)KERNBASE = 100;
```

```
+ printf("You should see this message here BEFORE turning on paging!\n");
```

```
    kvminithart();    // turn on paging
```

```
    procinit();       // process table
```

```
    trapinit();       // trap vectors
```

```
    trapinithart();   // install kernel trap vector
```

```
+ *(int *)KERNBASE = 100;
```

```
+ printf("You should NOT see this message here! Instead, you will see a page fault error message.\n");
```

```
    plicinit();        // set up interrupt controller
```

```
    plicinithart();    // ask PLIC for device interrupts
```

```
    userinit();        // first user process
```

```
// Make a direct-map page table for the kernel.
```

```
pagetable_t
```

```
kvmmake(void)
```

```
{
```

```
    // .....
```

```
    // map kernel text executable and read-only.
```

```
    kvmmap(kpgtbl, KERNBASE, KERNBASE, (uint64)etext-KERNBASE, PTE_R | PTE_X);
```

```
    // .....
```

```
}
```

## 内核地址空间设计 - 实验 2

```
diff --git a/kernel/main.c b/kernel/main.c
index 7254bbc..0922622 100644
--- a/kernel/main.c
+++ b/kernel/main.c
@@ -18,10 +18,14 @@ main()
    printf("\n");
    kinit();           // physical page allocation
    kvminit();         // create kernel page table
+   *(int *)KERNBASE = 100;
+   printf("You should see this message here BEFORE turning on paging!\n");
+   kvminithart();     // turn on paging
    procinit();        // process table
    trapinit();        // trap vectors
    trapinithart();    // install kernel trap vector
+   *(int *)KERNBASE = 100;
+   printf("You should NOT see this message here! Instead, you will see a page fault error message.\n");
    plicinit();        // set up interrupt controller
    plicinithart();    // ask PLIC for device IDs
    userinit();        // first user process
```

```
// Make a direct-map page table for the kernel.
```

```
pagetable_t
```

```
kvmmake(void)
```

```
{
```

```
    // .....
```

```
    // map kernel text executable and read-only.
```

```
    kvmmap(kpgtbl, KERNBASE, KERNBASE, (uint64)etext-KERNBASE, PTE_R | PTE_X);
```

```
    // .....
```

```
}
```

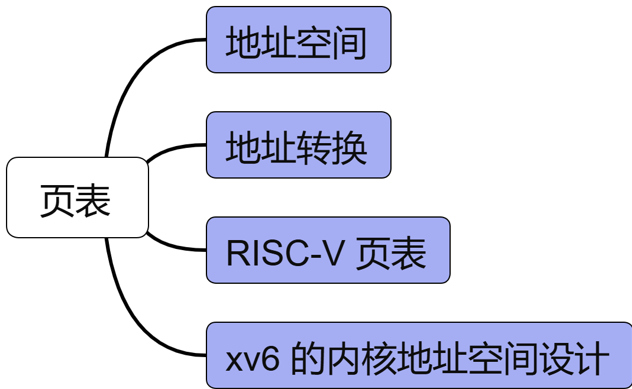
```
xv6 kernel is booting
```

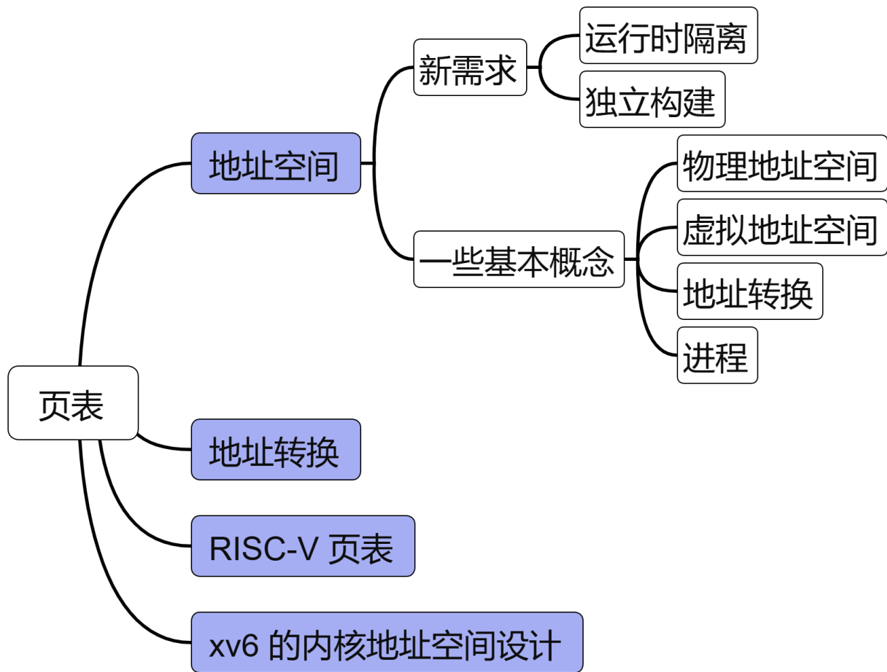
```
You should see this message here BEFORE turning on paging!
scause=0xf sepc=0x80000e9a stval=0x80000000
panic: kerneltrap
```

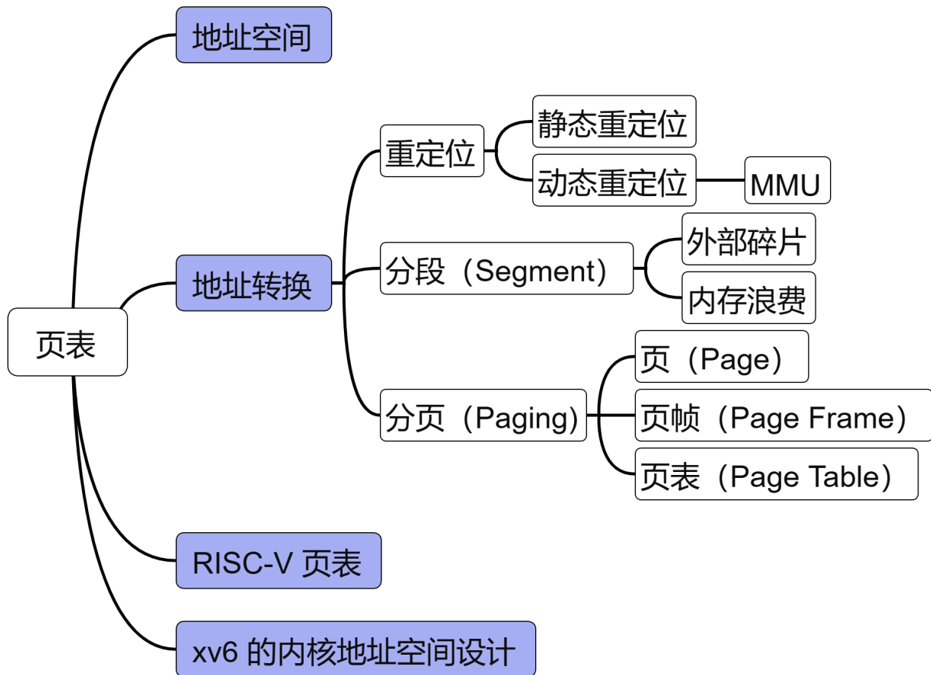


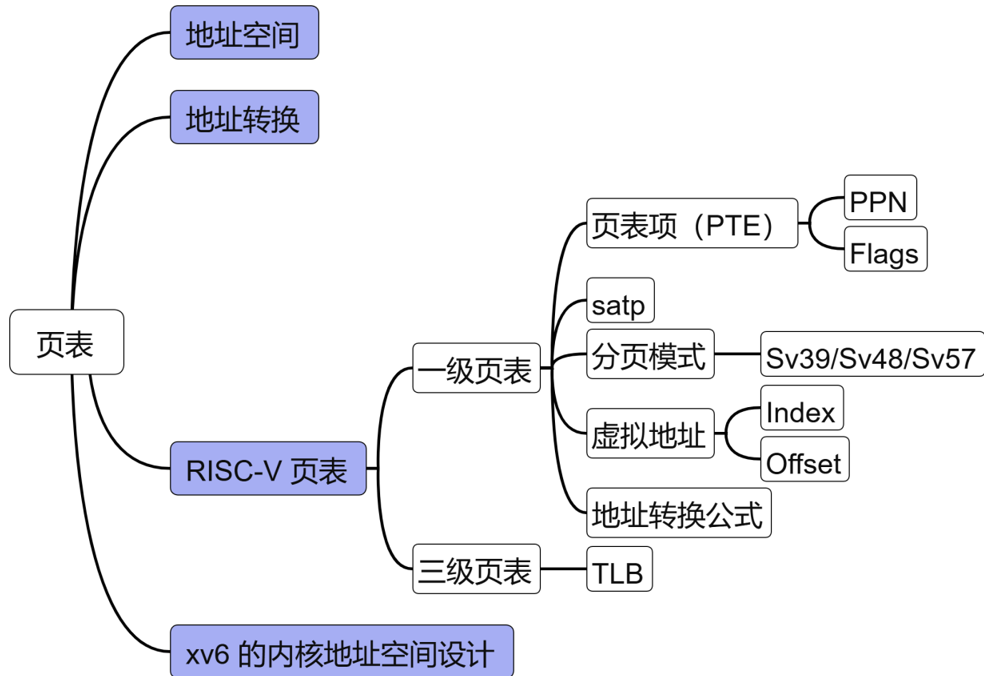
# 本章总结

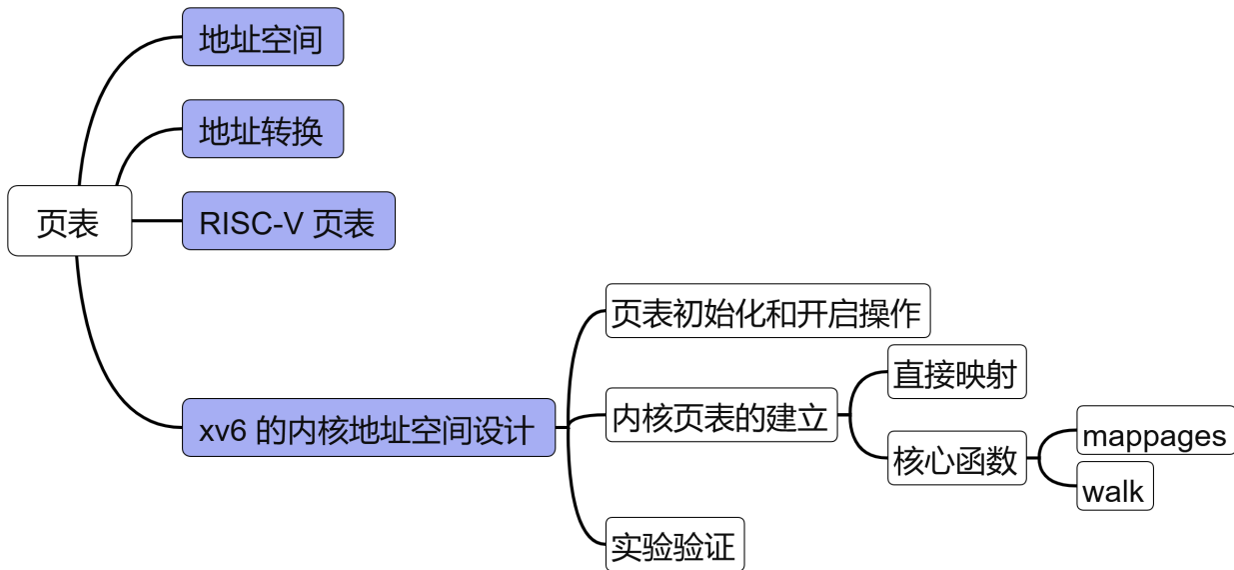
---













# 谢谢

